CS365- Winter 2021 Notes

Models of Computation

Jane Shi

Computer Science
University of Waterloo
April, 2021

Contents

1 Week 1 **4**
- 1.1 Lesson 1. Alphabets, Strings, and Languages . . . 4
- 1.2 Lesson 2. String Operations . . . 5
- 1.3 Lesson 3. Language Operations . . . 9
- 1.4 Lesson 4. Countability and Uncountability . . . 11
  - 1.4.1 The cardinality of sets . . . 11
  - 1.4.2 Finite and countable sets . . . 12

2 Week 2 **15**
- 2.1 Lesson 5. Deterministic Finite Automate (DFAs) . . . 15
  - 2.1.1 Definition of DFAs . . . 15
  - 2.1.2 Languages of DFAs . . . 16
  - 2.1.3 Regular Languages . . . 17
- 2.2 Lesson 6. Nondeterministic Finite Automatas . . . 18
  - 2.2.1 The languages of NFAs . . . 19
  - 2.2.2 Regular languages and NFAs . . . 20
- 2.3 Lesson 7. Rabin Scott theorem . . . 21
- 2.4 Lesson 8. Regular expressions . . . 22
  - 2.4.1 Languages of Regular Expressions . . . 23
  - 2.4.2 Bonus question: short regular expressions . . . 23

3 Week 3 **25**
- 3.1 Lesson 9. Kleene's theorem . . . 25
- 3.2 Lesson 10. Pumping Lemma . . . 27
- 3.3 Lesson 11. Non-Regular languages . . . 29
  - 3.3.1 Slight variants of this . . . 30
  - 3.3.2 A non-regular unary language . . . 30
  - 3.3.3 Determining Regularity . . . 31
  - 3.3.4 Bonus: limitations of the pumping lemma . . . 31
- 3.4 Lesson 12. Properties of Regular Languages . . . 32
  - 3.4.1 Closure under language operations . . . 32
  - 3.4.2 Topological separation . . . 34
  - 3.4.3 Bonus: Testing Emptiness and Equivalences . . . 34

4 Week 4 **35**
- 4.1 Lesson 13. Pushdown Automata . . . 35
  - 4.1.1 Description of PDAs . . . 35
  - 4.1.2 Definitions of PDAs . . . 36
  - 4.1.3 Languages of PDAs . . . 37
  - 4.1.4 Regular Languages and PDAs . . . 38
- 4.2 Lesson 14. Context Free Grammars . . . 38
  - 4.2.1 Definitions of CFGs . . . 39
  - 4.2.2 Context Free language . . . 40
- 4.3 Lesson 15. Equivalence of CFGs and PDAs . . . 40
- 4.4 Lesson 16. Chomsky Normal Form . . . 45
  - 4.4.1 Right regular grammar, left regular grammar, and linear grammar . . . 47

5 Week 5 Materials **50**
5.1 Lesson 17. Pumping Lemmas for CFLs . . . 50
5.1.1 Finite and Infinite CFLs . . . 50
5.1.2 The pumping lemma . . . 51
5.1.3 Using the pumping lemma . . . 52
5.2 Lesson 18. . . . 53
5.2.1 Properties of Context-Free-Languages . . . 53
5.2.2 Deterministic CFLs . . . 55

6 Week 6 Materials **58**
6.1 Lesson 19 . . . 58
6.1.1 Decidable languages . . . 61
6.1.2 Recognizable languages . . . 61
6.2 Lesson 20. Church Turing Thesis . . . 61
6.2.1 Turing Machines with Registers . . . 62
6.2.2 Subroutine Turing Machines . . . 62
6.2.3 Multi-tape Turing Machines . . . 63
6.3 Lesson 21. Universal Turing Machines . . . 64
6.3.1 Turing Completeness . . . 66
6.4 Lesson 22. Non-deterministic Turing Machines . . . 67

7 Week 7 **71**
7.1 Lesson 23. Decidable Languages . . . 71
7.1.1 Clique . . . 72
7.1.2 Accepting DFAs . . . 72
7.1.3 Bonus: Closure Properties . . . 73
7.2 Lesson 24. Undecidable Languages . . . 73
7.2.1 Existence . . . 73
7.2.2 First Undecidable Language . . . 74
7.3 Lesson 25. More Undecidable languages . . . 75
7.3.1 Halting language . . . 75
7.3.2 Emptiness . . . 76
7.3.3 Equality . . . 76
7.4 Lesson 26. Rice's Theorem . . . 77
7.4.1 Statement of Rices Theorem . . . 78

8 Week 8 **79**
8.1 Lesson 27. Recognizability . . . 79
8.1.1 Decidability and Recognizability . . . 79
8.2 Lesson 28 TIME class . . . 80
8.2.1 First examples . . . 81
8.2.2 The Linear Speedup Theorem . . . 82

9 Week 9 **84**
9.1 Lesson 29. Time Hierarchy Theorem . . . 84
9.1.1 Weak Time Hierarchy Theorem . . . 84
9.1.2 Time Hierarchy Theorem . . . 84
9.1.3 Necessity of Time Constructibility . . . 86
9.2 Lesson 30. P . . . 86
9.2.1 Motivations for studying the class P . . . 86
9.2.2 First Observations . . . 87

10 Week 10 **89**
10.1 Lesson 31. NP . . . 89
10.1.1 Definitions . . . 89
10.1.2 Polynomial Time Verifiers . . . 91
10.1.3 P vs NP . . . 92
10.2 Lesson 32. NP-Completeness . . . 92
10.2.1 Polynomial-Time reductions . . . 92
10.2.2 NP-hardness and completeness . . . 93
10.2.3 Bonus- Existence of an NP-complete language . . . 93
10.3 Lesson 33. Satisfiability . . . 93
10.3.1 Boolean formulas and SAT . . . 94
10.3.2 Tebleaux . . . 95
10.3.3 Tableaux and SAT . . . 96
10.4 Lesson 34. Cook-Levin Theorem . . . 96
10.4.1 Proof overview . . . 96
10.4.2 Cell constraints . . . 96
10.4.3 Initial and Final Constraints . . . 97
10.4.4 Valid Tableau Constraints . . . 98
10.4.5 Putting it together . . . 99

11 Week 11 **100**
11.1 Lesson 35 Beyond P and NP . . . 100
11.1.1 coNP . . . 100
11.1.2 $NP \cap coNP$ . . . 100
11.1.3 Ladner's Theorem . . . 101
11.1.4 Polynomial Hierarchy . . . 102
11.2 Lesson 36. SPACE complexity classes . . . 103
11.2.1 Time and space . . . 104
11.3 Lesson 37. PSPACE . . . 104
11.3.1 Savitch's Theorem . . . 105
11.3.2 Nondeterministic Space complexity . . . 105
11.3.3 The Derivation Language . . . 105
11.3.4 Proof of Savitch's Theorem . . . 106
11.4 Lesson 38. Logarithmic Space Complexity . . . 106
11.4.1 Basic definitions . . . 106
11.4.2 Space complexity classes . . . 107
11.4.3 L and NL . . . 107

12 Week 12 **109**
12.1 Lesson 39. NL Completeness . . . 109
12.1.1 NL completeness . . . 110
12.1.2 Connectivity . . . 110
12.2 Lesson 40. Immerman-Szelepcsenyi Theorem . . . 111
12.3 Lesson 41 . . . 112
12.3.1 PSPACE-completeness . . . 112
12.3.2 Small space complexity classes . . . 113
12.3.3 Time and Space . . . 114

1 Week 1

1.1 Lesson 1. Alphabets, Strings, and Languages

Note (Mathematical objects):
Note that for different types mathematical theory, we study different mathematical objects. For example, we study functions in functional analysis, vector spaces for linear algebra, and homotopy in topology. Similarly, the mathematical objects we study in theoretical computer science are: alphabets, strings, and languages. These simple concepts are good for rigorous mathematical analysis.

Definition (Alphabet):
An alphabet is a nonempty finite set.

Examples of an alphabet includes $\Sigma_a = \{1\}, \Sigma_b = \{0\}, \Sigma_{abc} = \{a, b, c, \ldots z\}$, $\Sigma_c = \{1, \ldots 2^{400}\}$. However, $\mathbb{Z}$ is not an alphabet.
Question. Does the set of all 301100 words in the Oxford English dictionary form an alphabet?
Answer: Yes. This is because the all of words in Oxford English dictionary forms a set, and that this set is nonempty (math is one of them), and finite (the set has 30110 words.)

Definition (Symbols):
Symbols are elements of an alphabet.

Note that for an alphabet with k elements, $0 < k < \infty, k \in \mathbb{Z}$, we could represent its k elements with any arbitrary symbols. However, the exact symbols we choose does not matter. So we refer to the alphabet with k symbols $\Sigma_k = \{0, 1, \ldots k-1\}$. We can also represent the unary alphabet by $\Sigma_1 = \{0\}$, the binary alphabet by $\Sigma_2 = \{0, 1\}$.

Definition (Strings):
A string over the alphabet Σ is a finite sequence of symbols from Σ.

Examples include the empty string (of any alphabet), the text content of a book (when the alphabet is all the ascii characters), and $9*5+6-2$ (with the alphabet consists of $1-9$, and $+, -, *, /$), and ω (the alphabet is ω). An non-example would be an infinitely long line 11111..... when the alphabet is Σ_2.

Definition (Length, empty string, and representing the ith component):

- The length of a string is the number of symbols in x, which is denoted by $|x|$.
- The empty string is the sequence that has length 0, which is unique. We denote empty string by ϵ.
- Lastly, for any nonempty string x such that $|x| = n$, we can write $x = (x_1, \ldots x_n)$. This way, given $i \in \{1, \ldots, n\}$, we see that x_i is the ith symbol in the string x.

List of questions

- Q1. Describe how each of the following types of objects can be encoded as strings over Σ_2.
  - Integers: binary representation.
  - Rational numbers: for a/b, we can use 1^a0^b.
  - Text files: turn each character into unicode, then turn each unicode character into binary. Finally, concatenate the binary representation of the unicode characters.
  - Graphs: No time to answer details. Consider the adjacency matrix.
  - Finite set of strings: Answer omitted.
- Q2. Describe a type of mathematical object that cannot be encoded using strings over Σ_2.

  A: function $\mathbb{R} \to \mathbb{R}$.
- Q3. Describe how for every alphabet Σ, there is a way to encode strings over Σ as strings over Σ_1.

  A: For Σ_2: We can assume the alphabet is $\{1, 2, \ldots n\}$. Then, a string can be encoded into $abcde = 1^a01^b01^c01^d01^e0$, so on.

  But for Σ_1, when alphabet has size n, the string $abcde$ can be encoded in $2^{n^5a} + 2^{n^4b} + 2^{n^3c} + 2^{n^2d} + 2^{ne}$ copies of 1.

Note that an encoding is a map from the objects to the set of strings, such that is it an injection.

Definition (Languages):
A language over the alphabet Σ is a set of strings over Σ.

Note that a language does not need to be a finite set. However, is a language over Σ countable? (Yes.)
Example of languages over Σ_2 are : $\varnothing$, $\{1, 11, 111, \ldots\}$, $\{0, 1, 11, 101\}$.

Definition (Decision problem):
A decision problem is a computational problem such that we intend the algorithm to output True or False. $\{2, 3, 5, 7, 11, \ldots\}$ is a language over Σ_{10} such that it correspond to the problem "is the number prime"?
What are some other example of languages that represent natural decision problems? (palindromes)

Definition (Finite language, empty language, etc):

- A language is finite if it has a finite number of strings.
- Cardinality of a finite language L is denoted $|L|$, and denotes the number of strings that it contained.
- The empty language is denoted by $\varnothing$, which is the unique language that contains no strings.
- The language that contains every string over Σ is Σ^* and the language that contains strings of length exactly n over Σ is denoted by Σ^n.

1.2 Lesson 2. String Operations

After learning the definitions of alphabets, strings, and languages, our next step is to learn how different mathematical operations will affect those mathematical objects.

Definition (Concatenation):
Suppose there are two strings $x, y \in \Sigma^*$ with lengths $|x| = k, |y| = \ell$, then the concatenation of x and y is the string

$$xy = (x_1, \ldots, x_k, y_1, \ldots, y_k.)$$

Note:
For example, if $x = \epsilon, y = \epsilon$, then $xy = \epsilon$. If $x = 101101$ and $y = 101$ then $xy = 101101101$.
Note that concatenation is associative: $(xy)z = x(yz)$ is true for all x, y, z. Therefore, we just omit the parenthesis and write $x_1 x_2 \ldots x_n$ for the concatenation of n strings.

Proof: To prove concatenation is associative: Suppose $x = x_1 \ldots x_a$, $y = y_1 \ldots y_b$, and $z = z_1 \ldots z_c$, then $xy = x_1 \ldots x_a y_1 \ldots y_b$ and $(xy)z = x_1 \ldots x_a y_1 \ldots y_b z_1 \ldots z_c$. Also $yz = y_1 \ldots y_b z_1 \ldots z_c$ and $x(yz) = x_1 \ldots x_a y_1 \ldots y_b z_1 \ldots z_c$. Hence we have shown $(xy)z = x(yz)$, so the string concatenation is commutative.

$\square$

Note:
Note that string concatenation is not commutative. One counter example is that $x = 0, y = 1$, then $xy = 01, yx = 10$, which are not equal.

Definition (Squares, Powers, and Primitive Strings):
Note that being able to square and being able to raise to a power are natural operations for numbers. This operation is also natural for strings. So we can defined analogues operations based on strings concatenation.

- The square of $x \in \Sigma^*$ is the string $x^2 = xx$.
- Generally, for a power $n \geq 1$, then the nth power of the string x is the string $x^n = \underbrace{x \ldots x}_{n \text{copies}}$. In other words, the nth power of the string x is obtained by concatenating n copies of x.
- It is also natural to define the 0th power of the string x as $x^0 = \epsilon$. Note that $\epsilon^0 = \epsilon$.

Note:

- Concatenation and string power give us freedom to write very long strings in shorter amount of spaces. For example, $1101111111100011 = 1^2 01^8 00011$. Also $1(101)^4 1 = 11011011011011$. Note that parenthesis should **not** be part of the underlying alphabet in order to prevent confusion.
- When does $x^2 = x$? We would have $2|x| = |x^2| = |x|$. Since string lengths are finite, we must have$|x| = 0$. Therefore, $x^2 = x$ if and only if $x = \epsilon$.
- Does there exists $x \neq y$ such that $x^n = y^n$ for some $n \geq 1$? The answer is no. We know that $n|y| = n|y^n||x^n| = n|x|$, so x, y have same length. Then, matching alphabet-by-alphabet to result in equality.

Definition (Primitive string):
A string $x \in \Sigma^*$ is *primitive* if $x \neq y^n$ for any $y \in \Sigma^*$ and any $n \geq 2$.

For example, the string $101, 1001, 100001$ are not primitive, but the string 2020 is primitive because $2020 = (20)^2$. Note that ϵ is not primitive because $\epsilon \in \Sigma^*$ and $\epsilon = \epsilon^2$.
This situation is similar to how 1 is not a prime.
Q: is the string 010100 primitive? Yes. Is the string 00100010 primitive? No. This is because $00100010 = (0010)^2$.
Question: Is there way to check primitive?
To check if a string is primitive, we need to check it at least number of prime factors of length time $O(\phi(|x|))$ time.

Definition (Substrings and Subsequences):
For any two strings $x, y \in \Sigma^*$, we have the following definition:

- x is a prefix of y if there exists a string $z \in \Sigma^*$ such that $y = xz$.
- x is a suffix of y if there exists a string $z \in \Sigma^*$ such that $y = zx$.
- x is a substring of y if there exists $w, z \in \Sigma^*$ such that $y = wxz$.

Remark: This is a definition we often encounter!

Note:

- For example, if $y = 000011110000$, we have $x_1 = 0000$ is both a prefix, a suffix, and a substring. Note that $x_2 = 011110$ is a substring, but not a prefix nor a suffix. Also, note that ϵ is a prefix, suffix, substring of all strings. In addition, prefix, suffix are stronger definition than substring.
- Note that for a string x, it is simultaneously a prefix, suffix, and a substring of itself.

Questions:

- For which strings $x \in \Sigma_2^*$, there is a string $y \in \Sigma_2^*$ of length $|x| < |y| < 2|x|$ such that x is both a prefix and a suffix of y?

Answer: If x is both a prefix and a suffix of y, we know that y is too long to be just x, too short to be x^2, then there must be some "overlap". An overlap occurs if and only if there is a string z of length $1 \leq |z| < |x|$ such that z is simultaneously a prefix and a suffix of x.

- Are there any distinct strings $x \neq y$ over some alphabet Σ such that x is a substring of y and Y is a substring of x?

  Answer: no. Note that a is a substring of b implies $|a| \leq |b|$. Form this, we conclude $|x| = |y|$. Then the only way to be substrings is that they are identical.

Definition (Subsequence):
Let $x, y \in \Sigma^*$. If there exists indices $j_1 < j_2 < \ldots < j_{|x|}$ such that $x_i = y_{j_i}$, for each $i = 1, 2, \ldots |x|$, then we say x is a subsequence of y.

Note:
For example, $x = 101$ is a subsequence of $y = 111000111$, but $z = 010$ isn't. Note that substring is a stronger definition that subsequence. This reminds me a lot of wildcard matching.

Q. What is the length of the shortest string over Σ_2 that contains all 16 possible subsequences of length 4?
Answer: 10101010 works. This is the shortest because it cannot contain less than $4 + 4$ characters. This works because we can obtain every possible sequence from this string.

Definition (Reversals and Palindromes):

- Let x be a string such that $|x| = n$. Then the reversal of x is:

$$x^R = (x_n, x_{n-1}, \ldots, x_2, x_1)$$

- The string x is a palindrome if $x = x^R$.

Note:
For example, 10101 is palindromes, so is $1^n 0^n 1^n$ for $n \in \mathbb{N}$. For example in Σ_{abc}, kayak is a palindrome.

Q: how many palindromes of length n over Σ are there?
A: The answer is $|\Sigma|^{\lfloor \frac{n}{2} \rfloor}$.
Q: How many distinct palindromes occur as substring of the string 0100100?
A: Enumeration yields: $\epsilon, 0, 1, 00, 11, 101, 010, 111, 000, 0000, 1001, 0110, 00000, 00100, 01010, 010010$. This is done by brute force, I think there is an algorithmic solution.
Q: What is the length of the longest string over Σ_2 that does not contain any palindromes of length at least 4 as a substring?
A: this question definitely needs programming.

1.3 Lesson 3. Language Operations

Note:
Since a language is a set of strings over an alphabet Σ, it is natural for us to consider set operation on languages. We also propose multiple langauge operations.

Definition (Set operations):
Given two langauges A and B over Σ,

- Union: $A \cup B = \{x \in \Sigma^* : x \in A \text{ or } x \in B\}$
- Intersection: $A \cap B = \{x \in \Sigma^* : x \in A \text{ and } x \in B\}$
- Set difference: $A \setminus B = \{x \in \Sigma^* : x \in A \text{ and } x \notin B\}$
- Symmetric difference: $A \Delta B = (A \setminus B) \cup (B \setminus A)$
- Complement: $\overline{A} = \Sigma^* \setminus A$

List of questions

- Let $A = \{2, 3, 5, 7, 11, 13, \ldots\}$ and $B = \{0, 1, 4, 6, 8, 9, 10, 12, \ldots\}$ be languages over $\Sigma = \{0, \ldots, 9\}$ that denote the set of primes and non-primes respectively. Is it true that $A \cup B = \Sigma^*$?

  A: no. Consider 000000001. It is in neither of those languages, but it is a string over this alphabet.

- For which finite sets A and B does $|A \cup B| = |A|$?

  A:When $B \subseteq A$.

- For which finite sets A and B does $|A \cap B| = |A|$?

  A: when $A \subseteq B$.

- Is there a language A such that both A and $\overline{A}$ are infinite? Yes. For example primes (set A in the first pointer) and $\overline{A}$, all the other strings.

Definition (Concatenation of languages):
Let A, B be two languages over the alphabet Σ. Then the concatenation of the two languages is

$$AB = \{xy \mid x \in A \text{ and } y \in B\}$$

This language is obtained by concatenating every string in A with every string in B.

For example, if $A = \{101, 010\}$ $B = \{00, 11\}$ then $AB = \{10100, 10111, 01000, 01011\}$.
List of questions

- For which finite sets A and B does $|AB| = |A|$?

  A: When B only has one element.

- Does $A\{\epsilon\} = A\varnothing$?

  A: No. This is because $A\{\epsilon\} = \{xy : x \in A, y = \epsilon\} = \{x \in A\} = A$. However, $A\varnothing$ is $\{xy : x \in A, y \in \varnothing\}$ which is an empty set.

- When does $AB = BA$?

  A: omitted.

Definition (Squares and powers of languages):

- The square of a language L over Σ is the language $L^2 = LL$ obtained by concatenation of L and L.
- For integer $n \geq 1$, the nth power of L is the language obtained by concatenating L to itself n times. Note that language concatenation is associative and not commutative.

$$L^n = \{x^{(1)}x^{(2)} \dots x^{(n)}, x^{(1)}, x^{(2)}, \dots, x^{(n)} \in L\}$$

- The $0th$ power of L is defined as $L^0 = \{\epsilon\}$.

Note:
Note that the $0th$ power of language also apply to the empty language ϵ. That is, $\varnothing^0 = \{\epsilon\}$.

Question: True or false?

- $A^2 = \{x^2 : x \in A\}$

  False. Suppose $A = \{0, 1\}$, then $A^2 = \{00, 01, 10, 11\}$ yet $\{x^2 : x \in A\} = \{00, 11\}$.
- For each $n \geq 1, A^n = AA^{n-1}$.

  True, this can be done by induction.
- $A^2 = A$ if and only if $A = \{\epsilon\}$.

  False. $A = \varnothing$ is also a solution.

Definition (Star operation):
The following is an operation that uses concatenation of language yet it is not defined for strings. This operation is only available to languages and is the star operation.
For any language L, the star operation on L is the language

$$L^* = \bigcup_{n \geq 0} L^n$$

Note:
The star operation is called the Kleene star operation, which is named after Stephen Kleene.

<u>List of propositions:</u>

1. A^* is always a non-empty language.
2. $A^* = \Sigma^*$ if and only if A includes every string of length 1 in Σ.
3. A^* is finite if and only if $A = \varnothing$ or $A = \{\epsilon\}$.
4. $A^*A^* = A^*$

5. $(A^*)^* = A^*$

Proof is omitted.

1.4 Lesson 4. Countability and Uncountability

In this section, we study the notion of uncountability, and we revisit proof techniques known as diagonalization method.

1.4.1 The cardinality of sets

Definition (Cardinalities):
Given two sets S and T. The cardinalities of S and T, denoted by $|S|, |T|$ respectively, satisfy the inequality that $|S| \leq |T|$ if and only if there is a one-to-one mapping from the elements of S to the elements of T.

Note:

- Note that $|S| \leq |T|$, if and only if there is an injection $\phi : S \to T$.
- We also note that $|S| \leq |T|$ if and only if there is a surjection $\psi : T \to S$.
- We define the "greater or equal" as: $S \leq T$ if and only if $|T| \leq |S|$.

Definition (Same cardinality):
Two sets S, T have the same cardinality, denoted $|S| = |T|$, if and only if $|S| \leq |T|$ and $|T| \leq |S|$.

Note:
Note that $|S| = |T|$ if and only if there exists a bijection between S and T. The proof of this is rather difficult. It is the **Cantor-Schroder-Bernstein theorem.**

Question: Show that the set cardinality satisfy the following properties:

- Transitivity: For every sets, S, T, U, if $|S| \leq |T|$ and $|T| \leq |U|$, then we have $|S| \leq |U|$.

  Answer: Let $\phi : S \to T$, $\psi : T \to U$ be injections. Then $\psi \circ \phi \mid_S$ is an injection from S to U.
- Reflexivity: For every set S, $|S| \leq |S|$.

  Answer: using the identity function.

Note:
The solution to the above problem shows that we can define a partial order using cardinality. Assuming axiom of choce, we will be able to define a total order.

1.4.2 Finite and countable sets

Definition:

- The set of natural numbers is $\mathbb{N} = \{1, 2, 3 \ldots\}$
- A nonempty set S is finite if $|S| = |\{1, 2, \ldots n\}|$ for some $n \in \mathbb{N}$. When this is true, we write $|S| = n$, and say that S has the cardinality $|S| = n$.
- The empty set $\varnothing$ has cardinality $|\varnothing| = 0$ and it is finite.
- The cardinality of $\mathbb{N}$ is $|\mathbb{N}| = \aleph_0$.
- The set S is countable if and only if

$$|S| \leq |\mathbb{N}| = \aleph_0$$

Note:

- A countable set can be finite or infinte.

Prove the following sets are countable

1. $\mathbb{Z} = \{\ldots, -3, -2, -1, 0, 1, 2, 3, \ldots\}$
2. $2\mathbb{N} = \{2, 4, 6, 8, 10, \ldots\}$
3. $\mathbb{Q}^+ = \left\{\frac{m}{n} : m, n \in \mathbb{N}\right\}$
4. $\{1, 2, \ldots 100\}$

Proof is omitted.

Proposition:
For any alphabet Σ, the set Σ^* of strings over Σ is countable.

Proof: To show $|\Sigma^*| \leq |\mathbb{N}|$, it suffices to show that there exists an injection $\Sigma^* \to \mathbb{N}$. Let $k = |\Sigma|$. Without loss of generality, each element of the alphabet Σ can be represented by one of $\{1, 2, 3, \ldots k\}$.
We first consider non-empty strings. Note that $x = \{x_n, x_{n-1}, \ldots, x_1, x_0\}$ can be seen as a base $k+1$ representation of a number, as it takes digits in the set $\{1, 2, 3, \ldots k\}$. This string representation of numbers is unique because there will not be any 0 as prefix (that is, situations like $01234 = 1234$ won't happen because no alphabet is mapped to 0). In particular, $\{1, \ldots k\}^n$ for $n \in \mathbb{N}$ represents numbers in base $k+1$ that does not have any 0 digits.

Then, consider the injective map $\phi : \Sigma^* \to \mathbb{N}$ where for $x = \{x_n, x_{n-1}, \ldots, x_1, x_0\}$

$$\phi(x) = 1 + \sum_{i=0}^{n} (k+1)^i * x_i = 1 + (\text{consider } x \text{ as a base } k+1 \text{ representation, and convert it to decimal})$$

We define $\phi(\epsilon) = 1$, because there does not exist any nonempty strings that are mapped to 1, and because $\mathbb{N}$ starts with 1.
This mapping indeed is an injection, because base $k+1$ to base 10 conversion is injective.

$\square$

Uncountable sets

Definition:

- A set is uncountable if it is not countable.
- For any set S, the power set $\mathcal{P}(S)$ is the set of all subsets of S.

Note:
One notion we will use often throughout the course is that the set of all languages over any alphabet is uncountable.

Theorem (Cantor's theorem):
For any infinite countable set S, the power set $\mathcal{P}(S)$ is uncountable.

Proof:
Assume towards a contradiction such that $P(S)$ is countable. Then, there exists an surjection $\phi : S \to \mathcal{P}(S)$. We consider a $\mathbb{N} \times \mathbb{N}$ table. The way we construct this table is that we first realize that $|S| = |\mathbb{N}|$, so we have a bijection between S and $\mathbb{N}$, such that we can label elements of S as $S_1, S_2, \ldots$. We label the rows of the chart by $S_1, S_2, \ldots$. Then, we label the columns of this chart by $\phi(S_1), \phi(S_2), \phi(S_3), \ldots$, which are elements of the powerset. We then fill out the table, such that the $(S_i, \phi(S_i))$ entry is 1 if $S_i \in \phi(S_i)$ and 0 otherwise. The following is an example of the table construction if we have $\phi(S_1) = \varnothing$, $\phi(S_2) = \{S_1\}$, $\phi(S_3) = \{S_2\}$, $\phi(S_4) = \{S_1, S_1\}$, and so on.

$S \setminus P(S)$	$\varnothing$	$\{S_1\}$	$\{S_2\}$	$\{S_1, S_2\}$	$\ldots$	$\ldots$
S_1	0	1	0	1	$\ldots$	$\ldots$
S_2	0	0	1	1	$\ldots$	$\ldots$
S_3	0	0	0	0	$\ldots$	$\ldots$
$\ldots$	$\ldots$	$\ldots$	$\ldots$	$\ldots$	$\ldots$	$\ldots$

Now, we consider a set $K \in \mathcal{P}(\mathcal{S})$. For $S_i, 1 \leq i$, we let $S_i \in K$ if and only if the entry of $(S_i, \phi(S_i)) = 0$. Clearly, K is a subset of S because K consists of elements from S. Also, by the surjectivity of ϕ, we know that there exists some $j \in \mathbb{N}$ such that $\phi(S_j) = K$. Now, there are two possible situations:

- $S_j \in K$. Then the $(S_j, \phi(S_j))$ entry is 1. Yet by construction, $S_i \notin K$.
- $S_j \notin K$. Then the $(S_j, \phi(S_j))$ entry is 0. Yet by construction, $S_i \in K$.

This yields a contradiction. Therefore, there does not exist a surjection from S to $\mathcal{P}$, equivalently, there does not exist an injection from $\mathcal{P}(S)$ to S, so $|\mathcal{P}(S)| \not\leq |\mathbb{N}|$, hence the power set is uncountable.

$\square$

Corollary:
For any alphabet Σ, the set of languages over Σ is uncountable.

Proof:
Note that Σ^* is the set of all strings over the alphabet Σ. We know that $|\mathbb{N}| \leq |\Sigma^*|$ because we can fix any $a \in \Sigma$ and then consider the injection $\phi : \mathbb{N} \to \Sigma^*$ given by $\phi(n) = a^n$.
By Cantor's Theorem proven above, the power set $\mathcal{P}(\Sigma^*)$ is uncountable. Note that a language over the alphabet Σ is a set of strings over Σ, therefore, the set of all languages are all possible sets of strings over Σ, hence it is the power-set of Σ^*. Therefore

$$|\text{language over } \Sigma| = |\mathcal{P}(\Sigma^*)| \nleq |\mathbb{N}|$$

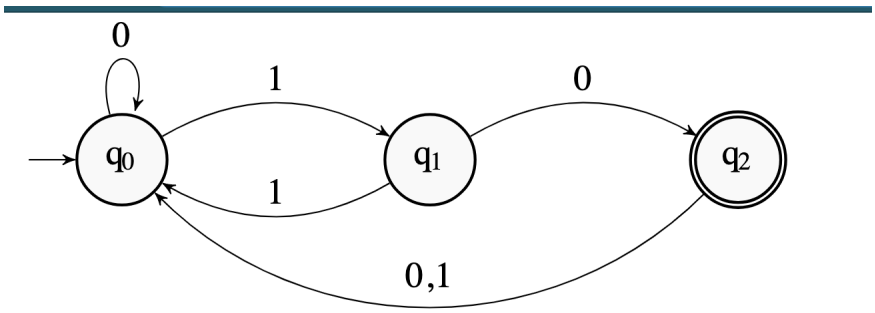
Hence it is uncountable.

$\square$

2 Week 2

2.1 Lesson 5. Deterministic Finite Automate (DFAs)

A DFA is an abstract machine, such that we can use transition diagrams to describe their behaviour:



In this diagram,

- The **states** of the machine are the nodes
- The **state transitions** are indicated by edges. We follow along the edges whenever we read into the next symbol of a string.
- A machine accepts a string if and only if it ends up at a **double circle** after reading through the entire string.

<u>List of questions</u>

- Which strings in Σ_2^5 does the DFA accept?
- Draw the transition diagram of a DFA such that it accepts exactly the set of strings $x \in \Sigma_2^*$ that gives the binary representations of integers that are multiplies of 3.

2.1.1 Definition of DFAs

Though transition diagrams are good to represent DFAs, using a mathematical definition is also crucial.

Definition (DFA):
A DFA is an abstract machine described by the 5−tuple M:

$$M = (Q, \Sigma, \delta, q_0, D)$$

where

- Q is a finite set of **states**
- Σ is the **input alphabets**
- $\delta : Q \times \Sigma \to Q$ is the **transition function**
- $q_0 \in Q$ is the **start state**
- $F \subseteq Q$ is the set of **accepting states**

Q: what is the formal definition of the DFA represented at the top of the page?
A: $\{\{q_0, q_1, q_2\}, \Sigma_2, \delta, q_0, q_2\}$, where δ sends $(q_0, 0), (q_0, 1), (q_1, 0), (q_1, 1), (q_2, 0), (q_2, 1) \to (q_0, q_1, q_0, q_0, q_0, q_0)$.

Definition (Size of a DFA):
The size of a DFA is the number of elements in Q.

Definition (DFA string acceptance):
The DFA $M = (Q, \sigma, \delta, s, F)$ accepts the string $w \in \Sigma^n$ if and only if there is a sequence of states $r_0, r_1, \ldots r_n \in Q$ such that

- $r_0 = q_0$
- $r_n \in F$
- $r_i = \delta(r_{i-1}, w_i)$ for each $1 \leq i \leq n$.

Q: Does the DFA at the top of the section accept the string 0010110? Use the formal definition to justify your ansewr.
Q: Does it accept the string 1001101? Again use the formal definition to justify your answer.

2.1.2 Languages of DFAs

Definition (DFA rejecting a string):
We say that a string $x \in \Sigma^*$ is rejected by a DFA if the DFA does not accept the string x.

Definition (Language recognized by a DFA M):
The language that is recognized by a DFA M is:

$$L(M) = \{x \in \Sigma^* \mid \ M \text{ accepts } x\}$$

This way, we associate a DFA precisely with the language in a natural way. Hence, we can sometimes call $L(M)$ the *language accepted by* M, or *the language of* M.

<u>List of questions</u>

- Design a DFA that recognizes each of the following languages:
  1. $\{x \in \Sigma_2^* : x \text{ ends with } 1\}$
  2. $\{x \in \Sigma_2^5 : x \text{ ends with } 1\}$
  3. $\{x \in \Sigma_2^* : \text{the second last symbol of } x \text{ is } 1\}$
  4. $\{x \in \Sigma_2^* : x \text{contains the same number of 01 and 10 substrings}\}$

Please see tablet notes for the solutions.
For each of the values of k below, design a DFA that recognizes the language of all strings $x \in \Sigma_2^*$ which give the binary representation of non-negative integers that are multiples of k. For a bonus challenge, design a DFA wich the number of states indicated in parenthesis.

- $k = 2$ (2 states suffice)

- $k = 3$ (3 states suffice)
- $k = 4$ (4 states suffice)
- $k = 5$ (5 states suffice)
- $k = 6$ (6 states suffice)
- Note: to solve the above question, we can consider states corresponding to the modulo residues with respect to k, and consider appending $0, 1$ (i.e. appending $0, 1$ to a number is equal to multiply by 2, then add 0 or 1. We can do this to mirror the modulo residue.)
- Let A be a language over Σ_2 such that it contains only strings of length at most n. Show that there is a a DFA with $O(2^n)$ states that recognize A.

  Idea: why don't we do a strip of $n + 2$ states, such that $q_i \to q_{i+1}$ for each alphabet, but only the first $n + 1$ states are circled, and the $n + 2^{th}$ state goes back to itself for both $0, 1$.

2.1.3 Regular Languages

Definition (Regular language):
The language A is regular if and only if there exists a DFA M such that $A = L(M)$.

Q: If A is a regular language, must $\overline{A}$ also be regular?
A: does reversing the circling work? i.e. change F to $Q \setminus F$.

Proposition:
Every finite language is regular.

Proof: Let A be a finite language. Then, let n be the length of the longest string in the language. Then, we create the following states identified by strings

$$Q = q_0 \cup \{x_0x_1 \dots x_k \mid x_0x_1 \dots x_k \text{ is a prefix of some strings in the language}\} \cup q_{NO}$$

Since there are finite strings, so there must be finite prefixes of all the strings.
We have q_0 represent the empty string. The set of alphabets Σ is the same. The set of accepting states are precisely represented by strings in the language. The transitions are: (regard q_0 as ϵ.) at $x_0x_1 \dots x_k$ with $\sigma \in \Sigma$, if $x_0x_1 \dots x_k\sigma$ is a prefix of some strings in A, then $\delta(x_0x_1 \dots x_k, \sigma) = x_0x_1 \dots x_k\sigma$. If it is not, $\delta(x_0x_1 \dots x_k, \sigma) = q_{NO}$. For all $\sigma \in \Sigma$, $\delta(q_{NO}, \sigma) = q_{NO}$. This DFA precisely accepts the langauge A. $\square$

Note that since not every regular languages are finite, the set of regular languages is a strict superset of the set of all finite languages. We now show that the set of regular languages is a strict subset of the set of all languages.

Proposition:
There exist languages that are not regular.

Proof: Remark: The language that consists of palindromes over Σ_0^* is not regular.
We have seen that the set of all languages over an alphabet Σ is uncountable.
Consider the set of all DFAs over an alphabet Σ. Let the alphabet have size n. Let the DFA have m states. Consider the definition of DFA using the 5 tuple $(Q, \Sigma, \delta, q_0, F)$. Then, number of transition functions is m^{km}, because each state and alphabet combination maps to a choice of m states. The total choices for starting states is m. The total choice for the accepting state is 2^m, the power set of all states. Together, for each n, m, there is a finite number of DFAs we can build. Hence, the cardinality of total number of DFA cannot exceed $\mathbb{N} \times \mathbb{N} \times \mathbb{N}$, which is countable. That means there are at most countable regular langauges.
The set of regular languages is countable. However, the set of all languages is not countable, so no injection of set of all languages to set of all DFAs.

□

Bonus problem: The Separating Words Problem
The main idea of this problem is to look for the smallest DFA that distinguishes between to strings x, y. There are four questions:

- (Easy) Prove that every two distinct strings $x, y \in \Sigma^n$, there exists a DFA with at most $n + 2$ states that accepts x but not y.
- (Medium) Prove that every two strings $x \in \Sigma^n$ and $y \in \Sigma^m$, $m < n$, there exists a DFA with $O(\log(n))$ states that accepts x but not y.
- (Hard) Prove that every two distinct strings $x, y \in \Sigma^n$, there exists a DFA with $o(n)$ states that accepts x but not y.
- (Open) The best bound on the Separating Words Problem was established by Zachary Chase, $\tilde{O}(n^{1/3})$ states suffice. But the only lower bound for the problem is $\Omega(\log(n))$. Do you think you can improve either bounds?

2.2 Lesson 6. Nondeterministic Finite Automatas

Note that DFA are "deterministic" in the sense of moving between states are completely determined by the symbols in the string. That is, with the same DFA and the same string, it will always have the same "path" in the DFA. For NFAs, the machine is allowed to make some choices during its execution. That is, the same string does not always finish at the same state, depending on the machine's choice.

Definition (Nondeterministic Finite Automata):
A nondeterministic finite automaton (or NFA) is an abstract machines described by

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

- Q is a finite set of *states*
- Σ is the *input alphabet*
- $\delta : Q \times (\Sigma \cup \epsilon) \to \mathcal{P}(Q)$ is the *transition relation*
- $q_0 \in Q$ is the *start state*
- $F \subseteq Q$ is the set of *accepting states.*

Definition (ϵ-transition):
An ϵ transition in the NFA M is a transition of the form $\delta(q, \epsilon)$ for some $q \in Q$. At this state, the NFA M can choose to follow this transition from q at anytime without processing any symbol from the input. That is, M can follow that ϵ transition (or not follow) without eating any symbols in the string.

Definition ($\epsilon-$ reachable states):
A state $r \in Q$ is ϵ reachable from a state $q \in Q$ in the NFA M, if there are states $s_0, s_1, s_2, \ldots, s_l$ for some $l \geq 0$, such that $s_0 = q, s_l = r$, and that $s_i \in \delta(s_{i-1}, \epsilon)$ for each $i = 1, 2, \ldots, \ell$. The state $r \in Q$ is ϵ-reachable from a set $S \subseteq Q$ of states if it is reachable from some states $q \in S$. (That is, the state is ϵ reachable from a set of states if it is reachable from one of the states in the set.)

Definition (NFA acceptance):
The NFA M accepts the string $x \in \Sigma^n$ if and only if there are states such $r_0, \ldots r_n \in Q$ such that

- r_0 is ϵ reachable from q_0
- the state r_i is ϵ reachable from $\delta(r_{i-1}, x_i)$ in M for each $i \in [n]$
- $r_n \in F$

List of questions

- Prove that for every NFA M, there exists another NFA N with exactly 1 accepting state that accepts exactly the same set of strings as M.

  Answer is to create a new state A, and add an ϵ transition from every accepting state to A, and make the accepting states as non-accepting, and only marking A as acceptance state.

- Is it possible for two NFAs $M = (Q, \Sigma, \delta, q_0, F)$ and $M = (Q, \Sigma, \delta, q_0, Q \setminus F)$ to both accept some strings $x \in \Sigma^*$?

  Answer: yes, we can consider the following example. Suppose in a NFA, A and B are some of its states. Originally, we have $A \to B$ with ϵ transition, $A \in F, B \notin F$, but then switching over to $A \notin F, B \in F$, the new NFA and the old NFA will still accept the same string, as the before.

2.2.1 The languages of NFAs

Definition (Language recognized by NFA):
The langauge recognized by the NFA N is:

$$L(N) = \{x \in \Sigma^* \mid \text{N accepts x}\}$$

List of questions: Try to design an NFA that recognizes the following languages:

1. $\{w \in \Sigma_2^* : w \text{includes 00, or 010 as substring}\}$
2. Bonus: make the above with only 5 states.
3. $\{w \neq 01\}$
4. Bonus: make the above with only 4 states.

Please see my tablet notes for solutions.

2.2.2 Regular languages and NFAs

Proposition:
Every regular language can be recognized by an NFA.

Proof:
Let A be a regular language. Let N be the DFA that recognizes A. We write $N = (Q, \Sigma, \delta, q_0, F)$. We now define a NFA M. Let

$$M = (Q, \Sigma, \delta_1, q_0, F)$$

It has the same set of states, alphabets, accepting states and initial state. However, the transition function is defined as follows:

$$\delta_1(q, \sigma) = \{\delta(q, \sigma)\} \text{ for } \sigma \in \Sigma$$

$$\delta_1(q, \epsilon) = \{q\}$$

This NFA accepts a string if and only if N accepts the string, because we made transitions deterministic and following the original transitions.

□

Two motivations to study NFAs are:

- nondeterminism
- we can recognize some langauges more easily with NFAs

Consider the two challenge questions:

Problem 1 (Challenge #1):

- Let M_1, M_2 be two DFAs. Design a DFA M such that $L(M) = M_1 \cup M_2$
- Let N_1, N_2 be two NFAs. Design a NFA M such that $L(N) = M_1 \cup N_2$

The answer to the second problem is easier. We can add a beginning state such that it ϵ transitions to both N_1 and N_2. The answer to the first problem requires making states of the form $S_1 \times S_2, S_1 \in M_1(\text{states}), S_2 \in M_2(\text{states})$, that is, the cartesian product of the states. The transition is defined by $\delta((q_1, q_2)) = (\delta_1(q_1), \delta_2(q_2))$.

Problem 2 (Challenge #2):
Let $A \in \Sigma_2^*$ be the language of all strings of length $\geq n$, such that their n^{th}-last symbol is 1. For example, if $n = 3$, then the third last symbol is 1. That is $100, 111, 1100, 1111$, so on.

- Design the smallest DFA M such that $L(M) = A$.
- Design the smallest NFA N such that $L(N) = A$.
- Can you identify $B \in \Sigma_2^*$ with an even bigger difference between the number of states to recognize B with a DFA instead of an NFA?
- For a bigger challenge, can you prove your answer to the first two items achieve the minimal size?

2.3 Lesson 7. Rabin Scott theorem

Theorem (Rabin-Scott Theorem):
The set of languages that can be recognized by a DFA is exactly the same set of languages that can be recognized by NFAs.

Proof: Let A be a language that can be recognized by a DFA. By the previous lemma, A can also be recognized by a NFA. Let B be a language that can be recognized by a NFA. By the following lemma, B can also be recognized by a NFA. $\square$

This result should be somewhat surprising, because it says that the nondeterministic property of NFAs does not add more power to identify languages. Since we already saw that every language that can be recognized by a DFA can also be recognized by NFA, it suffices to complete the proof of the following lemma in order to complete the proof.

Lemma 2.1:
For every NFA N, there is a DFA M that recognizes $L(N)$.

Proof:
Let L be a language that the NFAs N can recognize. We will construct a DFA D such that it recognizes L. Let N be described by $(Q, \Sigma, \delta : Q \times (\Sigma \cup \{\epsilon\}) \to P(Q), q_0, F)$.
We construct D as follows. Its states are described by $P(Q)$. Given state $\{q_{i,1}, q_{i,2}, \ldots q_{i,k}\}$ and alphabet σ, its transition functions are defined by

$$\delta_1(\{q_{i,1}, q_{i,2}, \ldots q_{i,k}\}, \sigma) = \bigcup_{1 \leq j \leq k} \delta(q_{i,j}, \sigma) \cup r\left(\bigcup_{1 \leq j \leq k} \delta(q_{i,j}, \sigma))\right) \text{ for } \sigma \in \Sigma$$

we let $r(P)$ (where P is a set of states) denote the states that are ϵ reachable from P.
The initial state is $\{q_0\}$. The accepting states are described F_1, which is the power set of Q that contains at least an element that is also in F, the power set of the final states except the empty set.
Note that N accepts a string x if and only if there exists states $r_0, \ldots, r_n \in Q$ such that

- r_0 is ϵ reachable from q_0
- the state r_i is ϵ reachable from $\delta_1(r_{i-1}, x_i)$ in M for each $i \in [n]$
- $r_n \in F$.

This occurs if and only if for at least one computational path $q_o, q_1, \ldots q_k$ of x, we have $q_k \in F$, which occurs if and only if some final state of a possible computation path of x is in F_1. This is equivalent condition of D accepting x. Therefore, we have proven that D accepts exactly the same set of strings as N.

$\square$

2.4 Lesson 8. Regular expressions

They are useful to describe many set of strings. In fact, we can express all strings in Σ_2^* that contains 111 as

$$(0 \mid 1)^* 111 (0 \mid 1)^*$$

Definition (Regular expression):
A regular expression over the alphabet Σ is a nonempty string r over

$$\Sigma \cup \{|, ^*, (,), \epsilon, \varnothing\}$$

such that it satisfies **one of** the following conditions:

- $r = \varnothing$
- $r = \epsilon$
- $r = \sigma$ for some $\sigma \in \Sigma$
- $r = (s \mid t)$ for some regular expressions s, t
- $r = (st)$ for some regular expressions s, t, or
- $r = (s^*)$ for some regular expressions s.

Note that Σ should not contain the strings from $\{|, *, (,), \epsilon, \varnothing\}$ for ambiguity.

Note:

- Note that none of $\{|, ^*, (,), \epsilon, \varnothing\}$ is in Σ.
- We often omit writing too many parenthesis for the sake of the order of operation. In order to do this, we define the following order of operation
  1. star operation
  2. concatenation
  3. unions

Q. How many parentheses do we need to write the regular expression at the beginning of the section, if we don't omit parenthesis?

A: $(((0 \mid 1)^*)(((11)1)((0 \mid 1)^*)))$: 8 sets

2.4.1 Languages of Regular Expressions

Definition 2.1 (Language described by a regex):
The language described by a regular expression r over Σ is the language $L(r) \subseteq \Sigma^*$, where

- $L(r) = \varnothing$ if $r = \varnothing$
- $L(r) = \{\epsilon\}$ if $r = \epsilon$
- $L(r) = \{\sigma\}$ if $r = \sigma$ for some $\sigma \in \Sigma$
- $L(r) = L(s) \cup L(t)$ if $r = (s \mid t)$ for some regular expression s, t
- $L(r) = L(s)L(t)$ if $r = (st)$ for some regular expression s, t
- $L(r) = L(s)^*$ if $r = (s^*)$ for some regular expression s

Note that language described by a regex is recursively defined.

List of questions

- Design a regular expression that describe each of the following languages:
  1. $\{w \in \Sigma_2^* : w \text{ ends with a } 1\}$ Answer: $(0 \mid 1)^*1$
  2. $\{w \in \Sigma_2^* : w \text{ contains even numbers of 0s}\}$ Answer: $(1^*01^*01^*)^*$
  3. $\{w \in \Sigma_2^* : w \text{ contains even numbers of 0s and even number of 1s}\}$ $(11|00)^*((10|01)(11|00)^*(10|01))^*(11|00)^*)^*$
  4. $\{w \in \Sigma_2^* : w \text{ does not contain 01 as substring}\}$ Answer: $(0^* \mid (1)^*0^*)$
  5. $\{w \in \Sigma_2^* : w \neq 01\}$ Answer: $(((1 \mid 0)(1 \mid 0)(1 \mid 0)(1 \mid 0)^* \mid (((0 \mid 1) \mid (00 \mid 11)) \mid (10))) \mid \epsilon)$
- Which languages do the following regular expressions describe?
  1. 0^*110^* strings with only two 1s, and those 1s are adjacent to eachother
  2. $(0 \mid 1)^*11(0 \mid 1)^*$ strings having 11 as substring.
  3. $0^*(10^*0)^*(1 \mid \epsilon)$ strings do not have 11 as substring??
- Prove all finite languages can be described by regular expressions: we can just construct all of them as $(abcde)$ and construct them by using union.

2.4.2 Bonus question: short regular expressions

Earlier, we say it is hard to come up with the smallest finite automaton that achieves certain goals, same with regular expression. We ask the question: given a language, what is the smallest size regular expression that describes that language?

Definition (Length of a regular expression):
The length of a regular expression r over Σ is the number of symbols from Σ in the string r.

Note that the length of the regular expression does not equal to the length of string that represents it. For example, (0^*) has length 4 as a string but length 1 as a regular expression.

Is it true that the regular expression and the string that defines it differ by at most a constant multiplicative factor? Not necessarily. Consider $(((0^*)^*)^*)^*$ can go on forever. It is valid though it is not the most convenient way to write it. We can ask a question regarding length of the regex and the length of the shortest string that represents the same language.
An open problem is: for each $k \geq 2$, we let A_k represent the language over $\Sigma_k = \{1, 2, \ldots, k\}$, which contains all the nonempty string in which no two consecutive symbols are identical. For example:

$$A_2 = \{1, 2, 12, 21, 121, 212, \ldots\}$$

$$A_3 = \{1, 2, 3, 12, 13, 21, 23, 31, 32, 121, 123, 131, 132, 212, 213, \ldots\}$$

What is the shortest regular expression that can represent A_k? The current bound is $O(k^{2.17})$ and the correct bound is $O(k^2)$.

3 Week 3

3.1 Lesson 9. Kleene's theorem

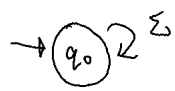
Theorem (Kleene's Theorem):
The language A is regular if and only if it can be described by a regular expression.

Note that the Rabin Scott theorem showed that the set of languages that can be recognized by NFAs are exactly the set of languages that can be recognized by DFAs, namely the regular languages. Kleene's theorem shows us that this is equivalent to being able to be described by regular expressions.
We break the proof into two.

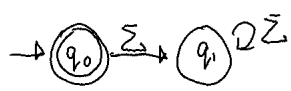
Lemma (RegEx to NFA):
For every regular expression r, the language $L(r)$ described by r can be recognized by an NFA.

Proof: We proceed by induction.
<u>Base cases</u>

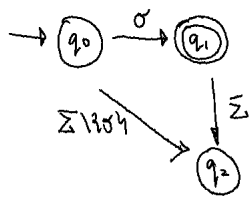
$L(r) = \phi$



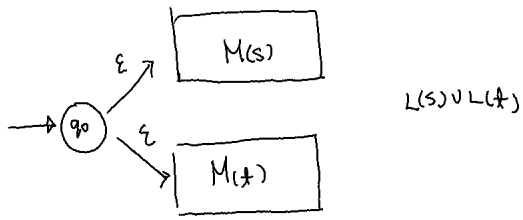
$L(r) = \{\varepsilon\}$

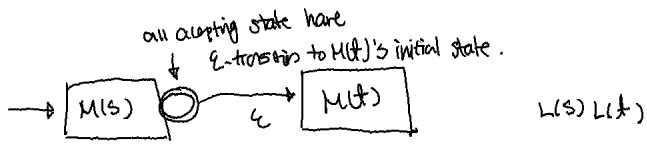


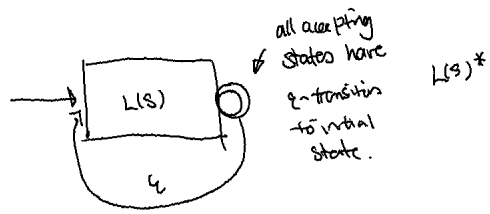
$L(r) = \{\sigma\}, \sigma \in \Sigma$



<u>Induction hypotheses</u> Assume for all lengths $\leq k$, a regular language can be recognized by a DFA.
<u>Induction proof</u> For the length $k+1$, it is either the concatenation, union or kleene star of union/concatenation of some languages that has a shorter regular expression length with shorter lengths.







(For Kleene star, add an epsilon transition from the start to a new state marked accepting, which makes it take the empty string.) □

Lemma (DFAs to regular expressions):
Every language L that can be recognized by a DFA can be described with a regular expression.

Proof: Let $M = (Q, \Sigma, \delta, s, F)$ be a DFA that recognizes L. Let $Q = \{1, 2, \ldots m\}$.
We show that for every $q, r \in Q$, and $k \in \{0, 1, 2 \ldots m\}$, we can define $L_{q \to r}(k)$ to be the language of all strings $x \in \Sigma^*$ that induces path from q to r while only using $1, \ldots k$ many states as intermediate states.
We use induction on the set k, that is $k = 0, k = \{0, 1\}, k = \{0, 1, 2\}, \ldots$.
When $k = 0$, we think about languages induce from q to r with no intermediate states. There are the cases:

- If q, r are the same, and there is no loop from q to q, then empty string. If $s \subseteq \Sigma$ induce the loop, then it is s^*
- If different, then if s_0, s_1, s_2, s_3 are subsets of Σ that induce path in in $(q, q), (q, r), (r, q), (r, r)$ respectively, then the languages $L_{q \to r}(0)$ is

$$(s_0)^*(s_1)(s_2)^*((s_3)(s_0)^*(s_1)(s_2)^*)^*$$

Now, suppose that by induction, we have a language representing paths from q to r going thru any number from $\{0, 1, 2 \ldots, k\}$ intermediate states. Now, we look at language representing paths from q to r going thru $\{0, 1, 2 \ldots, k, k+1\}$ state.
Possibilities include:

- Go thru k intermediate state, arrive at state p, then p goes to r, and r has no arcs back to the previous states (q,k along with the k intermediate states) This can be defined by

$$(L_{q \to k} \mid L_{k \to r})$$

- It arrives at r but goes back to one of the states (q,k along with the k intermediate states) In this case, if it goes back to L, then it is

$$(L_{q \to k} \mid L_{k \to r})^*$$

  otherwise, we concatenate the possibilities, as if j is a state that r maps to, then we know the intermediate state between j, r is less than $k+1$, so we have $L_{j \to r}$. In this case, we model loops from r to r as

$$|_{j, \text{ such that there exist arc } r \to j} \ (L_{r \to j})(L_{j \to r})$$

I believe the above should cover all the possible cases.
At the end, we have languages from $Q \times Q$. Eventually, we take the union of paths (languages) from the start state to the final states.

□

3.2 Lesson 10. Pumping Lemma

Suppose we are given a specific language A. If it is not regular, how do we prove it? The pumping lemma is a tool that can help us to prove that some languages are non-regular.

Definition (Non Regular):
A language is non-regular if it is not a regular language. (no NFAs, DFAs or Regex.)

Proposition:
The language $L = \{0^n 1^n \mid n \geq 0\}$ is not regular.

Proof: Let there be m states. Consider parsing the strings $0, 0^2, \ldots, 0^{m+1}$, by pigeonhole, two must end up at the same state. Say $0^a, 0^b$. Then the DFA would accept $0^a 1^a$, but it will also accept $0^b 1^a$ which is incorrect. □

Theorem (The pumping lemma for regular langauges):
For every regular language L, there is a number p such that for any string $s \in L$ of length at least p, we can write $s = xyz$ where

- $|y| > 0$
- $|xy| \leq p$
- For each $i \geq 0, xy^i z \in L$

Proof:
Suppose we are given the language A. We let the M be a DFA such that $L(M) = A$. Let $M = (Q, \Sigma, \delta, q_0, F)$. Then, we set the number $p = |Q| + 1$.
We will show that for each string $s = (s_1, \ldots, s_\ell) \in A$ with $len(s) \geq p$, we can write $s = xyz$ such that it follows the conditions.
Let $\ell = len(s) \geq |Q| + 1$. Consider the sequence of states $q_0, q_1, \ldots q_\ell$ be the unique sequence of states that is needed to parse s. That is, we have the states such that $q_0 = q_0$ and for all $i \in \{1, \ldots \ell - 1\}$, we have $\delta(q_i, s_i) = q_{i+1}$. Consider the set $S = \{q_0, q_1, \ldots q_{|Q|}\}$. Note that $|S| = |Q| + 1$. By pigeonhole principal, there are at exactly $|Q|$ states, and $|Q| + 1$ states visited, this means that one state is visited twice. This implies that some q_i, $0 \leq i \leq |Q|$ such that it is visited twice when parsing the first $|Q|$ symbols of s. For the second time that the string appeared, we label it as index j.
We can write the fist $|Q| + 1$ visited states as:

$$q_0, q_1, q_2, \ldots, q_{i-1}, q_i, q_{i+1}, \ldots, q_{j-1}, q_j = q_i, q_{j+1}, \ldots q_{|Q|}$$

We claim that we set

- $x = s_1, \ldots, s_i$
- $y = s_{i+1}, \ldots, s_j$
- $z = s_{j+1}, \ldots, s_\ell$

We now verify that x, y, z it satisfies the conditions for pumping lemma.

- To check that $|y| > 0$:

  Note that since $i \neq j$, they are different indices, so the string $s_{i+1}, \ldots, s_j$ has length at least 1. This proves $|y| > 0$.

- To check that $|xy| \leq p$:

  Consider the string $xy = s_1, \ldots, s_j$. Note that $j \leq |Q| = p - 1 < p$. So the length of substring xy is at most $p - 1$. So $|xy| < p$.

- We will show that for each $n \geq 0$, $xy^n z \in L$.

  When $n = 1$, we know that M accepts $s = xyz$. So $s \in L$.

  When $n = 0$, we claim the following sequence of states is the unique sequence of states to parse xz :

$$xz = s_1 \ldots, s_{i-1}, s_i, s_j, s_{j+1} \ldots s_\ell$$

$$q_0, q_1, q_2, \ldots, q_{i-1}, q_j = q_i, q_{j+1}, \ldots q_\ell$$

This is because for $k \in \{0, 1, \dots i-1\}$ we know that $\delta(q_k, s_{k+1}) = q_{k+1}$. We also know $\delta(q_i, s_{i+1}) = \delta(q_j, s_{i+1}) = q_{j+1}$. For $k \in \{j+1, j+2, \dots, \ell-1\}$, we also have $\delta(q_k, s_{k+1}) = q_{k+1}$.

Now, for $n \geq 2$, we have $xy^iz = s_1, \dots, s_i(s_{i+1}, \dots, s_j)^i s_{j+1}, \dots, s_\ell$.

We claim that the sequence of states of parsing this string is exactly

$$q_0, q_1, q_2, \dots, q_{i-1}, q_i, (q_{i+1}, \dots, q_{j-1}, q_j = q_i)^n, q_{j+1}, \dots q_\ell$$

Now, we see that for $k \in \{0, 1, \dots i-1\}$ we know that $\delta(q_k, s_{k+1}) = q_{k+1}$. Similarly, parsing $k \in \{i, i+2, i+3, \dots j-1\}$, we obtain $\delta(q_k, s_{k+1}) = q_{k+1}$. Now, we will parse the same string $s_{i+1}, \dots, s_j$ again, we see that $\delta q_j, s_{i+1} = \delta q_i, s_{i+1} = q_{i+1}$. Then the next symbols $s_{i+2}, \dots, s_j$ also parses and arrive at the same states $q_{i+1}, \dots q_j$. This repeated substring is parsed for i times. After parsing the last instance of the repeated string, we arrive at state q_j with string s_{j+1}. Note that $\delta(q_j, s_{j+1}) = q_{j+1}$. Afterwards, for $k \in \{j+1, j+3, \dots \ell-1\}$, we have $\delta(q_k, s_{k+1}) = q_{k+1}$. Therefore, we end up at state q_ℓ after parsing the string. From previously we know that q_ℓ is an acceptance state, hence the DFA M will accept the string xy^nz.

□

Definition 3.1 (Pumping property):
A language L that satisfies the conditions of the lemma is said to satisfy the pumping property.

Definition 3.2 (Pumping length):
A value p for which L satisfies the pumping property is known as a pumping length of L.

We consider the contrapositive of the pumping lemma. This can let us show that some languages are non-regular.

Theorem (Contrapositive of the pumping lemma):
If for every positive integer p, there exists a string $s \in L$ of length at least p such that for every decompositions $s = xyz$, with $|y| > 0$ and $|xy| \leq p$, there exist a value $i \geq 0$ for which $xy^iz \notin L$, then L is not a regular language.

We can easily show this is true for 0^n1^n.

3.3 Lesson 11. Non-Regular languages

Proposition:
The language $L = \{0^n1^n : n \geq 0\}$ is not regular.

Proof: Contrapositive of the pumping lemma.
For any $p \geq 1$, choose $s = 0^p1^p$.

Then, we have $s \in L$ and $|s| \geq p$.
Any decomposition $s = xyz$ with $|xy| \leq p$ and $|y| > 0$, must have $|xy|$ being a run of 0s. (This is because the first 1 character appears at the index $p+1$, yet the substring xy has length at most p.) This means that y is also a run of 0 with length k where $k \geq 1$. Therefore, for $i = 2$, we have $xy^2z = xyyz$, yet it has k more $0s$ than $1s$. So $xy^2z \notin L$. By the pumping lemma, L is not regular.

□

3.3.1 Slight variants of this

Proposition:
The language $L_< = \{0^m 1^n : n > m \geq 0\}$ is not regular.

Proof: Contrapositive of the pumping lemma.
For any $p \geq 1$, choose $s = 0^p 1^{p-1}$.
Then, we have $s \in L$ and $|s| \geq p$.
Any decomposition $s = xyz$ with $|xy| \leq p$ and $|y| > 0$, must have $|xy|$ being a run of 0s. (This is because the first 1 character appears at the index $p+1$, yet the substring xy has length at most p.) This means that y is also a run of 0 with length k where $k \geq 1$. Therefore, for $i = p-1$, we have $xy^{2(p-1)}z$, we know that there is at least $2p-2$ zeros yet also only $p-1$ $1s$. This means there run of 0 is way longer. So $xy^iz \notin L$. By the pumping lemma, L is not regular.

□

Proposition:
The language $L_> = \{0^m 1^n : m > n \geq 0\}$ is not regular.

Proof: Contrapositive of the pumping lemma.
For any $p \geq 1$, choose $s = 0^{p+1} 1^p$
Then, we have $s \in L$ and $|s| \geq p$.
Any decomposition $s = xyz$ with $|xy| \leq p$ and $|y| > 0$, must have $|xy|$ being a run of 0s. (This is because the first 1 character appears at the index $p+2$, yet the substring xy has length at most p.) This means that y is also a run of 0 with length k where $k \geq 1$. Therefore, for $i = 0$, we have xy^0z, we know that there is at least one less zero. Yet losing any zero would make the length of the run of 0 less than or equal the length of run of 1. So $xy^0z \notin L$. By the pumping lemma, L is not regular.

□

3.3.2 A non-regular unary language

Though we worked with binary alphabet, we can also work withh the language over unary alphabets that are non regular.

Proposition:
The language $L = \{0^{2^n} : n \geq 0\}$ is not regular.

Proof:
The proof idea is also using the pumping lemma. That would describe that a arithmetic sequence of numbers, each represent 2^i for some $i \in \mathbb{N}$. This is clearly absurd.

□

3.3.3 Determining Regularity

Now, we have established tools to prove whether A is regular or not. (Regular: DFAs or NFAs or RegEx.) (Non-regular: pumping lemma.)
We come to the following challenge that we can determine if a language is regular or not.

- $L_{pal} = \{w \in \Sigma_2^* : w = w^R\}$ Not regular. The proof almost exactly same as 0^n1^n.
- $L_{dup} = \{ww : w \in \sigma_2^*\}$ Not regular. The proof is using pumping lemma.
- $L_{prime} = \{0^q \mid q \text{ prime }\}$ Not regular. The proof is using pumping lemma. We would then get an infinite list of arithmetic sequences that are prime. This is not true (because of some number theory theorem, I believe.) Reference is below:

  `https://math.stackexchange.com/questions/2741007/arithmetic-sequence-where-every-term-is-prime`
- $L_{\neq} = \{0^n1^m : n \neq m\}$. We consider $L = \{0^a1^b\}$ and $L_{=} = \{0^n1^n\}$. If this language $L_{\neq}$ is regular, we would have a DFA for it. We already have a DFA for L (0^*1^*). Then we can have a DFA for $L \cap (L_{\neq})^C = L_{=}$. Contradiction.
- $L_{\geq'} = \{0^n1^m : n \geq m \text{ and } m \leq 1000\}$ Yes. This is regular. We first parse 1000 0s. Then, have a 0 state that has a 0 edge to itself and 1 to the next state. After the 1 is hit, parse 1000 1s. Then set the 1000th 1 as final state. Any 1 edge goes back to itself and 0 edge goes to fail. All the other edges implicitly go to fail.
- $L_{\geq''} = \{0^n1^m : n \geq m \text{ and } m \geq 1000\}$. Not regular. Same with the pumping lemma where you can shrink numbers.
- $L_{prime'} = \{10, 11, 101, 111, 1011, 1101, 10001, \ldots\}$ the language of binary representation of primes. My guess is no. If we can have xy^iz for all i, this would mean that we would have some sort of arithmetic-geo sequence that parse primes. This is good for further exercise.

3.3.4 Bonus: limitations of the pumping lemma

Note that the converse of the pumping lemma is not true. That is there could exist languages that are not regular but still satisfy the pumping lemma's properties. That is, the conditions of the pumping lemma is a necessary but not sufficient condition for language that are regular. In the future, we will see how Myhill-Nerode Theorem provides the conditions that are necessary and sufficient for a language to be regular.

Proposition 3.1:
The language

$$L_{pal^+} = \{ww^R x \in \Sigma_2^* : |w| \geq 1\}$$

satisfy the pumping property. But it is not regular.

Proof: It is not regular because its complement are strings do not start with 00 or 11.? Why does it satisfy the pumping property?

□

3.4 Lesson 12. Properties of Regular Languages

3.4.1 Closure under language operations

Note 3.2:
We have seen that the regular languages are closed under union, complement, concatenation, and star.

List of questions

- Can you see or remember why without looking back at earlier notes? Yes, I can visualize how to prove them via DFAs and RegEx.
- Use the closure properties for union and complement, to show class of regular langauges are closed under intersection, set difference, and symmetric difference.
  1. Intersection: $A \cap B = \overline{\overline{A} \cup \overline{B}}$
  2. Set difference: $A \setminus B = A \cap \left(B^C\right)$
  3. Symmetric difference: $(A \setminus B) \cup (B \setminus A)$

There are other operations under which regular languages are closed. For example, the expansion operator. We all strings that are "almost " in A to it.

Definition 3.3 (Distance):
The distance between two strings x and y over the same alphabet is $d(x, y)$, is the number of coordinates i such that $x_i \neq y_i$.

Definition 3.4 (Expansion operation of a language):
The expansion of the language A over Σ is the language

$$A^{\uparrow} = \{x \in \Sigma^* : \exists y \in A, |y| = |x|, d(x, y) \leq 1\}$$

The class of regular language is closed under language expansion.

Proposition 3.3:
For every regular language A, the language $A^{\uparrow}$ is also regular.

Proof: We do this by induction.
Base cases:

- $L(r) = \varnothing$, then $A^{\uparrow} = \varnothing$
- $L(r) = \epsilon$, then $A^{\uparrow} = \epsilon$
- $L(r) = \sigma$, then $A^{\uparrow} = \Sigma$

Now, suppose for the base case, the language $A^{\uparrow}$ can be described in regular languages, then

- If $r = (s \mid t)$ then if s' describe $L(s)^{\uparrow}$, t' describe $L(t)^{\uparrow}$ then

$$(L(s) \cup L(t))^{\uparrow} = (s \mid t) \mid (s \mid t') \mid (s' \mid t)$$

  One invariant we keep is that there is always at most 1 different symbol throughout the induction process.

- If $r = (st)$ then if s' describe $L(s)^{\uparrow}$, t' describe $L(t)^{\uparrow}$ then

$$(L(s)L(t))^{\uparrow} = (st) \mid (st') \mid (s't)$$

  One invariant we keep is that there is always at most 1 different symbol throughout the induction process.

- If $r = (s)^*$ then if s' describe $L(s)^{\uparrow}$,

$$L(s)^{*\uparrow} = s^*(s' \mid \epsilon)s^*$$

□

Note that there are also some language operations that are not closed under regular languages.

Problem (Challenge 1): Show that regular languages are closed under the following operations: Given regular languages A and B, we define:

- Reversal: $A^R = \{x^R : x \in A\}$. To show this, we also do induction with RegEx. Empty string, empty language, and one alphabet are the same. Then, for $s \mid t$: have $s^R \mid t^R$, and st have $t^R s^R$ and s^* have $(s^R)^*$.
- Shuffle: $sh(A, b) = \{x \in \Sigma^m : m \geq 0, x_1x_3x_5 \ldots x_{m-1} \in A\}$ and $\{x \in \Sigma^m : m \geq 0, x_2x_4x_6 \ldots x_m \in B\}$: build a DFA somehow... This one is difficult, i will leave it for now.
- Subsequqnece: subseq$(A) = \{x \in \Sigma^*, \exists z \in A$ that is a subsequence of x$\}$. Given the NFA for A, add ϵ and Σ transformations in the beginning states and after the acccepting states.

Problem (Challenge 2): Identify an operation on languages under which regular languages are not closed.
B such that B is a subset of A. There exist some subsets B of A that is not regular, if $|A| = \mathbb{N}$. Or intersection with a nonregular language.

3.4.2 Topological separation

Proposition 3.4:
There exists a language L over Σ_2 such that every infinite language $L' \subseteq L$ is non-regular.

Proof: $L = \{\omega \in \Sigma_2^*, \omega = 1^n 0^n, 1, 0 \in \mathbb{N}\}$.
I suspect the language $L = \{x = 0^n 1^n \mid n \geq 0\}$. We could use pumping lemma to show any infinite subset is nonregular. □

Proposition 3.5:
For every infinite regular language L, there is an infinite language $L' \subsetneq L$ that is regular.

Proof: Use the pumping lemma. Take the p and arbitrary s, and then delete $s = xz$, and we obtain the strict subset. This is regular because a regular language takeaway an element (that can be represented by regular language), is still regular.

□

<u>Topological separation problem</u>

Proposition 3.6:
For every infinite regular language L, there is a regular language $L' \subseteq L$ such that L' and $L \setminus L'$ are both infinite regular languages.

Proof:
Proof idea: look at the DFA, pick a state in a loop, and separate strings into the ones visiting it even or odd number of times. To track this, we can have states as $Q \times \{0, 1\}$ for tracking.

□

3.4.3 Bonus: Testing Emptiness and Equivalences

In the computability section (two weeks down the road), we will focus on three particular problems:

- Does it accept or reject every input string in a finite amount of time? Or, does it halt?
- Does the machine accept any string?

- Given two machines M_1, M_2, does $L(M_1) = L(M_2)$?

Note that the first question is trivial for DFAs, because they terminate after reading all the symbols, and there is only finite number of symbols in a string. The two other questions are more interesting.

Problem (Challenge 1): Describe an algorithm that, given the description of a DFA M, determine if $L(M) = \varnothing$ or not.
One way is to convert into a RegEx. and check (like the CNF forms) if any of the "concatenation" components are empty. If yes, then empty. Otherwise it is not empty. I feel like we need some sort of tree to do it. I think a DFS algorithm will be the best to accomplish this.

Problem (Challenge 2): Describe an algorithm that, given the description of two DFAs M_1 and M_2, determine whether $L(M_1) = L(M_2)$.
We just build corresponding DFAs to see whether $L(M_1) \setminus L(M_2)$ is the empty language as well as if $L(M_2) \setminus L(M_1)$ is also the empty language.
Given the two NFAs, we can see if they recognize the same language by converting into $DFAs$.

4 Week 4

4.1 Lesson 13. Pushdown Automata

Note:
Note that the languages like $\{0^n 1^n \mid n \in \mathbb{N}\}$ cannot be recognized by NFAs or DFAs or RegEx. The main reason seems like the NFAs/DFAs/RegEx does not have the "memory" to remember the runs of 0s and 1s. However, CFG and PDAs can.
The pushdown automaton models memory as a *stack*, and we will show that it can recognize a greater class of languages.

4.1.1 Description of PDAs

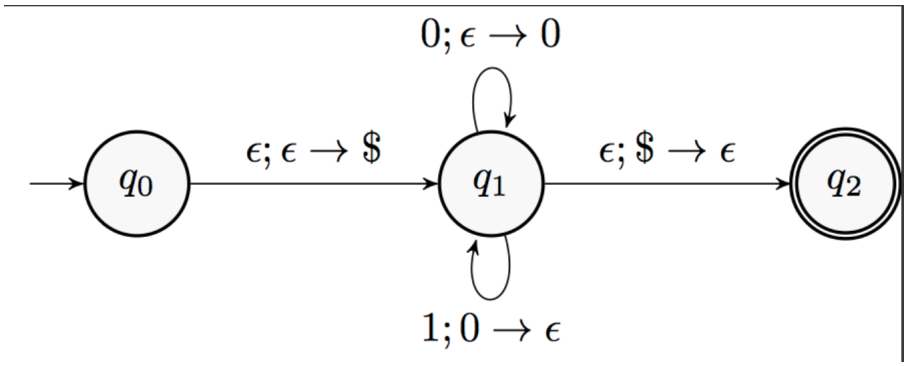
Definition (PDA):
A pushdown automaton is a nondeterministic finite automaton that has access to a stack. It can be described using a transition diagrams. A transition $a; b \to c$ is followed when

- Next symbol from the string is a
- Pop b from the stack
- push c onto stack

Any of the 3 symbols can be ϵ.

- When $a = \epsilon$, the transition does not read or advance in the input string
- When $b = \epsilon$, the automaton does not read or pop a symbol off the stack
- When $c = \epsilon$, no symbol is pushed onto the stack during transition.
- An ϵ transition is labeled ϵ when $a = b = c = \epsilon$

This is an example:



Note 4.1:

- When simulating reading PDA on an input string, we want to keep track of three pieces of information:
  - The symbol that PDA is currently examining
  - The current state of the automaton
  - The string in the stack
- It is possible multiple paths lead to acceptance.
- Which string does the above PDA accept?
  - 0010: no
  - 1010: no
  - 0110: no
  - 001011: Yes
  - 0100110: no
  - 00110101:yes
- How many strings of length 8 does the above accept?

  Answer: catalan numbers? Same number of $0, 1$ and at any point, there is always more 0 before it than 1s.

4.1.2 Definitions of PDAs

Notation: we use $\Sigma_\epsilon := \Sigma \cup \{\epsilon\}$

Definition (Nondeterministic pushdown automaton (PDA)):
A Nondeterministic pushdown automata (PDA) is an abstract machine defined by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

where

- Q is a finite set of *states*

- Σ is the *input alphabet*
- Γ is the *stack alphabet*
- $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \to \mathcal{P}(Q \times \Gamma_\epsilon)$ is the *transition relation*
- $q_0 \in Q$ is the *start state*
- $F \subseteq Q$ is the set of *accepting states*.

What exactly are different between PDA and NFA definitions? The main difference comes from the necessity to add a stack!

Note (Observations and questions):
Question: My question is, is there a way to make a PDA based on a DFA, instead of a NFA? If so, what does that imply? Also, what if we allow both queues and stacks in PDA? Would that be more powerful? I think yes. This is because the language $\{ww \mid w \in \Sigma_2^*\}$ actually can be implemented via queue, but just not stack. But using both queue and stack, or two stacks, we are able to identify $0^n 1^n 2^n$, which is not even context free!

Definition (PDA accepting a string):
The PDA M accepts the string $w \in \Sigma^*$ if and only if there exist a parameter $M \geq |w|$ with symbols $y_1, \ldots, y_m \in \Sigma_\epsilon$, states $r_0, \ldots, r_m \in Q$ and strings $s_0, \ldots, s_m \in \Gamma^*$ such that

1. $w = y_1 \ldots y_m$
2. $r_0 = q_0$ and $s_0 = \epsilon$
3. $r_m \in F$
4. For each $i = 1, \ldots, m$, there exists $a, b \in \Gamma_\epsilon$ and $t \in \Gamma^*$ such that
   - $(r_i, b) \in \delta(r_{i-1}, w_i, a)$
   - $s_{i-1} = at$
   - $s_i = bt$

Note:
What if we can do this with a queue?

4.1.3 Languages of PDAs

Definition (Language of a PDA):
The langauge of a PDA M is:

$$L(M) = \{w \in \Sigma^* : M \text{ accepts } w\}$$

We say M recognizes $L(M)$.

Question: Is it true that every PDA M, there exists a PDA M' with $L(M') = L(M)$ and and M' only accepts when its stack is empty? The answer is yes. We can have an extra state with ϵ transitions to itself, where it reads in nothing and always pop symbols.

4.1.4 Regular Languages and PDAs

Proposition 4.2:
Every regular language can be recognizes by a PDA.

Proof: Let $N = (Q, \Sigma, \delta, q_0, F)$ be a NFA that accepts L. Then, the PDA $M = (Q, \Sigma, \Gamma, \delta', q_0, F)$ where Γ is one element ϵ, and $\delta'(q, a, b) \to \delta(q, a) \times \{\epsilon\}$ is the transition relation. Then PDA accepts the language L.

□

Proposition 4.3:
There is a PDA that recognizes the langauge $L = \{0^n 1^n : n \geq 0\}$.

Proof: The construction is similar to the PDA at the beginning of the notes, except the middle is broken into two states, with first one having loops $0; \epsilon \to 0$ and second one having loops $1; 0 \to \epsilon$ with an ϵ transition between then. □

Question Why do we have a beginning of string and end of string symbol?
List of challenges

- Build a PDA that recognizes the following language:
  - $L_{pal} = \{w \in \Sigma^* : w = w^R\}$
    Idea: have two main states, one is for pushing, and one is for popping. Pushing are transitions to itself of form $a; \epsilon \to a$ and popping transitions are of form $b; b \to \epsilon$. There is only one ϵ transition between pushing to popping.
  - $L_{012} = \{0^i 1^j 2^k : i, j, k \geq 0, i = j \text{ or } i = k\}$
- Open question: x is primitive if it cannot be expressed as $x = y^n$ for some string y and some integer $n \geq 2$.

  The open problem is: does there exist a PDA that recognizes primitive binary strings?

$$L = \{w \in \Sigma_2^*, w \text{ is primitive}\}$$

4.2 Lesson 14. Context Free Grammars

CGFs are much more powerful than RegEx!
Note that these two CFGs below, one is a shorthand of another, generate the string of runs or at least one zeros followed by runs of at least one ones.

$$
\begin{aligned}
&S \to AB \\
&A \to 0A \\
&A \to 0 \\
&B \to 1B \\
&B \to 1
\end{aligned}
$$

$$
\begin{aligned}
&S \to AB \\
&A \to 0A \mid 0 \\
&B \to 1B \mid 1
\end{aligned}
$$

4.2.1 Definitions of CFGs

Definition (Context-Free grammar):
A Context free grammar (CFG) is a grammar

$$G = (V, \Sigma, R, S)$$

such that

- V is a set of *variables or nonterminal symbols*
- Σ is a set of terminals
- R is a finite set of *rules* mapping V to $(V \cup \Sigma)^*$
- S is the *start variable* in V

We remark that $G = (\{S, A\}, \{0, 1\}\{S \to A, A \to AA\}, S)$ is not a valid CFG because it can not be expanded into nonterminal symbols. Or it could be valid if it just represent the finite language?

Definition 4.1 (String yields another string):
The string uAv *yields* the string uwv in the CFG $G = (V, \Sigma, R, S)$,

$$uAv \Rightarrow uwv$$

when $A \in V$ is a variable, $u, v, w \in (\Sigma \cup V)^*$ are some strings, and the rule $A \to w$ is in R.

Definition 4.2 (String derives another string):
The string u derives the string v in G, denoted by

$$U \Rightarrow^* v$$

when $u = v, u \Rightarrow v$ or there is a sequence of $k \geq 1$ strings $u_1, \ldots, u_k \in (V \cup \Sigma)^*$ such that $u \Rightarrow u_1 \Rightarrow u_2 \Rightarrow \ldots \Rightarrow u_k \Rightarrow v$.

Note 4.4:
Note that it is possible that a string can derive many, countably many strings.
Also, it is possible that a string can derive a smaller, because something can be mapped to ϵ.

4.2.2 Context Free language

Definition (Language generated by a CFG):
The language generated by a CFG $G = (V, \Sigma, R, S)$ is

$$L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$$

which is the set of strings that can be derived in G from the initial string containing only the start variable of S.

Note 4.5:
Find a CFG that generate the following languages

1. $L = \{0^n 1^n \mid n \geq 1\}$
   - $S \to AB$
   - $A \to 0A \mid 0$
   - $B \to 1B \mid 1$
2. $L = \{$ well nested string of parentheses$\}$
   - $S \to A$
   - $A \to ()A \mid (A) \mid A() \mid \epsilon$
3. $L = \{w \in \Sigma_2^* : w = w^R\}$
   - $S \to A$
   - $A \to 1A1 \mid 0A0 \mid 0 \mid 1 \mid \epsilon$
4. $L = L(G_1) \cup L(G_2)$ where G_1, G_2 are two CFG grammars. Let $L(G_1)$ be recognized by (V_1, Σ, R_1, S_1)

   Let $L(G_2)$ be recognized by (V_2, Σ, R_2, S_2) with $V_1, V_2, R_1, R_2, S_1, S_2$ disjoint.

   Then $L(G)$ is recognized by $(V_1 \cup V_2, \Sigma, R_1 \cup R_2 \cup S \to S_1 \mid S_2, S)$.

Definition (Context free language):
A language is context free if it can be generated by a context free grammar.

4.3 Lesson 15. Equivalence of CFGs and PDAs

The main result of this section is to show that the set of languages recognized by CFG and PDAs are the same.

Theorem (CFGs and PDAs equivalence):
A language can be generated by a context-free grammar if and only if it can be realized by a pushdown automata.

Theorem ($CFGs \implies PDAs$):
For every context-free grammar G, the language $L(G)$ can be recognized by a pushdown automata.

Proof:
We recall the parsing stack from the cs241 on how to parse context free language.
Suppose we are starting with the context free grammar $G = (V, \Sigma, R, S)$, then we will turn it into a PDA as follows:

- We will construct a PDA as follows. We start with

$$M = (\{q_0, q_1, q_2\}, \Sigma, V \cup \{\Sigma\}, \delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon, q_0, q_2)$$

- We set δ send everything to $\varnothing$ in the beginning.
- Assume dollar sign \$ not in Σ. $q_0 \to q_1$ has the transition $\epsilon; \epsilon \to \$$, and $q_1 \to q_2$ has the transition $\epsilon; \$\epsilon$. This ensures that the stack is popped empty, by making sure that it can read the \$ sign that it put in in the very very beginning.
- For each $a \in \Sigma$, add the $q_0 \to q_0$ transition $a; a \to \epsilon$. This corresponds to the action of popping the symbol form the parsing stack.
- For each rule $V \to x_0 x_1 \ldots x_n$ where the $x_i s, 0 \leq i \leq n$, we will introduce the states $q_1 = q_{v0}, q_{v1}, q_{v2}, \ldots, q_{v(n-1)}, q_{vn} = q_1$ such that there is a transition $\epsilon; \epsilon \to x_i$ between $q_i \to q_{i-1}$ for $i = 1, \ldots n$. This allows us to pop the states $x_n, x_{n-1} \ldots, x_2, x_1, x_0$ onto the stack. So later we pop it off while reading in the string.

The proof of correctness:
Recall that in 241, that the CFG matches a string if and only if there is a corresponding leftmost derivation. This involves starting with the S symbol, and expand it by the rules, while greedily read into the inputs. Note that at each state, there might be more than one way to expand the variable symbols, and that is fine. All we require is for at least one combination of the possibilities to arrive at the correct output. The property of a NFA allows us to do this.

$\square$

Now, we shift our attention to show that the set of languages recognized by PDAs can be recognized by CFGs.

Lemma (Lemma needed for PDAs $\implies$ CFGs):
For every pushdown automaton M, there is a pushdown automaton

$$M' = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

that satisfies:

- F consists of a single accept state
- M' always empties the stack before accepting
- Each transition $a; b \to c$ in δ has exactly one b, c equal to ϵ, so that each transition either pushes a symbol onto the stack or pops a symbol from the stack.

and also satisfies that $L(M) = L(M')$.

Proof:
We will modify the PDA M into the form of M'. We first make M' the same as M, and we apply modifications to it.

- We let F' be the accepting states for M. In F, we copy is the accepting state of M', and make them no longer the accepting states. We have an ϵ transition from each of F' to F.
- We add a new start state q_0 before the old state state p_0 with transition $\epsilon; \epsilon \to \$$. Between each pair of accepting state C and a state prior to an accepting state A, we add a state B in between, such that $A \to B$ is the same as $A \to C$, the original transition, and $B \to C$ is $\epsilon; \$ \to \epsilon$. We have B as a state that the string run as many times as needed to "pop" all elements from the stack, so it has a transition to itself $\epsilon; \Sigma \to \epsilon$. This allows the string to keep on "popping" the elements off the stack.
- For any two states q_1, q_3 with transition $a; b \to c$ between them, we will remove this transition, add the state q_2, with the $q_1 \to q_2$ transition $a; b \to \epsilon$, and $q_2 \to q_3$ with $\epsilon; \epsilon \to c$. So, this will first pop b from the stack, and then in a separate transition, pop c onto the stack, and these together reads in a once.

$\square$

Theorem (PDAs $\implies$ CFGs):
For every pushdown automaton M, the language $L(M)$ can be described by a context-free grammar.

Proof: We first assume without loss of generality that $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ satisfies three conditions. (Note that the sketch proof is given in the notes section)

1. $F = \{q_{\text{accept}}\}$ consists of a single accept state.
2. M always empties the stack before accepting.
3. each transition $a; b \to c$ in δ has exactly one of b, c equal to ϵ, so that each transition either pushes a symbol onto the stack, or pops a symbol from the stack.

Let $G = (V, \Sigma, R, S)$ be a context free grammar with $V = \{A_{p,q} : p, q \in Q\}$, $S = A_{q_0, q_{\text{accept}}}$, and the set R of rules defined as follows:

- $A_{p,p} \to \epsilon$ for every $p \in Q$
- $A_{p,q} \to A_{p,r}A_{r,q}$ for every $p, q, r \in Q$
- $A_{p,q} \to aA_{r,s}b$ for every $p, q, r, s \in Q$ and $a, b \in \Sigma_\epsilon$, where there exists $u \in \Gamma$ such that $(r, u) \in \delta(p, a, \epsilon)$ and $(q, \epsilon) \in \delta(s, b, u)$.

We will complete the proof by showing that $A_{p,q}$ derives the string $x \in \Sigma^*$ if and only if x can bring M from state p with any empty stack to state q with an empty stack.

<u>$A_{p,q}$ derives $x \implies x$ bring M from state p with any empty stack to q with empty stack</u>

Suppose that $A_{p,q}$ derives x. We will show that x can bring M from state p with an empty stack to state q with an empty stack.
We use induction on the number of steps required to derive $A_{p,q}$.

<u>Base case</u>:
when the number of steps required to derive $A_{p,q}$ is 1, we see that the only case this is possible is following the rule $A_{p,p} \to \epsilon$. (For all the other derivation rules, there is at least one other variable on the right hand side of the rule). This implies that $p = q$ and the rule for derivation is of the form $A_{q,q} \to \epsilon$. In the automaton M, if we are at state q with an empty stack, reading in the symbol ϵ is equivalent to not reading anything. Furthermore, we do not change the stack in any way. This corresponds to an ϵ transition between state q and q, so it indeed beings empty stack to empty stack.

<u>Induction hypothesis:</u>
If $A_{p,q}$ derives x in at most k steps, then x brings M from state p with empty stack to state q with empty stack.

<u>Inductive argument:</u>
Suppose $A_{p,q}$ derives x in $k + 1$ steps. Consider the first step. Since $k \geq 2$, we know that it is not the case $A_{p,p} \to \epsilon$. So there are two cases for the first step:

- The first step is of the form $A_{p,q} \to A_{p,r}A_{r,q}$. The next k steps will derive $A_{p,r}A_{r,q}$ to x. We write $x = x_1x_2 \ldots x_n$. This means that we can partition x into $x = yz$, such that $A_{p,r}$ derives y and $A_{r,q}$ derives z. Now, by induction, each of $A_{p,r}$ deriving y and $A_{r,q}$ deriving z takes less than or equal to k steps. By induction, M brings y from state p to state r, empty stack to empty stack, and z brings M from state r to q, empty stack to empty stack. Now, we start at state p with empty stack, parsing y leads us to state r to empty stack, and we can then parse z to state q in empty stack. This implies that $x = yz$ brings p to q, empty stack to empty stack.
- The second step is of the form $A_{p,q} \to aA_{r,s}b$. This implies we can write $x = ayb$, where $A_{r,s}$ derives y in at most k steps from state r to s, empty stack to empty stack. Now, we will show we can parse x from $A_{p,q} \to x$ empty stack to empty stack.

  Starting at state p and empty stack, we parse a, pop ϵ and follow $(r, u) \in \delta(p, a, e)$. Now the stack consists of u and we are at state r. We then follow the transitions that takes from r to s by parsing y. Note that u always stays at the bottom of the stack because none of the transitions popped u off the bottom-most position of the stack. When we arrive at state s, the stack consists of only u and we follow the transition $(q, \epsilon) \in \delta(s, b, u)$ by parsing b and popping u. This allows us to arrive at state q, with an empty stack. Hence this proves the inductive argument.

By induction, if $A_{p,q}$ derives x in arbitrarily large finite steps, we know that x brings M from state p to q from empty stack to empty stack. This implies that if $A_{q_0,q_{accept}}$ a string x, then in M, we start from q_0 with empty stack, we can parse x into q_{accept} empty stack to empty stack, so M accepts x.

x bring M from state p with any empty stack to q with empty stack $\implies A_{p,q}$ derives x

Suppose that x bring M from state p with any empty stack to q with empty stack. We will show that $A_{p,q}$ derives x by inducting on the number of steps that M took in order to go from state p to q, empty stack to empty stack.

Base case:

Suppose that it takes 0 step in M to go from state p to q, empty stack to empty stack. This means that $p = q$, no symbols are pushed or popped from the stack, and that the string parsed is exactly ϵ. In this case we follow the transition $A_{p,p} \to \epsilon$.

Induction hypothesis:

If parsing x in M from p to q, empty to empty stack, in at most k steps, $A_{p,q}$ derives x.

Inductive argument:

Now, suppose parsing x in M from p to q, empty stack to empty stack, takes $k+1$ steps. There are two cases to consider.

- First case: the stack is empty at step i for some $1 < i < k+1$. Suppose that at this step, we arrive at an intermediate state r with empty stack. We can split x into $x = yz$ such that y can bring M from state p to state r with an empty stack and z bring M from state r to state q with empty stack, both with at most k steps. By induction $A_{p,r}$ derives y and $A_{r,q}$ derives z. Since we have $A_{p,q} \to A_{p,r}A_{r,q}$, we have $A_{p,q}$ deriving $yz = x$.

- Second case: throughout parsing x, the only time that the stack is empty is at before step 1 and after $k+1$. We let a correspond to the parsed symbol in Σ_ϵ in the first δ transition and b correspond to the parsed symbol of Σ_ϵ in the last δ transition. Since each transition either pushes exactly one element or pops exactly one element from the stack, first transition must be $\delta(p, a, \epsilon)$. We let $(r, u) \in \delta(p, a, \epsilon)$ be the transition that M followed. Now, u is the bottom symbol in the stack.

  Since the stack is not empty after step 1 and before step $k+1$, the bottom-most u has not been popped until the $k+1^{th}$ step. Therefore, we can deduce that at the kth step, the stack contains exactly one symbol, u, and this u is popped at the $k+1$th step, such that M followed the transition $(q, \epsilon) \in \delta(s, b, u)$. Furthermore, writing $x = ayb$, M parses y from r to s with empty stack to empty stack. (As the bottom most u is present at step 1 and k, and it is never popped when M is reading y). Since this takes $k-1$ steps, by induction, $A_{r,s}$ derives the string y. By the rule $A_{p,q} \to aA_{r,s}b$, we see that $A_{p,q}$ derives $ayb = x$.

This finishes both directions of the proof.

$\square$

Note (Observations, ideas and my list of questions):

- Now we have seen that CFL are a larger set than Regular languages. Can we build something recognizes even bigger set of languages?
- Is there some finite automata that can recognize all languages? I am not sure about this because of powerset of $\mathbb{N}$ is uncountable. But what if we loosen the requirement to have infinite automata that can be described finitely?
- Note that we can identify both the above language via this: ww identified with a queue instead of a stack. $0^n1^n2^n$ can be identified with two queues instead of a stack.
- Now the question comes to: if we are able to have finite queues and multiple stacks, How about if we are able to increase number queues and stacks on the fly (arbitrarily many can be used)? What is that class of language?

4.4 Lesson 16. Chomsky Normal Form

We will introduce the notion of context free grammars in CNF (Chomsky Normal Form). CFGs can have rules that can take many possible forms. We will present CFGs in CNF, which will enable us to prove theorems in a more convenient way.

Definition (CNF):
The context-free grammar $G = (V, \Sigma, R, S)$ is in *Chomsky normal form* (or CNF) if every production rule is of the form

- $A \to BC$
- $A \to a$
- $S \to \epsilon$

for some variable $A \in V$, some terminal symbol $a \in \Sigma$ and some variables $B, C \in V \setminus \{S\}$.

Note (My questions towards CNFs):

- Given a CNF with a rules, each rule having less than b symbols. Then, what is the upper bound of rules on the CNF form?
- Since regular languages can be corresponded with NFAs, which can be expressed as a PDA, is there a way to transfer regular languages directly to CNFs? I think yes. Unions can be achieved via $A \to B \mid C$, stars can be achieved via $A \to AA$ where A generates the repeated strings, and concatenations are achieved via $A \to BC$.

Note:

- This definition is quite restrictive because every rule has exactly two variables or exactly one terminal symbol on the RHS. S can never appear in the RHS, though it is possible in the normal CFGs. ϵ can only appear on the RHS if it is the start variable.
- This means that a CFG that can be represented in one line would need multiple lines to be expressed in the CNF. For example, the following two CFGs, the first one is not in CNF and the second one is.

$$S \to 0S1 \mid 01$$

$$\begin{aligned} S &\to ZA \\ A &\to BN \mid 1 \\ B &\to ZA \\ Z &\to 0 \\ N &\to 1 \end{aligned}$$

- The CNF that generates $\{0^{2n} : n \geq 1\}$ is:
  - $S \to A$
  - $A \to AC \mid DE$
  - $C \to DE$
  - $D \to 0$
  - $E \to 0$
- The CNF that generates $(01)^*$
  - $S \to A \mid \epsilon$
  - $A \to AC \mid DE$
  - $C \to DE$
  - $D \to 0$
  - $E \to 1$
- The CNF that generates $L = \{0^m 1^n \mid m \geq n \geq 0\}$.

  Note that this is $\{0^m 1^n \mid m \geq n \geq 0\} = \{0^{m-n} 0^n 1^n \mid m \geq n \geq 0\}$. Note that cases are:
  - $n \geq 2$, $0^n 1^n$, $00^n 1^n, 0^m 1^n (m \geq 2 + n)$
  - $n = 1$, multiple ways
  - $n = 0$, multiple ways

  the biggest setback is that whenever we have a pattern repeated for once or twice, it won't be too hard. But we must account the possibility of the pattern occur 0 times. or if there is a single pattern, it might be difficult to expand it as well because $A \to CD$ expands into at least two symbols. In this case, when the string is only one letter, we need to include the possibility using $S \to a$.

Theorem (Converting CFG into CNF):
For every context-free grammar G, there is a context-free grammar G' in CNF such that $L(G) = L(G')$.

Note 4.6 (Original ideals and observations):
I noticed that in the proof, we have four steps to implement, in order to convert CFG into CNF. I think the order of applying these changes is extremely crucial. We also must prove that after fixing one item, it cannot "unfix" another item. That is, if we make the start variable not appear on the RHS, then later in step 2,3,4, we must make sure this still holds when fixing $2, 3, 4$! Otherwise it might end up in a cycle to fix things that will be unfixed later. Indeed, I had to carefully check my rules such that no rules will introduce a "glitch" introduced in the earlier rules.

Proof:
We will construct G' step by step.
We first notice that applying each rules in any time and order does not mess up the arguments.

1. Make the start variable not appear on the right hand side of any rule.

   We introduce S'. We change all instances of S to S'. We then introduce $S \to S'$. Note that though this has only one rule on the right, it can be reduced in the below rules.

2. Remove all ϵ rules from the non-start variables. Suppose we have $A \to \epsilon$. Then we delete this rule, and any rules of the form $B \to uAw$, we write it as $B \to uw \mid uAw$. We repeat this step until there is no ϵ rules from non-start variables. The only way this can go wrong is if we have a cycle. $A \to \epsilon, B \to A, C \to B, A \to C$. This will still not result in an error because we eliminated the ϵ in the first place.

3. Remove all unit rules $A \to B$. Suppose we have $B \to str$, we remove this rule, and add the rule $A \to str$. Note that each time doing this, we either decrease the unit rules by 1 or 0. The only weird case we don't eliminate all the unit rules is that if we have a cycle. Even if we have a cycle, we will obtain $A \to A$ eventually. In this case, we delete $A \to A$. We are always able to achieve the result by repetition.

4. Remove all rules that generate strings of length greater than 2. This is done by induction. We will show we are always able to do so for arbitrarily long length. For 2, we have $A = x_0x_1$. Suppose we have $A \to x_0x_1 \dots x_k$. Then, we set $A \to BC$, $B \to x_0$, $C \to x_1 \dots x_k$. Since the rule $C \to x_1 \dots x_k$ can be written in rules that generate strings less than or equal to 2, we are set.

□

4.4.1 Right regular grammar, left regular grammar, and linear grammar

Definition (Right regular language):
Every rule is of form

- $A \to aB$
- $A \to a$
- $A \to \epsilon$

Definition (Left regular language):
Every rule is of form

- $A \to Ba$
- $A \to a$
- $A \to \epsilon$

Note (Examples that I came up with):

1. Right regular $\{x \in \Sigma^*_{a,b} : x = ababa.....\}$
   - $S \to \epsilon A$
   - $A \to aB \mid \epsilon$
   - $B \to bA \mid \epsilon$
2. Right regular $\{x \in \Sigma^*_{a,b,c} : x = abcabcab.....\}$
   - $S \to \epsilon A$
   - $A \to aB \mid \epsilon$
   - $B \to bC \mid \epsilon$
   - $C \to cA \mid \epsilon$
3. $\{x \in \Sigma^*_2 : x = 1^*01^*\}$. How to you generate this with left regular language? I do not see a way!

Definition (Linear grammar language):
Every rule is of form

- $A \to aB$
- $A \to Ba$
- $A \to a$
- $A \to \epsilon$

Problem: Identify whether which, if any, of the following claims are true:

- L is regular $\iff$ L can be generated by a right regular grammar
- L is regular $\iff$ L can be generated by a left regular grammar
- L is regular $\iff$ L can be generated by a linear grammar

Note (Observations):

- I think all the above are not context free languages, because 0^n1^n cannot be generated by any of the grammar above. In fact, i think the above languages are not as strong as DFAs or NFAs or RegEx.
- Though I am unsure how to prove this, but I see right off the bat that it is difficult to express elements of the form 1(01)*1 in the left and right regular languages.

5 Week 5 Materials

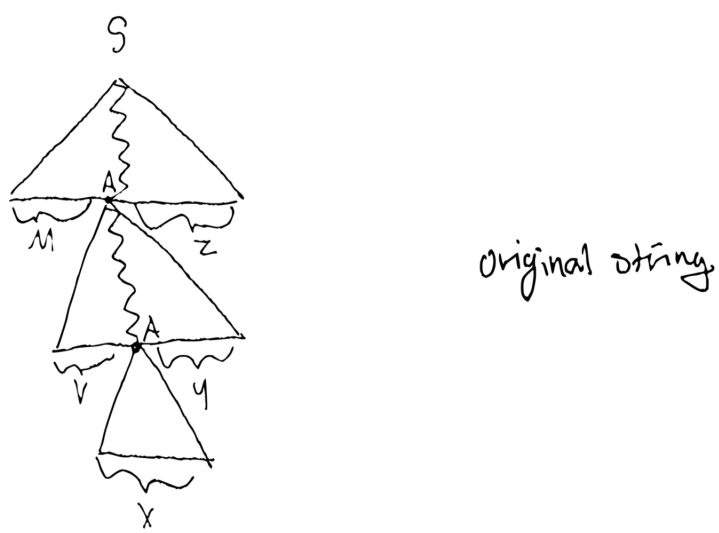
5.1 Lesson 17. Pumping Lemmas for CFLs

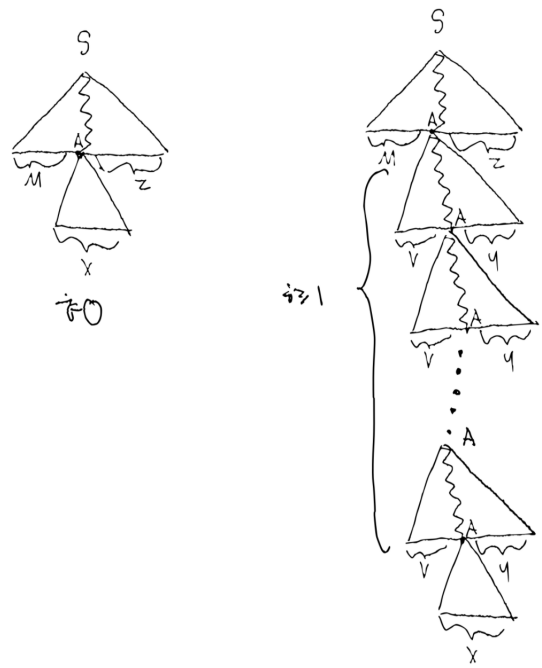
5.1.1 Finite and Infinite CFLs

While the pumping lemmas for Regular Languages are mainly considered through automata, context free language is considered through Chomsky Normal Form. Note that the strings in the finite language cannot be very long unless the CNF is very long. This brings us to the following proposition:

Proposition:
If $G = (V, \Sigma, R, S)$ is a context free grammar in Chomsky normal form with $|V| = m$ variables and G generates a string x of length $|x| \geq 2^m$, then $L(G)$ is an infinite language.

Proof:
Consider the parse tree of the string x. Note that each variable node has either two children that are both variable, or one child that is a symbol.
Since $|x| \geq 2^m$, this implies that there are 2^m leaves, so at least 2^m variables. We argue that there is at least one root-to-leaf path with length $m + 1$. To show this, suppose towards contradiction, that all root to leaf paths have length $\leq m$, then the number of variables in the tree is at most $2^m - 1$. So this is contradiction and we can find a root to leaf path with length at least $m + 1$. Since there are m variables, some variables appeared at least twice in this root to node path.
Now, we use drawing to show that the set of string $uvxyz$, where u, z are corresponding substrings generated by left, right of the first A, respectively. v, y are correspondinng substrings genrated by left, right of the second A respectively. And x be the string generated from the second A. Then we can see that for any $i \geq 0$, $uv^ixy^iz \in L$ can be generated by the same CFL as follows: (note that $|x = 0|$ is possible here!)





□

5.1.2 The pumping lemma

Lemma (The pumping lemma for Context Free Language):
For every context-free language $L \subseteq \Sigma^*$, there is a number p such that for every string $s \in L$ of length $|s| \geq p$, we can write $s = uvxyz$ where

- $|v| + |y| > 0$
- $|vxy| \leq p$
- For each $i \geq 0, uv^ixy^iz \in L$

Proof: For the proof of this theorem, we adopt the proof for the above for a string with length $p = 2^m + 1$. This way, the can consider the parse tree of string $x, |x| \geq 2^m + 1$. We consider a similar argument as above, there we select the longest root to leaf path, and travel bottom up. We let A be the first repeated variable. There must be a root to leaf path with length at least $m + 1$. Again we can separate the string as in the last proof.
Now here are the reasons why $s = uvxyz$ it satisfies the rules:

- $|v|+|y| > 0$. This is because, if $A \to BC$ is a rule for the first A, then if A appears in the subtree rooted at B, then C must not be empty, as $C \to \epsilon$ is prohibited. Vice versa. So $v + y$ must be nonempty.

- $|vxy| \leq p$. This is because A is the first repeated variable. So all other variables appeared at most once in the subtree rooted at the first A. Hence since the height of tree is at most m, the complete tree has most 2^m nodes. Adding the node in the bottom, this yields the length is at most $2^m + 1$.
- For each $i \geq 0, uv^ixy^iz \in L$. This is true by examing the parse tree above.

□

5.1.3 Using the pumping lemma

Proposition 5.1:
The language $\{0^n1^n2^n : n \geq 0\}$ is not context free.

Proof: Consider the string $s = 0^p1^p2^p \in L$. Now, for any partition of the string into $s = uvxyz$, with $|vxy| \leq p$, we would have vxy cannot contain all three symbols of $0, 1, 2$. Say $i \in \{0, 1, 2\}$ is not in the string vxy. However, $uxz \in L$, we would have this string has strictly more letters of i than the other two. Hence $uxz \notin L$. Therefore, by contrapositive of the pumping lemma, this set of language is not context free. □

Proposition 5.2:
The language $\{ww : w \in \Sigma_2^*\}$ is not context free.

Proof: Consider the string $0^p1^p2^p0^p1^p2^p \in L$. Now consider any partition of the string into $s = uvxyz$. Again, any of such decomposition vxy with length at most p contains no more than two symbols. There are 11 such possibilities. (That is, the string lies entirely within i^p, or is between adjacent two letters.) In all such cases, uv^ixy^iz with $i = 0$, yields uxz, are not in L. Therefore, by contrapositive of pumping lemma, we are done. □

Note 5.3:
Note that how $\{0^n1^n : n \geq 0\}$ is context free but not $\{0^n1^n2^n : n \geq 0\}$. Now, how about the following

- $\{0^n1^n2^n3^n : n \geq 0\}$?
- $\{0^n1^n0^n : n \geq 0\}$?
- $\{0^n1^n0^n1^n : n \geq 0\}$?

Note that all the above are not context free by the pumping lemma with the string $0^p1^p2^p3^p$, $0^p1^p0^p, 0^p1^p0^p1^p$.

Note 5.4 (Observations, ideas, and questions):
Note that we can identify both the above language via this: ww identified with a queue instead of a stack. $0^n1^n2^n$ can be identified with two queues instead of a stack.
My other observation is that, the above strings, $\{0^n1^n2^n3^n : n \geq 0\}$, $\{0^n1^n0^n : n \geq 0\}$, $\{0^n1^n0^n1^n : n \geq 0\}$ all can be identified with two stacks! The main reason why we can't build a PDA for them, is because there is only one stack. So, we were able to record 0^n1^n, but by the time we start the second 0^n1^n, we lost track of what the first n is. So generally speaking, for CFLs, the same exponent cannot appear more than twice.
Now the question comes to:

- if we are able to have finite queues and multiple stacks, is there something that can recognize all languages?
- How about if we are able to increase number queues and stacks on the fly (arbitrarily many can be used), can we have some sort of finite automaton that identifies all language?

Note 5.5 (Prove following languages are not context free):

- $L = \{0^\ell 1^m 2^n : 0 \leq \ell \leq m \leq n\}$
- $L = \{wxw^R :: w, x \in \Sigma^*, |w| = |x|\}$
- $L = \{x \in \Sigma_2^* \mid x \text{ contains 2 1s in middle third}\}$
- $L = \{x\#w : x \text{ is a substring of } w\}$. Consider string of the form $w\#w$.
- $L = \{x \in \Sigma_2^* : x = x^R \text{ and has same number of 0 and 1}\}$

  This one is particularly interesting. Because having same number of $0, 1$ and palindromes each are both CFG?
- $L = \{0^k1^n : n \text{ is a multiple of } k\}$.

Note 5.6 (Original questions):
What do you observe about the non-context free languages? I noticed that though CFG has a stack (so it has memory), it can only use the stack once. So it does not have an accumulator. If it had an accumulator, then it would be able to remember the number of ns as soon as we pop out of a stack. So $\{0^n1^n2^n : n \geq 0\}$ would be able to be recognized.
What if CFG has multiple stacks? Would that make the CFG more powerful?
Why is it that we cannot do intersections with context free language? What is the intuitive idea behind this?

5.2 Lesson 18.

5.2.1 Properties of Context-Free-Languages

Though CFL is a set that is strictly larger than regular languages, there are strict characteristics that apply to regular languages but not CFG. For example, it is not closed under intersection and complement, just as seen earlier.

Theorem 5.7 (Closed operations on context free languages):
Context free languages are closed under the following operations:

- Union
- Concatenation
- Star operation
- Reversal

Proof: Let (Σ, V_1, R_1, S_1), (Σ, V_2, R_2, S_2) be the language representing L_1, L_2.

- Union: $(\Sigma, V_1 \cup V_2 \cup \{S\}, R_1 \cup R_2 \cup (S \to R_1, S \to R_2), S)$
- Concat: $(\Sigma, V_1 \cup V_2 \cup \{S\}, R_1 \cup R_2 \cup (S \to R_1 R_2), S)$
- Star: given a PDA that recognizes L_1, we first modify it to clear stack before accepting and have only one final state. We will make start state accepting, and ϵ transition between acceptance and the start state.
- Reversal: given (Σ, V, R, S) in CNF, we change all rules of $A \to BC$ to $A \to CB$. Then we are done.

$\square$

Note 5.8 (My own questions, obversations, and comments):
Why would CFLs not be closed under intersections?
Note that regular languages are closed under intersections. To prove this, we used a new DFA that has its states as the cross product of the states from the two DFAs that recognize each of the languages. However, we cannot do that to PDAs, because that would need us to use two stacks to keep track of what is going on in each of the PDAs. So this is a limitation that PDAs cannot achieve.

Theorem 5.9 (Other operations on context free languages):
Context free languages are not closed under the following operations:

- Intersection
- Complements

Proof:
<u>Main method</u>
Note that L_{pal} and L_{01} (strings have the same number of $0, 1$s) are both CFL. But their intersection is not CFL by pumping lemma to $0^p 1^{2p} 0^p$.
Now if CFL are closed under complement we would have $L_1 \cap L_2 = \overline{(\overline{L_1} \cup \overline{L_2})}$, so intersection is closed, contradiction.

Alternative method We first show CFLs are not closed under complement. Consider the union of the six context free languages:

- $\{0^i 1^j 2^k, i < j, i, j, k > 0\}$
- $\{0^i 1^j 2^k, i > j, i, j, k > 0\}$
- $\{0^i 1^j 2^k, k < j, i, j, k > 0\}$
- $\{0^i 1^j 2^k, j < k, i, j, k > 0\}$
- $\{0^i 1^j 2^k, i < k, i, j, k > 0\}$
- $\{0^i 1^j 2^k, k < i, i, j, k > 0\}$
- Complement of the strings of the form $\{0^i 1^j 2^k\}$. Note that this is regular, which is closed under complement.

The union is $\{0^i 1^j 2^k, k \neq i \text{ or } i \neq j \text{ or } j \neq k\}$ or strings not written in the form of $0^i 1^j 2^k$. Since CFL closed under union, this language is closed. Now, suppose towards contradiction that CFL is closed under complement, $0^n 1^n 2^n, n \geq 1$ will be CFL. This is contradiction.

□

5.2.2 Deterministic CFLs

We now consider the class of languages that can be recognized by deterministic pushdown automata. That is, we implement PDAs with deterministic automata instead of indeterministic. These languages are called DCFL (Deterministic context-free languages). Note that $DCFL \neq CFL$. One reason why is that $DCFL$ is actually closed under complements.

Note 5.10 (Bonus challenge):
Provide a formal definition of DPA (deterministic PDAs) and DCFLs (deterministic context free languages). Prove DCFL are closed under complementation.

Proof:
Formal definition for deterministic pushdown automata
We will essentially remove all ϵ transitions, and make the trasition relation into a function.
DCFLs are closed under complementation
Note that simply "flipping the accept state" is not good enough. We need to avoid possible ϵ loops. □

Theorem 5.11:
The classes of deterministic and standard context free language strictly satisfies

$$DCFL \subsetneq CFL$$

Proof:
$\subset$ can be proven by modifying a $DCFL$ PDA to a normal PDA.
$\subsetneq$ can be proven because at least one of the following CFL is not in $DCFL$.

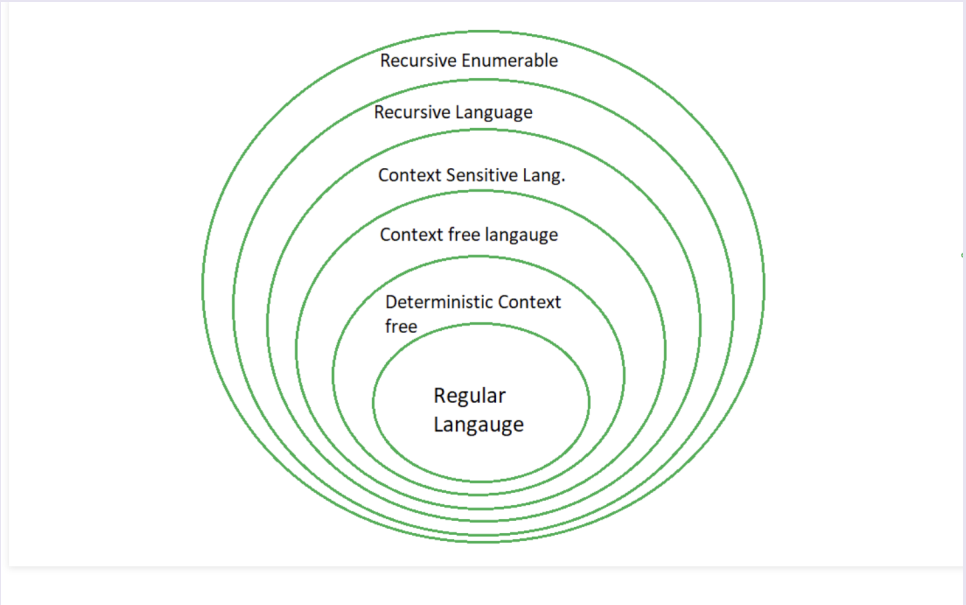
- $\{0^i 1^j 2^k, i < j, i, j, k > 0\}$
- $\{0^i 1^j 2^k, i > j, i, j, k > 0\}$
- $\{0^i 1^j 2^k, k < j, i, j, k > 0\}$
- $\{0^i 1^j 2^k, j < k, i, j, k > 0\}$
- $\{0^i 1^j 2^k, i < k, i, j, k > 0\}$
- $\{0^i 1^j 2^k, k < i, i, j, k > 0\}$
- Complement of the strings of the form $\{0^i 1^j 2^k\}$. Note that this is regular, which is closed under complement.

Because if they all are DCFL, then DCFL's complement would be $\{0^n 1^n 2^n\}$, and by the fact that DCFL is closed under complementation, this is DCFL and CFL. This is not true as we have shown this is not CFL.

$\square$

Note 5.12:
Open Problem: topological separation.
Recall that in the last section, we have shown that for every regular language L whose complement $\Sigma^* \setminus L$ is infinite, there exists a regular language $L' \supseteq L$ such that L' and $\Sigma^* \setminus L'$ are both infinite. Now, whether this is true for CFLs, is still an open problem.

Note 5.13 (My own questions, obversations, and comments):
I found this image, which is really interesting. It outlines a hierarchy of languages. It would useful to know what kind of automata can identify the bigger languages: context sensitive, recursive, and etc.

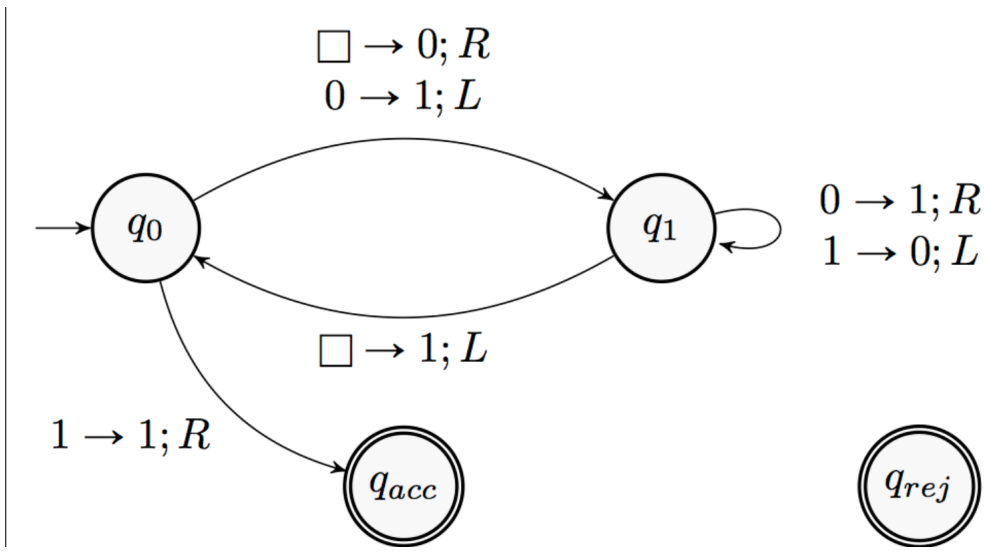


I retrieved the image from the following URL: `https://www.geeksforgeeks.org/lmn-toc/`

6 Week 6 Materials

6.1 Lesson 19

Definition 6.1 (Turing Machine):
It is a finite automaton that has access to an infinite tape, which is divided into individual cells, where each cell stores a single symbol. A turning machine only sees the content at the current tape head position, and it can only move tape head position one cell to left or right for each transition. Yet it is a more powerful model of computation compared to all the ones we learned before.



For example, the transition $A \to B$ labeled with "$a \to b; L$" means we are currently on state A, symbol at the current head is a. Then the Turing machine goes to state B, overwrites a with b, and move head one cell to the left.
Similarly with $a \to b; R$, the tape head moves right at the end of transition.
The special symbol denotes blank are cells with no content in them. So it's a blank square.

Definition 6.2 (Deterministic Turing Machine):
A Deterministic Turing Machine (DTM) is an abstract machine defined by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$$

where

- Q is a finite set of states
- Σ is the input alphabet
- Γ is the tape alphabet
- $\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$ is the transition function.
- $q_0 \in Q$ is the initial state
- $q_{acc} \in Q$ is the accept state
- $q_{rej} \in Q \setminus \{q_{acc}\}$ is the reject state.

Note 6.1:
Remarks

- To be a valid Turing machine description, the tape alphabet and the alphabet must be included in Γ.
- Whenever we hit q_{acc}, q_{rej}, the turing machine should halt. But these states are still in the domain of δ. Though technically they need to be defined, it does not matter where it gets sent to.
- How can I modify the definition of Turing machine to enforce no transition goes out of q_{acc}, q_{rej}. To do this, set $\delta : Q \setminus \{q_{acc}, q_{rej}\} \times \Gamma \to Q \times \Gamma \times \{L, R\}$.

Definition 6.3 (Configuration of a Turing Machine):
Configuration holds the information of what is on the tape.
A configuration of a Turing machine T is a string $wqy, w, y \in \Gamma^*, q \in Q$ such that

- q is the current state of the automaton
- wy is the current string on the tape
- the position of the tape head is on the first symbol in y.

The initial configuration on input x is q_0x. The configuration wqy is

- an accepting configuration if $q = q_{acc}$
- a rejection configuration if $q = q_{rej}$
- a halting configuration if it is an accepting or a rejecting configuration.

Definition 6.4 (Configuration yields another configuration):
For any string $w, y \in \Gamma^*$, symbols $a, b, c \in \Gamma$, and states $q, q' \in Q$, the configuration $waqby$ in M yields the configuration $wq'acy$, written as

$$waqby \vdash wq'acy$$

when $\delta(q, b) = (q', c, L)$ and

$$waqby \vdash wacq'y$$

when $\delta(q, b) = (q', c, R)$.

Note that since tape is infinite, we must be able to add blanks at either end of strings. We can add two definitions to yield: when the configuration is of the form $qx, x \in \Gamma^*$,, we have $qx \vdash \square qx$. When is it is of the form $xq, x \in \Gamma^*$, we have $xq \vdash xq\square$.

Note 6.2 (Question of my own):
Must each computational step of the Turing machine go left or right? Though the definition states so, I am still wondering why it must go left or right. Would it be possible for any steps to not move left or right and still change the current configuration?
The answer is no for the standard Turing Machine. This is because to change the current configuration, the tapehead must overwrite some alphabet. As soon as overwrite happens, the tapehead automatically moves. In the multi-tape TM, we do allow this to happen. If we really want to change the content of some cellls without moving the tapehead, we could do so by moving over, change the information, and moving back!

Definition 6.5:
The configuration wqy derives the configuration $w'q'y'$ in M, written

$$wqy \vdash^* w'q'y'$$

if there is a finite sequence of configuration $wqy \vdash w_1q_1y_1 \vdash \ldots \vdash w_kq_ky_k \vdash w'q'y'$

Note 6.3 (My Observations):
This is really similar to how a string yields another in the CFL! So for every PDA, there is a CFL such that recognizes the same language, and the way a CFL generates a string is exactly step by step like how the configurations work in the Turing machine!
That is, for every finite automata, there is exactly a way to generate strings in some "strings notation"! i.e. for (DFA, PDA, TM) they are (RegEx, CFL, configurations).
However, in the CFL generation, the transitions in PDA are not directly involved, but in the configuration, the transitions in the Turing machine is directly involved. We directly use the rules from the TM.

Definition 6.6:
The Turing machine M with initial state q_0

- accepts x if (q_0, x) derives an accepting configuration.
- rejects x if (q_0, x) derives a rejecting configuration.
- halts on x if it either acccepts or reject x.

Note 6.4 (My Own question):
Would it be possible for a string to be both accepted and rejected?
Answer: I think no. Because as soon as it hits either of the accepting state or rejection state, it stops. So these are exclusive situation. Furthermore, because of the deterministic nature (adding $\square$ does not change the result,) its computational path is unique.

6.1.1 Decidable languages

A turing machine decides a language when it accepts all the strings in language and rejects other strings. That is, for any string in the language, they halt.

Definition 6.7:
The Turing machine M decides $L \subseteq \Sigma^*$ if it accepts every $x \in L$ and rejects every $x \notin L$.
A language is decidable when there is a Turing machine that decides it.

Note 6.5:

- The set of decidable languages is also known as the set of recursive languages.
- Does every Turing machine decide a language? No. I think it is possible to have a Turing machine and construct a string such that the configuration would stuck somewhere and never reach either accept or reject.
- Is every regular language decidable? I think yes. We can mimic a PDA using Turing machine, by mimicking stacks with our cells.

6.1.2 Recognizable languages

Definition 6.8:
The Turing machine M recognizes the language L if it accepts every $x \in L$ and either rejects or does not halt for every $x \notin L$.
Note that this relaxes the constraint, it does not need to reject every string not in the language.
A language is recognizable when there is a Turing machine that recognizes it. For any Turing machine M, we let $L(M)$ to denote the language recognized by M.

Note 6.6:
The set of recognizable language is called the set of recursively enumerable languages.
Does every Turing machine recognize a language? Yes. We just set the language to be the set of all strings that Turing machine accepts.
Is the set of decidable languages a subset of the set of recognizable language? Yes. For any turing machine that decides a language, the same turing machine recognizes that language.

6.2 Lesson 20. Church Turing Thesis

Definition 6.9 (Church-Turing Thesis):
Any decision problem that can be solved by an algorithm on any computer, that we can construct in the universe, corresponds to a language that can be decided by a Turing machine.

Note 6.7:
We can't prove the Church Turing Thesis, and I have no idea why. This definitely seems like an advanced question to look into!

6.2.1 Turing Machines with Registers

Definition 6.10:
A register Turing machine has a finite number of registers which can each store one symbol from the tape alphabet. The transition of these register are determined by the current state, the content of tape at the current head position, and the content of all the registers.
On each transition, the register Turing machine can overwrite the content of current tape cell, as well as all the registers.

Proposition 6.8:
Every language that can be decided by a register Turing machine can also be decided by a standard Turing machine. So, having the registers does not make the machine more powerful.

Proof: We will represent the all the information of the registers as states. That is, we will embed all the possible register configurations in the states.
The high level proof is follows:
Let there be r registers and n symbols in the tape alphabet. There are total of $(1+n)^r$ different possible configurations for the register, where the extra 1 represents the register being empty. Now, we will let the set of all different possible configurations represented by C, and have $C \times Q$ to be the states of the new Turing machine. Now, for every transition in the old machine (with registers), we can encode the information into the new turing machine. That is, suppose in the register Turing machine with register configuration r, state p, and content of current head position σ, transition gives us new register configuration s, new state q, and direction $D \in \{L, R\}$. Then in the new Turing machine, we would have (r, p, σ) is transitioned to (s, q, D).

$\square$

6.2.2 Subroutine Turing Machines

Definition 6.11:
A subroutine Turing machine is a Turing machine that can call standard Turing machines to run as blackbox subroutines.
We can represent a call to a subroutine by a special type of states in the Turing Machine. After subroutine halts, all actions on the input tape will persist, where the special states will give two transitions to indicate whether it rejected or accepted.

Proposition 6.9:
Every language that can be decided by a subroutine Turing machine can also be decided by a standard Turing machine.

Proof: In here, I will present a high-level sketch proof, and I will use the fact that
Suppose we are given a subroutine turing machine.
Then we will turn it into a standard turing machine by adding symbols and a 2 spot register and have the states being the union of all the subroutines. The first element of the register indicates whether we are in subroutine and which subroutine we are in (0 indicates we are in the main routine, and $1, 2, \ldots$ indicate the corresponding subroutine). and the second register indicate the return value. Now, the transitions in the new Turing machine depends on what subroutine we are in. Therefore, we can always make sure to follow the computational path in the correct subroutine/routine.
Transitioning in the main routine to the subroutine n, we will change the first register $0 \to n$. Transitioning from the subroutine out into the main routine (that is, either it is accepted or rejected in the subroutine), we will indicate this information on register 2.
Therefore, we can use the register as a "state tracker" and yield the same outcome as if we are using a subroutine register.

□

6.2.3 Multi-tape Turing Machines

Definition 6.12:
A multitape Turing machine is a Turing machine that has $k \geq 1$ tapes instead of only one tape. It has k tapeheads, one per tape, which moves independently of one another. Similar to the standard Turing machines, the current symbol could be read and overwritten on each transition.
The transition function is of the form

$$\delta : Q \times \Gamma^k \to Q \times (\Gamma \times \{L, R, N\})^k$$

where L, R, N means left, right, no movements, respectively.
Initially, when we run a multi-tape Turing machine, the input is written in tape 1 and the other tapes are empty.

Theorem 6.10:
Every language that can be decided by a multitape Turing machine can also be decided by a standard single-tape Turing machine.

Proof:
The high level idea is that we will simulate the k tapes by only 1 tape, by "concatenating" the tapes together with indications of where each tape starts, as well as where the current head of each tape is at.
Suppose we start with configurations of k tapes: $h_1, h_2, \ldots, h_k$, where the current head is at the beginning of each of these configurations.
We would have the single tape with the tape head where they originally were $\#h_1\#h_2\#\ldots\#h_k\#$.
There are two main operations that we must implement in the single tape:

1. How to implement the transition?

   We will first "collect" the information at each tapeheads. i.e. which state it is at. To do this, we will just travel to the leftmost of the tape, to the rightmost of the tape, and decide which element in Γ^k we are. (to do this, we will need registers to tell us what steps we are, and "remember" the information

we collected so far). Now, once the "information collection" is done, (registers will tell us), we use δ to decide what to do with each tape. This will again be encoded in the registers, so we will modify and overwrite the tapehead positions tape by tape. Again the register will tell us if we are in the process of moving, or overwriting, or if we are done. At the end of this process, the tapehead returns to the start of the tape and we are done.

2. How to expand tape when we run out of space? Note that since at all time, the string on each tape is finite. So we won't run into the case when some tape becomes infinite. However, when we need to expand one of the tape in the middle, it might be difficult. The way we do it is, we use a register to indicate that we are in the middle of expanding, and another two register to r_1, r_2. So, at each position to be shifted rightward, we will write it in r_1, unload the previous symbol from r_2 to the tape, and move the r_1 to r_2 so that it can be written in the next step. To aid this process, we must create extra registers to identify "whether we are loading or unloading", "whether we are at the end of all tapes". This is tedious but easy to be implemented.

Last but now least. How do we know if we are in the middle of transition or in the middle of expanding? We will have a register that logs all this information so we can determine the current situation we are in! We will also have all the "procedures" in the register table to indicate what steps we are on! □

Note (My observations):
I observed that with a Turing machine, we can have so many powerful variants that resembles to some of the modern computer data structures and architectures we use! Especially, using registers reminds me of writing mips, and using multi tapes reminds me of using multiple stacks and queue. Now I can really believe that a Turing machine is as powerful as many computational machine we use today.

Note (My own question):
What are the other variants of TM we can do?
Can we do a TM with multiple tapes and with multiple register for each tape? The answer is yes. We can just start with the implementation of multi-tape turing machine, and apply the construction of the multiple register machine. It will work similarly.
Can we do a TM with a register with infinite memory? Or a TM with access to infinitely more many tapes?
I think we can. For register with infinite memories, we can preserve the left side of the tape to "remember" the content of the registers. Every time we have more information, we can "merge" all the information to the left to give more spaces. We can imagine it as a stack that grows leftward. Note that this construction will not work with my "cross product" construction the register TM. Similarly, we can implement arbitrarily large number of tapes, by adding ## to the right end of all tapes, to symbolize that we want to add another tape.

6.3 Lesson 21. Universal Turing Machines

Definition (The Universal Turing Machine (UTM)):
A universal Turing Machine is a Turing machine U that on any input $\langle M, x\rangle$ which encodes a Turing machine M and a string x,

- Accepts if M accepts x.

- Rejects if M rejects x.

Note 6.11 (My own ideas and observations):
Note that a universal machine is a Turing machine itself, but it accepts Turing machines and strings as inputs. Now, we make some relationships to other types of finite machines. Can we make a PDA that takes a PDA and a string as input? How and where would we encode this information? The answer is most likely not. The reason being that in the PDA, if we were to remember some information in the stack, once we pop symbols off, we would have forgotten the elements that we pop off. Hence the "memory" is very weak. Can we make a NFA that takes a NFA as an input? I think not. Because there are finite states.

We first prove that such object exists.

Theorem (Universal Turing Machine):
There exists a Universal Turning Machine.

Proof: I will give a sketch proof.
We will first give some assumptions. We assume that the Γ alphabet is fixed for the UTM. So we can just assume the description of the TM is just $Q; \delta; q_0; q_{acc}; q_{rej}$.
We will also have a register table that indicates what step of the Turing machine is on (non details about this one because it is quite tedious).

1. Have three tapes. The first one writes the description of the TM and the input string. The two other ones are blank for now.
2. The first step, we will copy the description of TM into the second tape. Then the string's initial configuration on the third tape with $q_0 x$.
3. The second step we will mimic the transition δ. We will keep on iterating this until it halts.
   (a) Check if current state equals q_{acc}, q_{rej}. If so, we accept, or reject. Otherwise we proceed. To check if current state equal q_{acc}, q_{rej}, just put the second tape head at q_{acc}, and check whether it is equal to the current tapehead in the third, similar with q_{rej}.
   (b) Then, we have $q, a \in Q \times \Gamma^*$, we search through the second tape to find out where it match. Then we proceed with overwrite, move, and the new state for the third tape. We will again need a register to write recall the steps.
   (c) Repeat step 1.

The second tape records all the information about the TM, and the third tape replicates how the TM will read the input string.

$\square$

6.3.1 Turing Completeness

Definition 6.13:
A *Turing-complete* model of computation is such that for every decidable language L, there is a machine or algorithm in this model that decides L. That is, it is basically as powerful at least as Turing machine.

Note 6.12 (My own question):
Would there be a strict superset of decidable languages? Would there be a more powerful model of computation that recognize a bigger set of language? The Church Turing thesis seems to say no.
There is an interesting discussion that talks about this in depth:
`https://cs.stackexchange.com/questions/55489/theoretical-machines-which-are-more-powerful-than-turin`

Proposition 6.13:
If a universal Turing machine can be implemented in the model of computation M, then M is Turing-complete.

Proof: Let M be a model of computation. We encode the UTM in M, as UTM_M.
Let L be an arbitrary decidable language. Let T be the TM that decides L. Then if we put T, L into UTM_M, it will tell us whether it accepts or rejects, which means it decides L. Therefore UTM_M is precisely the machine in this algorithm that decides L. □

Note 6.14:
Using the approach of Universal Turing machine, we can show many programming languages and many other models of computation (Redstone computers in Minecraft, or Conway's game of life) are Turing complete.

Note 6.15 (My Own Question):
What does it mean for a programming language to be turing complete?
A: you can implement a TM in the language.
Why is it not data structures that can be used to help accept decidable languages? Or does it mean that we can write out data structures in those programming languages that can differentiate decidable languages?
A: I think we can identify a list of data structures D={ strings, numbers, stacks, queues, tables, etc} such that any languages that can implement all D is TC.

Note 6.16:
Challenge: Identify a model of computation that is Turing-complete.
Here is an interesting URL:
`https://beza1e1.tuxen.de/articles/accidentally_turing_complete.html`
It turns out the following (surprising) models of computations are TC:

- Microsoft Excel
- Microsoft Powerpoint
- Minesweeper
- Minecraft
- LaTeX
- Functional programming languages

The one that surprised me the most is Minesweeper.

6.4 Lesson 22. Non-deterministic Turing Machines

Now we can also consider about non-deterministic Turing Machines.

Definition 6.14 (Nondeterministic Turing Machine):
It is an abstract machine defined by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$$

such that

- Q is a finite set of states
- Σ is the input alphabet
- Γ is the tape alphabet
- $\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\})$ is the transition relation
- $q_0 \in Q$ is the initial state
- $q_{acc} \in Q$ is the accept state
- $q_{rej} \in Q \setminus \{q_{acc}\}$ is the reject state

All the other notions of non-determinism also carries over to the deterministic Turing machine.

Definition:
The nondeterministic Turing machine M decides $L \subseteq \Sigma^*$ if and only if

- for every $x \in L$, there exists a computational path that accepts x and all computational path halt

- for every $x \notin L$, all computational paths reject x.

Note that from this definition, a nondeterministic Turing machine that decides a language will always halt on any string.

Note 6.17 (Observations):
Compare this to the NFA. Note that for an NFA, we accept is there is one computational path that leads to acceptance. But in a NTM, every string, either all paths halts to accept or to reject. This is interesting.
This reminds me of ∀FAs that showed up in one of the previous homework assignments. It is equally powerful as DFA and NFAs. So "if we accept it if one of computational path accepts" vs "if we accept it if all computaional path accepts" makes no difference. Only the ability of the automata makes a difference.
I also realize that "decides" and "recognizes" are different because there are three types of states in a TM while only two types in NFA or DFA.
I am curious, if we have a 2-TM, that is, out of all computational paths, at least two accepts. Would the 2-TM be equally as powerful?

Definition 6.15 (NTM recognizes a string):
The nondeterministic Turing Machine M recognizes $L \subseteq \Sigma^*$ if and only if

- for every $x \in L$, there exists a computational path that accepts x
- for every $x \notin L$, no computational path accepts x.

For the NTM to recognize a language, it is not guaranteed to halt.
The Equivalence of DTMs and NTMs
DTMs and NTMs turn out to be equivalent for their decidability.

Theorem (Equivalence of DTMs and NTMs):
The set of languages that can be decided by DTM is the exact same as the set of languages can be decided by TMs.

Proof:
Let L be decided by some DTM D. Want to show L can be decided by NTM N.
Given D, we will construct N the following:

- $Q, \Sigma, \Gamma, q_0, q_{acc}, q_{rej}$ stay the same
- N's transition relation is δ_N and D's transition function is δ_D. We will set $\delta_N(q, \gamma) = \{\delta_D(q, \gamma)\}$.

Now, if $x \in L$, by configuration in D, we will also have N accepts x.
If $x \notin L$, by configuration in D, we will also have N reject it. Note that this is made much easier because D always halts on the string, and by how the transition relation is defined on N, it will run exactly the same on N.
Let L be decided by some NTM N. Want to show L can be decided by DTM D.

Let a language L be decided by NTM N.
Let L be recognized by some NTM N. Want to show L can be recognized by DTM D.
The basic idea is to realize that given $x \in \Sigma^*$, the set of all computational paths can be realized as a tree. We will use a DTM to minmic a DFS algorithm that exhaust all the possible computational paths for x. That is, at each point that we have a decision between multiple transitions, we recurse into the first transition. If we accept, $x \in L$. If we reject, we backtrack and try the second available transition.
We will have two tapes and a register table. The first tape will mimic a stack which remembers the computational path, the second tape will be exactly the corresponding configurations. The register table will tell us whether we are moving forwards or we are in the middle of backtracing.
There are three main points of the construction of the DTM.

1. Moving forwards: if we are at transitions that does not have a decision, we just follow the configuration on the second tape, and write down the transition we took on the first tape, and move forwards.
2. Decide what to do:
   - If state is accept, then accept.
   - If state is reject, move onto the backtracing step.
   - If we run out of choices after backtracing all options, reject.
   - If there are n elements in the transition relation, put $\#1$ on the second tape to indicate the choice of first relation, and proceed with the first relation.
3. Backtracing:

   We just pop elements from the first tape and reverse the tape configuration on the bottom tape, i.e. "unoverwrite" and "go opposite directions". Until we see the first $\#i$, and if there are n choices for this transition, we move onto the $i+1$ choices. If $i = n$, we keep on backtracing.

At the end, we know if there is one accept state, we are all set. If not, by our algorithm, it will eventually go back to the beginning, as all path halts in finite time.

□

Bonus

Theorem (Equivalence of DTMs and NTMs (recognizability)):
The set of languages that can be recognized by DTM is the exact same as the set of languages can be recognized by TMs.

Proof:
In here, we will provide sketch proofs instead of full length proofs.
Let L be recognized by some DTM D. Want to show L can be recognized by NTM N.
Given D, we will construct N the following:

- $Q, \Sigma, \Gamma, q_0, q_{acc}, q_{rej}$ stay the same
- N's transition relation is δ_N and D's transition function is δ_D. We will set $\delta_N(q, \gamma) = \{\delta_D(q, \gamma)\}$.

Now, if $x \in L$, by configuration in D, we will also have N accepts x.
If $x \notin L$, by configuration in D, we will also have N reject it. Note that this is made much easier because D always halts on the string, and by how the transition relation is defined on N, it will run exactly the same on N.

Let L be recognized by some NTM N. Want to show L can be recognized by DTM D.

The basic idea is to realize that given $x \in \Sigma^*$, the set of all computational paths can be realized as a tree. We will use a DTM to minmic a DFS algorithm that exhaust all the possible computational paths for x. That is, at each point that we have a decision between multiple transitions, we recurse into the first transition. If we accept, $x \in L$. If we reject, we backtrack and try the second available transition.

We will have two tapes and a register table. The first tape will mimic a stack which remembers the computational path, the second tape will be exactly the corresponding configurations. The register table will tell us whether we are moving forwards or we are in the middle of backtracing.

There are three main points of the construction of the DTM.

1. Moving forwards: if we are at transitions that does not have a decision, we just follow the configuration on the second tape, and write down the transition we took on the first tape, and move forwards.
2. Decide what to do:
   - If state is accept, then accept.
   - If state is reject, move onto the backtracing step.
   - If we run out of choices after backtracing all options, reject.
   - If there are n elements in the transition relation, put #1 on the second tape to indicate the choice of first relation, and proceed with the first relation.
3. Backtracing:

   We just pop elements from the first tape and reverse the tape configuration on the bottom tape, i.e. "unoverwrite" and "go opposite directions". Until we see the first $\#i$, and if there are n choices for this transition, we move onto the $i+1$ choices. If $i = n$, we keep on backtracing.

At the end, we know if there is one accept state, we are all set. If not, by our algorithm, it will eventually go back to the beginning, as all path halts in finite time.

$\square$

Note 6.18:
One note that I must clarify: I am actually unsure that all the paths halt. I originally thought some sort of pigeonhole principal would prove it. But I could not show that. This is the only difference between the proof of the bonus versus the proof of the theorem before the bonus.

7 Week 7

Note 7.1 (My own observations):

- What if we have a recognizable language that is not decidable? That means for every TM that recognizes this language, it must not halt on some strings! Otherwise it will be decidable. What does it mean by it must not halt on some strings? This could be a very complicated problem.
- This is regarding section 2: Church Turing Thesis states that any decision problems that can be solved on any algorithm on any computers corresponds to a decidable language. We have shown that recognizable language is a strict superset of decidable language. This mean that there are exists recognizable languages (that are not decidable) can correspond to some problems that cannot be solved by an algorithm or computer! So the recognizable langauges are strictly more powerful than modern day computers!

7.1 Lesson 23. Decidable Languages

We ask the question: What are some languages that can not be decided by Turing machines?

Theorem (Every context-free language is decidable):
Every context-free language is decidable

Proof:
Given CFL in CNF form.
We will write an algorithm to determine this. Let $L = |X|$. Consider all the possibilities of length $2L$ derivations. Each step there is exactly one extra terminal symbol, or one extra variable symbol. Hence, there exists a derivation at most $2L$ steps. Since there are finite possibilities of length $2L$ derivations, this always halts and check all possibilities of derivations. By the Church Turing Thesis, since we can write a program to determine if $x \in L$, then it is decidable. □

Proposition:
The language $L = \{0^n 1^n 2^n, n \geq 0\}$ is decidable.

Proof: Note that it cannot be recognized by a PDA!
But we can build a TM to recognize it.
Here is a high level description of the machine:

- First move the tapehead to the right of the string without changing anything, and make sure the string is in the form of $0^n 1^n 2^n$.
- Let tapehead travel back to point at the first symbol of the string.
- Overwrite the first 0 at $*$, go to the right, overwrite the first 1 as $*$, and go to the right, overwrite the first 2 as $*$.
- Now go back to beginning, repeat the process. If any run is unsuccessful, (i.e. erased a 0, unable to

erase the 1 or 2, or erased $0, 1$ unable to erase 2,) then reject.

- When there are no more 0, check no more $1, 2$. Then return accept, and otherwise reject.

$\square$

The above shows that $CFL \subsetneq DL$.

7.1.1 Clique

Now we can look at more complicated classes of languages.

Proposition (L_{CLIQUE} is decidable.):
The language L_{CLIQUE} consisting of the encoding of every graph G on $n \geq 1$ vertices that contains a clique of size at least $n/2$ is decidable.

Proof: All we need is to guarantee that the algorithm terminates after finite steps. We can also use the following fact:
Once you provide a pseudocode for an algorithm the decides L, by the Church Turing Thesis, there must exist a Turing machine that decides L.
The algorithm is described as follows:
Enumerate all $\lfloor n/2 \rfloor$ subsets of V. For each subset S of V, enumerate $S \times S$. Then write a nested loop to check if every two points $(p, q) \in S \times S$ is connected. If so, accept, otherwise, move onto the next cross product of the subsets. Eventually, reject. $\square$

7.1.2 Accepting DFAs

Proposition:
Let A_{DFA} be the language consisting of all encodings of (M, x), where M is a DFA and x is a string accepted by M. Then, A_{DFA} is decidable.

Proof: Proof sketch:

1. First, we can use a TM to encode M as a TM. We let the alphabets stay the same. The states be $Q \cup \{q_{acc}, q_{rej}\}$ where q_{acc}, q_{rej} are new states, and the acceptance and rejection states for the TM. Let q_0 stay the same. The tape alphabets are $\Sigma \cup \{\square\}$. The transition function be the same, but always to the right. And another transition function: for every accepting state q in M, we have $\delta(q, \square) = q_{acc}, \square, R$ and for every state not accepted in M, we have $\delta(q, \square) = q_{rej}, \square, R$. This guarantees that there is only one rec/acc state in the TM. This process always halts.
2. Then, we feed the TM and x into the Universal TM, which decides A_{DFA}.

Note that we are able to put two TMs together: that is, once the first TM is done, its tape output can be used as the tape input for the next TM. The second TM would return accept or reject.

$\square$

7.1.3 Bonus: Closure Properties

Proposition:
The class of decidable languages is closed under:

- Union
- Intersection
- Complementationn
- Set difference
- Star operations
- Reversal

Proof:

- Union: Use a NTM that have q_0 goes to both the initial state of the TMs (TMs realized as a NTM) that decides the respective languages. Now, if $x \in L_1 \cup L_2$, one of the computational paths will accept. We also know that both side always halts. If $x \notin L_1 \cup L_2$ the computational paths reject. Therefore, this machine decides the language $L_1 \cup L_2$.
- Intersection: $A \cap B = \overline{(\overline{A} \cup \overline{B})}$.
- Complementation: use a DTM and reverse the rej/acc state.
- Set difference: $A \setminus B = A \cap \overline{B}$
- Star operations: consider all the partitions of the string. We accept empty string. Check if there is a partition such that all the substrings are accepted. If so, then accept. otherwise reject. The reason why it always halts is that the number of partitions of a string is finite.
- Reversal: use a TM as preprocessor to reverse the string on the tape first.

□

7.2 Lesson 24. Undecidable Languages

Now we ask the question: What are some languages that cannot be decided by Turing Machines?

7.2.1 Existence

Theorem (Existence There exists undecidable languages):
There exists undecidable languages

Proof: Observe the description of a Turing machine. Note that the number of languages it can decide is at most the combination of all possible descriptions. Each Turing Machine decides at most one language. The

cardinality of all turing machines is a sum of countable cartesian products of $\mathbb{N}$, hence countable. Therefore, since there are uncountable languages, there exists a language that cannot be decided by a Turing machine.$\square$

Note (Relating this to some of the pure math concepts I have learned):
Consider $\mathbb{R}$. We have that open, closed sets, the borel set, and the measurable sets. But none of them can actually get to the powerset of $\mathbb{R}$.

Now, consider Σ_2^*. We have regular languages, context free languages, DCFL, decidable languages, and recognizable languages. None of which actually reach the powerset of Σ_2^*.

This is kind of similar. I wonder what type of computational model you need to use in order to recognize a bigger set? And how hard do you need to go in order to identify the elements in the powerset, but not in the most powerful tool we have? For example, Rice's theorem gives us the accepting language is not decidable. This is a much more complicated language than 0^n1^n or palindromes, which are the langauges that CFL can't recognize. Similarly, when learning Lebesgue integrals, the Vitali set is not measurable, but constructing such set requires the axiom of choice, which means this set is still really hard to get! I wont if there is any sort of general correlation between these ideas!

7.2.2 First Undecidable Language

Definition 7.1 (The accepting language):

$$A_{TM} = \{\langle M, x\rangle : M \text{ is a TM that accepts } x\}$$

This includes the encodings of all pairs of $\langle M, x\rangle$ such that M is a Turing machine and x is any string that M accepts.

Note 7.2:

- To be precise, how would we encode a TM M and the pair (M, x)? I would have some special symbols not used in M, x. I would encode all alphabets as is. I would use # as dividers, and encode all the states using decimal numbers. I would go

$$\#\underbrace{1, 2, \ldots 20}_{\text{states}}\#\underbrace{a, b, \ldots, z, aa, bb, cc, \ldots}_{\text{alphabets}}\#\underbrace{a, b, \ldots, z, A, B, \ldots, Z, AA, BB, CC, \ldots, \square}_{\text{tape alphabets}},$$

$$\#\underbrace{1\$a\$2\$b\$R\$}_{\text{one encoding}}\ldots\#\underbrace{5}_{q_0}\#\underbrace{6}_{q_{acc}}\#\underbrace{10}_{q_{rej}}\#\underbrace{x}_{\text{encoded in correct alphabet}}$$

- Why do you think we choose A_{TM} to be the first language that we show to be undecidable?

  Answer: I think because this is the easiest counterexample, and in some sense this is a contradiction from the non-existence of "something". It also serves as a base case to many other undecidable languages. Or, we can say that other languages representing hard computational problems will reduce to this one, which is already proven undecidable. This reminds me of the halting problem.

Theorem:
The language A_{TM} is undecidable.

Proof: We use a similar construction as Cantor's diagonalization proof.
Suppose towards contradiction that A_{TM} is decidable by a machine A.
There are countably infinite many Turing machines. We enumerate the turing machines as $M_1, M_2, \ldots$.
There are countably infinite many strings Σ_2^*. We enumerate the strings as $x_1, x_2, \ldots$
We will make a $\mathbb{N} \times \mathbb{N}$ table, where the rows represents the TMs, columns represent the strings. Since M exists, we can determine a machine D such that if $\langle M, x\rangle$ produce accept, D rejects. If $\langle M, x\rangle$ rejects, D accepts.
Now, consider D on the list. $D = M_j$ for some $j \in \mathbb{N}$. Consider x_j. If D accepts x_j, then $\langle D, x_j\rangle$ is accepted by M and so x_j rejected by D. The opposite direction is analogues.
Hence, either way, we yield a contradiction.

□

7.3 Lesson 25. More Undecidable languages

Note 7.3:
Now, we have shown one language is undecidable. We will use this one to show that other languages are undecidable using reductions.

7.3.1 Halting language

Definition 7.2 (The halting language):
The halting language is

$$H_{TM} = \{\langle M, x\rangle \mid M \text{ halts on input } x\}$$

Theorem 7.4:
H_{TM} is undecidable.
Suppose it is decidable. Let H decide H_{TM}. Now, we propose the following machine A decides A_{TM}.

- $A(\langle M, x\rangle)$: If $H(\langle M, x\rangle)$ is rejected, then M does not halt on x. So $A(\langle M, x\rangle)$ returns reject, as M does not accept x.
- $A(\langle M, x\rangle)$: If $H(\langle M, x\rangle)$ is accepted, then we can use a UTM that takes M, x and reject or accept. The UTM always halts because M always halt on x.

Hence we have a TM that decides A_{TM}. This is a contradiction hence H_{TM} is not decidable.

Note 7.5 (My own question):
So, suppose we are given a language L, and a TM M, and there exists some strings $x \in L$ where M does not halt on the input x. Then, would we have some TM that halts for every $x \in L$? and do we always have a TM that halts on L?
Answer: If the language L is decidable, we will always find a TM that halts. If the language L is not recognizable, and not decidable, then this means for all TMs, there exists some $x \in L$ such that x does not halt in that TM.

7.3.2 Emptiness

Definition (The Empty language):
The $EMPTY_{TM}$ language is

$$EMPTY_{TM} = \{\langle M \rangle : L(M) = \varnothing\}$$

Theorem:
The language $EMPTY_{TM}$ is undecidable.

Proof: Suppose towards contradiction, it is decidable, so we let E decide $EMPTY_{TM}$.
Now we will make a TM A that decides A_{TM}. Now, consider an input $\langle M, x \rangle$.
We can make a TM M_x that decides the language $L_x = \{x\}$. Now, since intersection of two languages are decidable, we can make a TM $L_{x,M}$ inside this that accepts $L_x \cap L(M)$.
Now, we input $L_{x,M}$ in E.

- If $\langle L_{x,M} \rangle$ accepts, so $L_x \cap L(M) = \varnothing$. So $x \notin M$. So A rejects.
- If $\langle L_{x,M} \rangle$ rejects, so $L_x \cap L(M) = x$. So $x \in M$. So A accepts.

Therefore, we can implement A that decides A_{TM}, from combining multiple TMs that achieve the above procedures. Therefore, we reach a contradiction.

□

7.3.3 Equality

Definition (Equality Language):
The EQ_{TM} language is

$$EQ_{TM} = \{\langle M_1, M_2 \rangle : L(M_1) = L(M_2)\}$$

Theorem:
The language EQ_{TM} is undecidable.

Proof:
Suppose towards a contradiction, that EQ_{TM} is decidable by a TM E. Then we can construct a TM F that decides $EMPTY_{TM}$.
Let M be the input. Consider M_0 which is a TM that has $q_0 = q_{rej}$. So M_0 decides the language $L = \varnothing$.
So, we construct F as follows

1. If E reject $\langle M, M_0 \rangle$, then F reject. So M decides the empty language.
2. If E accept $\langle M, M_0 \rangle$, then F accept. So the language that M decides is not empty.

So F decides $EMPTY_{TM}$. □

7.4 Lesson 26. Rice's Theorem

Definition 7.3 (Property):
A property of a recognizable language is simply a subset of the set of all recognizable language.

Note (My own question):
Q: Why do we do define properties for recognizable languages but for decidable languages?
A: I think it is for two reasons. First, decidable languages are a subset of a recognizable language. Hence a property of recognizable would automatically be property of decidable. Another reason is that if we switch the statement to Rice's theorem to decidable language, to see if it we have the description of a property $L_p = \{\langle M \rangle : M \text{ decides some } L \in P\}$, then we can always use a TM to solve this question!

Definition 7.4 (non-trivial property of recognizable language):
A property of recognizable language is non-trivial iif it is not the empty set or the set of all recognizable languages.

Note 7.6:
Equivalently, the property P of recognizable language is non-trivial if and only if

- There exists a TM M_1 such that $L(M_1) \in P$
- There exists a TM M_2 such that $L(M_2) \notin P$

Note that $L(M)$ is the recognizable by M. NOT decidable.

Note that properties of recognizable language can in turn be expressed as languages:

Definition 7.5:
For every Turing machine language property P, let

$$L_p = \{\langle M \rangle : L(M) \in P\}$$

denote the language of all encodings of Turing machine that recognizes a language in P.
Basically, all the TMs such that the recognizable languages defined for that TM, in a specific subset

of all Rec Languge.

7.4.1 Statement of Rices Theorem

Theorem 7.7 (Rice's Theorem):
For every non-trivial property P of recognizable language, the language L_P is undecidable.

Proof: It is useful to break it down into multiple steps.

1. Suppose towards a contradiction, we have a TM T_P such that T_p accepts $\langle M \rangle \iff L(M) \in P$. where M is the encoding of TM.
2. Now, we will make a Turing machine that decides A_{TM}. Given input $\langle M, x \rangle$, we will create a machine M' such that $L(M') \in P \iff M$ accepts x. This way we see that M accepts x if and only if T_p accepts M'.
3. WLOG, $E \notin P$, where E is the empty language. (Take $\overline{P}$ if necessary.)
4. Since $P \neq \varnothing$, (non-trivial,) there exists some string $w \in L_0 \in P$ and a TM R such that $L(R) = L_0$.
5. Now, on input $\langle M, x \rangle$, we will create a machine M' such that

$$L(M') = \begin{cases} \varnothing & \text{if M not accept x} \\ L(R) & \text{if M accept x} \end{cases}$$

6. To construct such machine M', we take the input $\langle M, x \rangle$. We first run the UTM on it. If it accepts, it has an $\epsilon-$ transition to the TM R.

   If UTM does not halt on $\langle M, x \rangle$, then don't worry about it.

   We modify the reject state in the following way: For the reject state, we have an $\epsilon-$transition to q_0. And we remove the reject state, and make a new reject state that is not reachable from any states. Therefore, we make the machine M' loop forever for the strings such that M reject on x.

   Therefore, if M accepts x, $L(M')$ is empty. Otherwise it is $L(R)$ which is not empty. Therefore, we can recognize A_{TM}.

   Note that we can implement ϵ transitions in TM easily by adding a transition after another to move left and right.

$\square$

8 Week 8

8.1 Lesson 27. Recognizability

Note 8.1:

- Prove every decidable language is recognizable
- Proof: Given L, if M decides L, then M recognzies L
- Prove there exists unrecognizable language:
- Proof: it is analogous as before. Every TM recognizes at most one language and there is countable number of TM

Theorem:
The language A_{TM} is recognizable.

Proof: Recall that this language is undecidable. So, recognizable language is a strict superset of the decidable languages.
Now we prove this language is recognizable. That is, there exists a TM such that given the input $\langle M, x\rangle$, TM accepts if and only if M recognizes x. We will use a UTM to simulate M on x.
Note that $\langle M, x\rangle \in A_{TM} \iff M$ recognizes $x \iff$ the UTM simulating M, given the input x, accepts. Hence we proven that the UTM recognizes A_{TM}.

□

8.1.1 Decidability and Recognizability

Theorem 8.2:
A language L is decidable if and only if both L and its complement are both recognizable.

Proof: $\implies$ Suppose L is decidable by a DTM U. It is obviously recognizable by U. Then, every $x \notin L$, U rejects it. We can build another TM $\overline{U}$ such that it accepts iff U rejects. We know that every string in L, $\overline{U}$ rejects, and every string in $\overline{L}$, $\overline{U}$ accepts. We are done.,
$\impliedby$ Let L be a language such that both L and its complement are recognizable. Let M be the TM that recognizes L and $\overline{M}$ recognizes $\overline{L}$.
Then, we can use a UTM to create a model N that simulates the simultaneous action of M on x and $\overline{M}$ on x. (For example, use a double tape and alternate the TM on each string per term). One of them is guaranteed to halt. If M accepts x, accept. If it reject, then reject. If $\overline{M}$ accepts x, reject, otherwise accept.

□

Definition (Corecognizable language):
A Corecognizable language is a language such that its complement is recognizable. The last theorem shows that the intersection of recognizable and corecognizable languages must be the set of decidable languages.

Theorem 8.3:
The language $\overline{A_{TM}}$ is not recognizable.

Proof: Suppose towards contradiction, $\overline{A_{TM}}$ is recognizable, and we know A_{TM} is recognizable, by previous theorem A_{TM} would have been decidable. Therefore, it is not recognizable. □

Note 8.4 (Some of my own questions regarding this section):

- Turing machines are very powerful, and the three largest classes we know are decidable, recognizable, and corecognizable. Note that they are either countable or their complements are countable. My question is, is there a class of language such that itself and the complement are both uncountable, and it also corresponds to some sort of model computation?

8.2 Lesson 28 TIME class

In this part of the course, we only consider Turing Machines that halt on all inputs.
We have a *global* measure of time cost of a Turing machine.

Definition:
The time cost of the Turing machine M on input x is the number of transitions it follows before it halts.

The (worst-case) time cost of the Turing machine M is the function $t : \mathbb{N} \to \mathbb{N}$ where $t(n)$ is the maximum time cost of M on any input x of length $|x| = n$.

Note 8.5:
Question: could we simplify the time cost to be a single number instead of a function? Why or why not?
My answer is that this will be very difficult. The worst time cost is really good because it captures all information in a $2\mathbb{N}$ array and it captures the relationships between input and time cost.

So, the time cost measures the efficiency of individual Turing machines. This brings us the next definition.

Definition 8.1:
For every function $t : \mathbb{N} \to \mathbb{N}$, the time complexity class $TIME(t)$ is the set of all languages that can

be decided by a multitape Turing machine with worst cast time cost bounded above by $O(t)$.

Note 8.6:
Bonus: We see that the above definition uses multi-tape Turing machines. If we let $TIME_1(t)$ be the analogous definition if we only consider single tape TMs. Does $TIME(t) = TIME_1(t)$ hold for all t?
I think yes, if we have n tapes, we can create symbols such that each symbol consist of n symbols, the ith symbol represent what is on the ith tape.

8.2.1 First examples

Definition 8.2:
The class $TIME(n)$ is the set of languages that can be computed by linear time Turing machines. This class is interesting. In this case, the function we consider is $n : \mathbb{N} \to \mathbb{N}$, $n(k) = k$. Hence the linear function.

Proposition 8.7:
The class $TIME(n)$ is a strict superset of the class of all regular languages.

Proof:
Let L be a regular language and let N be an NFA that recognizes L. We can use a TM that mimics N, where each transition is to the right, and the states are modified accordingly. (Converting N to a TM takes a fixed amount of time). Then, given a string x with length n, it takes n transitions in the DFA so it also takes n transitions in the TM to finish parsing. Then it takes $O(1)$ time to check whether it landed on the accepting state. Therefore, $TIME(n)$ is a super class of all regular languages. Note that $0^n1^n2^n$ is not a regular language but can be recognzied in $O(n)$ time, considering the triple tape configuration of Turing machine that recognizes it.

$\square$

Definition 8.3:
Consider languages in the sublinear time complexity class. There is an interesting challenge below describing the languages in sublinear time complexity.

Note 8.8:
Describe in words the class $TIME(1)$ of languages that can be decided by constant time Turing machines.
Consider $O(1)$. This means there exists $n_0, \forall n \geq n_0$, we have the run time to decide strings with length n are always at most c. Therefore, it always reads at most the first c symbols in the string.
Therefore, the class $TIME(1)$ is set of languages such that there exists some $k \in \mathbb{N}$ such that the first k symbols share a certain property.

Proposition 8.9:
For every $t \in o(n)$, we have $TIME(t) = TIME(1)$.

Proof:
The main idea is that for every $t \in o(n)$, the Turing machine must not read the entire string, hence the Turing machine must not know the exact length of the string. It turns out the the Turing machine must have either accepted or rejected the string before fully reading it.
Therefore, there must be a fixed constant c such that the Turing machine is guaranteed to end before reading this amount of information.
Now, suppose for contradiction, we have a function $t \in o(n)$. Then, we will show that it must be able to be decided by constant time. Say we fix a function in $o(n)$, for example, $\sqrt{n}$.
We will show that $t \notin \Theta(\sqrt{n})$. Suppose towards contradiction $t(n) \in \Theta(\sqrt{n})$, then exists $c > 0, n_0 > 0$ such that $\sqrt{n} \leq c * t(n)$ for all $n \geq n_0$. So in other words $\sqrt{n}/c \leq t(n)$. But in this case we cannot differentiate the two TMs running on different length of strings with a same prefix. It must not finish reading both strings. (for example, strings with $|x| = 100, |y| = 10000$ where the first 100 letters of them are equal, we say it reads at most 10 steps on x, but then it must also accept the string of length 10000.)
The above is just a sketch proof, not a rigorous proof, but I think the arguments make sense.
Therefore, $t(n) \in o(\sqrt{n})$, and for every function f, $t \in O(f(n)), t \in o(f(n))$ other than the constant function.

$\square$

8.2.2 The Linear Speedup Theorem

Now we ask the question, why do we define $TIME(t)$ to be the languages that can be decided in time $O(t)$ instead of just the function t itself? The answer is that both answers are equivalent, that is, if a TM decides L in $1000t(n)$, so there is a TM that decides L in $t(n)$, so is there one decides it in $\frac{1}{1000}t(n)$. So on and so on.

Theorem (The linear speedup theorem):
For every constant $\epsilon > 0$, if there is a multitape Turing machine M that decides L with time cost $t(n)$, then there is also another multitape Turing Machine that M' that decides L, and it has the time cost of $\epsilon \cdot t(n) + n + 2$.

Proof:
Suppose the multitape Turing machine M is represented by the below seven tuples: Q (states), Σ (input alphabet), Γ (tape alphabets), δ (transition function for multitape TM), q_0, q_{acc}, q_{rej} (initial, accept, and reject state.)
The idea is that we will make it possible to represent a string of n symbols in the new symbol. That is, the new symbol is the set $\Sigma + \Sigma^n$, where we will choose the n later. The new turing machine N will have twice as many tapes.
We will now define the new TM N:

1. both the string and the tape alphabets are added by the set $(\Sigma \cup \square)^n$.
2. The initial setup is that we copy each of the old tape into the new tape, except this time we put n symbols at once. For example, if what is on the tape is $abccbaa$ with $n = 4$ then we have to copy this $(abcc), (baa\square)$ onto the new tape.

3. Q_N the states are of the form that represents transitions from old alphabets into the states, but we need to modify it to correspond to the alphabets of the form Σ^n. This way, we can read symbols of the form Σ^n and do the transition that "covers" this transition at once, where it would have normally caused three transitions.
4. The q_0, q_{acc}, q_{rej} are also determined corresponding to the original setup of M.
5. We have register to keep track where the tape head is pointing to within the symbol. i.e. $(abcba), n = 5$, the register says 3, so it is pointing to the $3rd$ position.

Now, we will compute the time for N to decide strings. Let $n = \frac{8}{\epsilon}$. It takes at most $n + 1$ time to read the original tape alphabet, and convert into the Σ^n alphabets, which includes reading until the grid to see if it is blank. Then, for each transition of the turing machine it might read the left, and the right block, and it might overwrite the leftblock, or the rightblock. This is at most L, R, R, L, L, R, R, L, 8 steps.
Then, for each string of length n, we will take $n + 1 + t(n) * 8/n = n + 1 + \epsilon t(n)$ time to decide it. $\square$

Note 8.10 (My Observations):
This is a particular interesting theorem, that shows us that linear speedups are always possible, the only limitation is our hardware, i.e. how many alphabets, or how big could n be, such that making the strings in Σ^n as an alphabet still makes sense! Note that usually, we use 2^{32} as a unit to store information, and better advancement of hardware make these computations faster and faster. However, though, the speedup can be at most a linear speedup. That is, we cannot make some algorithm of $O(n^3)$ into $O(n^2)$. This also makes sense in real life. It is interesting to see how the linear speedup theorem relates to real life examples.

9 Week 9

9.1 Lesson 29. Time Hierarchy Theorem

Note:
Is it true that $TIME(t) \subsetneq TIME(t')$ when $t \in o(t')$? That is, the inclusion is strict? Intuitively, running Turine Machines for longer should allow us to detect the class of strictly more number of languages. We will show whether this is true later.

One question is that, given some examples of time complexity function $t \in o(t')$, such that $TIME(t) = TIME(t')$.
One example is that $t = 1, t' = \sqrt{n}$, satisfy that $1 \in o(\sqrt{n})$ and that $TIME(1) = TIME(\sqrt{n})$.

9.1.1 Weak Time Hierarchy Theorem

Theorem (Weak Time Hierarchy Theorem):
$TIME(n) \subsetneq TIME(n^3)$

Proof:
We first show $TIME(n) \subsetneq TIME(n^3)$.
Consider the following turing machine T : that takes input $\langle M \rangle, x$, and

- First count $n = |x|$ in base 10, then compute n^3
- Then simulate x on M for n^3 steps. If accept, output "reject", otherwise, (reject or not halt), we output "accept".

Note that the machine T always halts.
Note that T runs in $O(n \log n)$. The extra factor of $\log(n)$ comes from the counting and the cubing. Now, suppose towards contradiction, there exists a time n machine, M such that $L(M) = L(T)$. That is, they decide the same language.
In other words, we have a machine G that decides $L(T)$ within $O(n)$ time. So that $L(T) = L(G)$. Consider the result feeding the string 1^k to G, where k is large enough such that G finishes execution on k within n^3 steps. This is possible since G decides within $O(n)$ time. Now, suppose G rejects 1^k, then M will actually accept 1^k, and vice versa. So it must be that $L(T) \neq L(G)$. This completes the proof.

$\square$

9.1.2 Time Hierarchy Theorem

Note that the full time hierarchy theorem does not apply to all the time complexity measures $t : \mathbb{N} \to \mathbb{N}$. It only applies to time-constructible measures.

Definition:
The function $t : \mathbb{N} \to \mathbb{N}$ is time-constructible if for every n, there is a turing machine that writes $t(n)$ ones on the tape in time $O(t(n))$.

Note:
Note that the functions $n, n\log n, n^2, 2^n$ are all time constructible. What is a function that is NOT time constructible?
One example is $\sqrt{n}$. Note that in $O(\sqrt{n})$ time, the TM cannot identify $|n|$, so it cannot identify the value of $\sqrt{n}$ at all.

Note 9.1 (My own questions and observations):
Note that every regular language is in $TIME(n)$. Also, every $CDFL$ is in $TIME(n)$, by the definition. However, what classes do CFLs belong to? I believe that $CFLs$ belong to $TIME(EXP(n))$, because every CFL is decidable by a language that enumerates all the possibilities of the CFL derivation, and match to see if it fits.
Now, decidable languages are a even bigger class. They correspond to the languages that encode problems that can be solved by any real-life computers. Note that this class is very large, and it consists of all of the time classes.

$$\bigcup_{t:\mathbb{N}\to\mathbb{N}} TIME(t(n))$$

This even includes $TIME(n!)$. Note that this does not include undecidable languages, because $t(n)$ could be equal to ∞ for some n, which is not upper bounded by any number in $\mathbb{N}$.

Theorem (Time Hierarchy Theorem):
For every time-constructible function $t : \mathbb{N} \to \mathbb{N}$,

$$TIME(t(n)) \subsetneq TIME(t(n)\log t(n))$$

Proof: We will show $TIME(t(n)) \subsetneq TIME(t(n)\log t(n))$.
Consider the following turing machine T : that takes input $\langle M\rangle, x$, and

- First count $n = |x|$ in base 10, then compute $t(n)$.
- Then simulate x on M for n^3 steps. If accept, output "reject", otherwise, (reject or not halt), we output "accept".

Note that the machine T always halts.
Note that T runs in $O(t(n)\log t(n))$. We must have the function being time constructible, otherwise it won't be able to count $t(n)$ in time $O(t(n))$, and to make sure we have run for exactly $t(n)$ steps. (each count takes 1 time). The extra factor of $\log(n)$ comes from the counting and the computing of $t(n)$. Now, suppose towards contradiction, there exists a time n machine, M such that $L(M) = L(T)$. That is, they decide the same language.
In other words, we have a machine G that decides $L(T)$ within $O(t(n))$ time. So that $L(T) = L(G)$. Consider the result feeding the string 1^k to G, where k is large enough such that G finishes execution on k within $O(t(n)log(t(n)))$ steps. This is possible since G decides within $O(t(n))$ time. Now, suppose G rejects 1^k, then M will actually accept 1^k, and vice versa. So it must be that $L(T) \neq L(G)$. This completes the proof.

$\square$

9.1.3 Necessity of Time Constructibility

Theorem:
There exists non-time constructible functions $t : \mathbb{N} \to \mathbb{N}$ with $t \in \omega(n)$ for which

$$TIME(t(n)) = TIME(t(n) \log t(n))$$

Proof:
Note that the number of functions $t : \mathbb{N} \to \mathbb{N}$ are uncountable. (consider the set of functions n^x for exponents $x > 1$.)
However, there are only countable number of Turing machine, so there must be uncountable number of non-time constructible functions.
Let $t(n)$ be one of those non-time constructible functions that is in $\omega(n)$. Consider $h(n) = EXP(t(n))$, and $h(n) \log h(n) = EXP(t(n))t(n)$. We might be able to find a time complexity class such that $TIME(t(n)) = TIME(t(n) \log t(n))$, and this is defnitely not time constructible.

□

Note 9.2 (My thoughts and observations):
Note that is the restriction $t \in \omega(n)$ is dropped, we can solve the above question with $t(n) = \sqrt{n}$. In this case, $TIME(\sqrt{n}) = TIME(1)$ and $TIME(\sqrt{n} \log \sqrt{n}) = TIME(1)$, because both are in $o(n)$. Therefore, we can use this fact to show both are in the same time complexity class. I was wondering if I can use the above example to solve the question for $t \in \omega(n)$. This is a particularly difficult proof.

9.2 Lesson 30. P

Definition 9.1:
A polynomial time Turing machine is a TM with time cost $O(n^k)$ for some $k \geq 1$. The class P is the set of all languages that can be decided by some polynomial time Turing machine.
We also define

$$P = \bigcup_{k \geq 0} TIME(n^k)$$

Note:
Are the following definitions of P equivalent to the one above?

- $P = \bigcup_{k \geq 1} TIME(n^k)$ Yes, the $TIME(n)$ includes $TIME(1)$.
- $P = \bigcup_{k \geq 2} TIME(n^k)$ Yes, $TIME(n^2)$ includes $TIME(n), TIME(1)$.
- $P = \bigcup_{0 \leq k \leq C} TIME(n^k)$ for some very large constant C? No, this does not include $n^{\lfloor C+1 \rfloor}$

9.2.1 Motivations for studying the class P

There are three main motivations for studying this class.

Note:

- Strength. The statement concluding a language not in P is quite strong. If we study the class $TIME(n^2)$, concluding a language not in this class, it could be in $TIME(n^k)$ for bigger k. Showing a language is not in P concludes it is not in $TIME(n^k)$ for any k.
- Closure. P is closed under subroutine calls. That is, if you have a polynomial time Turing Machine, that uses polynomial time blackbox algorithms as subroutines, the time complexity will still be polynomial. This is especially important when we work with polynomial time reductions. That is, we can use these reductions to show that if A reduces to B, and if B is in P, A must also be in P.
- Robustness. The class P is certainly a robust class of language, there are so many computational problems whose time corresponds to the class P. Also all reasonable models of computation, the set of languages that can be decided in polynomial time corresponds exactly to P.

Theorem (Cobham-Edmonds Thesis):
Any decision problem that can be solved in polynomial time by an algorithm on any physically-realizable computer corresponds to a language that is in P.

Note that quantum computers might disprove the Cobham-Edmonds Thesis.

Note 9.3 (My Own questions and observations):

- Does a general version of Cobham-Edmonds Thesis exist? Why is the class P special?
- Is the following statement true?

  Any decision problem that can be solved in **exponential** time by an algorithm on any physically-realizable computer corresponds to a language that is in $EXP = \bigcup_{k \geq 0} EXP(k)$?
- Here is a paper by Alam Cobham:

  `https://www.cs.toronto.edu/~sacook/homepage/cobham_intrinsic.pdf`

  In this paper, the author mainly talked about the Cobham-Edmonds Thesis regarding polynomials. The author used turing machines as a measure to how complicated the problems are, which relates back to the idea of P vs NP in this lesson.

9.2.2 First Observations

Proposition:
Every regular language is in P.

Proof: Let L be a regular language. We have shown that $L \in TIME(n)$, because given a DFA D, we can construct a TM that simulates D. So we need exactly $|x|$ steps to decide the string x and $L \in TIME(n)$.$\square$

Proposition:
The language $L_{0^n1^n} = \{0^n1^n \mid n \geq 0\}$ and $L_{0^n1^n2^n} = \{0^n1^n2^n \mid n \geq 0\}$ are in P.

Proof: I will use the Cobham-Edmonds thesis. I consider a 2 or 3 tape Turing machine physically realizable. Note that we can decide both of the above languages in $O(n)$ time using the multi tape machine. (i.e. copying the string on all the tapes, have the tapehead pointing to the start of $0s$, $1s, 2s$, and move right simultaneously to decide).
Therefore by the thesis, these languages are in P. $\square$

Definition (Complexity classes E and EXP):

$$E = \bigcup_{k \geq 0} TIME(2^{k \cdot n})$$

$$EXP = \bigcup_{k \geq 0} TIME(2^{n^k})$$

Theorem:

$$P \subsetneq E \subsetneq EXP$$

Proof: Consider the time-hierarchy theorem.
We know that $P \subseteq E$ because for any k, $n^k \in O(2^n)$ (CS341?).
We know that $E \subseteq EXP$ because for any k, $2^{kn} \in 2^{n^2}$.
However, both of the subset relations are not strict.
Note that $TIME(2^n) \in E$, but $TIME(2^n) \notin P$ because $\forall k \in \mathbb{N}, 2^n \notin O(n^k)$. Similarly, $TIME(2^{n^2}) \in EXP$, but $TIME(2^{n^2}) \notin TIME(2^{kn})$ for any k.

$\square$

Note 9.4 (Observations and my own questions):
Do time classes correspond to languages?
I noticed that if L is a language with a finite possibility of n-prefix, then $L \in TIME(1)$. If $L \in RL \implies L \in TIME(n)$. If $L \in CFL \implies L \in TIME(EXP(n))$.
It is interesting that in this diagram

As we go to the bigger circles, we have more time classes.
However, is the reverse true? Does $L \in TIME(1) \implies L$ share the same prefix?
Does $L \in TIME(n)$ imply that L is regular? I think maybe this is true, but I am not sure how to prove it.

10 Week 10

10.1 Lesson 31. NP

10.1.1 Definitions

Definition 10.1 (NP):
The class NP of languages is the class that can be decided in nondeterministic polynomial time.

Note 10.1 (Question of my own):
Are recognizable but not decidable languages also in NP? In this case, we would consider recognizability with nondeterministic TMs. That is, for $x \in L$ at least one computational path accepts, and for $x \notin L$, all paths do not accept.
Note that the TIME and NTIME classes are only decided upon languages that are decidable. What will happen if we extend this notion to recognizable language? I think we cannot define TIME class for recognizable languages. This is because, if there is a function $t : \mathbb{N} \to \mathbb{N}$ such that the TM recognizes this language within time $O(t)$, otherwise it reject. Then, this language would actually be decidable! So recognizabile but not decidable languages must have some strings such that the TM will not halt. Therefore, it would be impossible to define the TIME class for recognizable language.
However, we can define Space complexity for recognizable language! This is because even though sometimes the TM will never halt, the space it uses can be upper bounded. That is, the TM just cycles among n cells, where $n \in \mathbb{N}$. So it is possible for a language such that TM never halts on (so never in any time class) but it uses a $O(f)$ space for some function $f : \mathbb{N} \to \mathbb{N}$. I have no idea how to classify this kind of language. But it is surely interesting to think about.

Definition 10.2 (Time cost, Worst time cost):
The time cost of the $NTMM$ on input x is the maximum number of transposition it follows before it halts, over all computational paths. Note that a NTM decides a language implies that it must halt on all inputs.
The worst-case time cost of a NTM is the function $t : \mathbb{N} \to \mathbb{N}$ where $t(n)$ is the maximum cost of M on any input x of length $|x| = n$. Note that this is the longest computational path along all the inputs of a fixed length.

Definition 10.3 (NTIME class):
For every function $t : \mathbb{N} \to \mathbb{N}$, the nondeterministic time complexity class $NTIME(t)$ is the set of all languages that can be decided by a NMTM (nondeterministic multitape Turing machine), with the worst-case time cost bounded above by $O(t)$.

Definition 10.4 (NP):

$$NP = \bigcup_{k \geq 0} NTIME(n^k)$$

Note 10.2 (My observations):
Consider the class NP. In CS341, the way we defined it is that the class of computational problems with a polynomial time verifier. However, the definition of NP in this chapter seems very remote from what the CS341 taught.
I can see how these two concepts relate to eachother. That is, if there is a polynomial time verifier, it would have to be the accepting path that the NTM took. This way, we can verify that it indeed leads to an accepting path in under polynomial time.
So does the problem of NP originally stem from the idea of "nondeterministic turing machine" or "polynomial time verifier"?
After reading the article `https://www.britannica.com/science/P-versus-NP-problem`, I think it stemmed from the latter. It is interesting how some modern cryptography problem relies on the assumption that factorins (which is in NP) is not in P.

Recall the $NTIME$ and $TIME$ refers to having a corresponding TM with determininistic/ nondeterministic TM. Other than that, all the other components of the definition are actually equal.

10.1.2 Polynomial Time Verifiers

Definition 10.5 (Verifier, certificate tape):
A verifier is a Turing machine with an additional certificate tape. (So it has two tapes) The input to a verifier is a pair (w, c). The input w is written on the main tape, and the certificate string c is written on the certificate tape.

Definition 10.6 (Verifier):
Note that the execution of verifier follows the same rule as multitape Turing Machine when doing operations on a string. However, they decide / recognize strings differently.
The language recognized by verifier V is

$$L(V) = \{w \in \Sigma^* : \exists c, \text{ such that } V \text{ accepts } w, c\}$$

The verifier V decides $L(V)$ if it always halts on all input pairs (w, c).
A polynomial-time verifier is a verifier with time cost $O(n^k)$ for some constant $k \geq 0$.

The connection between polynomial time verifiers and the class NP is as follows:

Theorem 10.3:
A language L is in NP if and only if it can be decided by a polynomial-time verifier.

Proof:
$\underline{\Longrightarrow}$
Suppose $L \in NP$. So $L \in NTIME(n^k)$ for some $k \geq 0$. This implies there exists a Nondeterministic Multitape TM that decides L in worst time cost bounded above my $O(n^k)$. Since the NTM N decides L in worst time $O(n^k)$, it can write down the states travelled in the NTM as a verifier.
The verifier takes (w, c) and follow the states in c while reading w. If it passes, then c is a certificate for w.

Also, for any $w \in L(V)$, we know there must exist some c that serves as a certificate of the NTM deciding w. Therefore, there exist a polynomial time verifier.

$\underline{\Longleftarrow}$

Suppose that L can be decided by a polynomial-time verifier V with time cost $O(n^k)$. Consider the TM, given w, run it on V with (w, c) for all possible strings $c \in \Sigma^k$. This NTM takes at most $O(n^k)$ time. So the language L is in NP.

$\square$

10.1.3 P vs NP

It is clear that $P \subseteq NP$, but the question of whether $P = NP$ still remains open today. The implication if this is true, then there are problems in many areas of CS such that they are solvable with poly-time algorithms if and only if $P = NP$.

Note 10.4:
Justify that $NP \subseteq EXP$.

An interesting philosophical interpretation of $P\ VS\ NP$ is that, is it as easy to find a solution to a problem as to verify it? The intuitive answer should be no, but it has not been proven yet.

10.2 Lesson 32. NP-Completeness

10.2.1 Polynomial-Time reductions

Definition 10.7 (Polynomial time reduction):
Given two languages $A, B \subseteq \Sigma^*$, the language A is polynomial time reducible to B, denoted

$$A \leq_P B$$

if there is a function $f : \Sigma^* \to \Sigma^*$ such that

1. For every $x \in \Sigma^*$, we have $x \in A \iff f(x) \in B$
2. There is a polynomial-time Turing machine M that on input $x \in \Sigma^*$, erases it and replace it with $f(x)$ on the tape and then halts.

Lemma 10.5:
For every two languages A, B that satisfy $A \leq_P B$,

- If $B \in P$ then $A \in P$ as well.
- If $B \in NP$ then $A \in NP$ as well.

Proof:
$\underline{B \in P \text{ implies } A \in P}$

Suppose $B \in P$. This means there is a polynomial algorithm to compute whether $x \in B$ on a Deterministic TM. Let $x \in \Sigma^*$. Compute $f(x)$, which can be done in polynomial time. Then feed $f(x)$ into the TM that decides B. If it pass, then $x \in A$, otherwise $x \notin A$. What was described is a poly-time algorithm to decide whether $x \in A$.

<u>$B \in NP$ implies $A \in NP$</u>

Suppose $B \in NP$. This means that there exists a NMTM, that decides B in polynomial time. Let $x \in \Sigma^*$. Compute $f(x)$, which can be done in polynomial time. Then feed $f(x)$ into the NTM that decides B. If it pass, then $x \in A$, otherwise $x \notin A$. What was described is a poly-time algorithm to decide whether $x \in A$ using a NMTM. □

10.2.2 NP-hardness and completeness

Definition 10.8:
The language L is in $NP-$hard if every language $A \in NP$ satisfies $A \leq_P L$.

Note that it is possible to have a language L being NP-hard without beinng in the class NP.

Definition 10.9 (NP complete):
The language L is $NP-$complete if

1. $L \in NP$ and
2. L is $NP-$hard.

Is it possible that there exist a language that is both $NP-$complete and in P? We don't know. The proof or dis-proof of the statement would resolve the P vs NP problem.

Proposition 10.6:
If NP hard language L is in P, then $P = NP$.

Proof:
We know $P \subseteq NP$. It suffices to show $NP \subseteq P$. Let $A \in NP$ be arbitrary. We know that $A \leq_P L$. By the previous lemma, $A \in P$. So $NP \subseteq P$. This completes the proof. □

10.2.3 Bonus- Existence of an NP-complete language

Theorem 10.7:
There exists an NP-complete language.

10.3 Lesson 33. Satisfiability

The Cook-Levin theorem shows something really strong: a simple, and natural language is also NP-complete.

10.3.1 Boolean formulas and SAT

Definition 10.10:

- A boolean variable is a variable that can take on two possible values: True or False.
- A literal is a boolean variable x or its negation $\overline{x}$.
- A clause is the OR (or disjunction) or one or more literals.
- A boolean formula in CNF form is an AND (or conjunction) of one or more clauses.
- A boolena formula is satisfiable if there exists some assignemnt of True or False values to its underlying Boolean variables, such that it causes the formula to evaluate to true.

Note 10.8:

- Is the boolean formula in CNF, is it satisfiable?

$$\phi(x_1, x_2, x_3) = x_1 \wedge (\overline{x_1} \vee x_2) \wedge (\overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee \overline{x_2} \vee \overline{x_3})$$

  It is in CNF but not satisfiable.

- What is the maximum number of distinct clauses that a Boolean Formula on n variables can have? If we ignore duplicates, but accept $x \vee \overline{x}$, it would be $2^{2n} - 1$.

Definition 10.11:
The satisfiability language is

$$SAT = \{\langle \phi : \rangle \mid \phi \text{ is a satisfiable formula}\}$$

We will show that SAT is NP complete. We begin by showing that the language is indeed in NP.

Lemma 10.9:

$$SAT \in NP$$

Proof: We will show that it has a polynomial time verifier.
Note that $x \in SAT$ with k variables, $\iff$ there exists an assignment $\{T, F\}^k$ such that it satisfy the assignment. This happens if and only if we have c of length k, encoding the assignment. We can run (w, c) through V, which is a polynomial time deterministic TM that verifies this assignment in polynomial time.$\square$

10.3.2 Tebleaux

Definition 10.12:

- A tableau is a sequence of configurations of a Turing machine.
- A tableau is valid if the sequence corresponds exactly to the configuration of a Turing Machine starting from its initial configuration all the way to the configuration where the machine halts.
- A valid tableau is accepting when the Turing machine is in an accepting state in its last configuration.
- The name "tableau" comes from the fact that we can represent one using a table, with each row representing a configuration.
- This is an example

q_0	0	1	0	$\square$	$\square$
0	q_1	1	0	$\square$	$\square$
0	1	q_0	0	$\square$	$\square$
0	1	0	q_1	$\square$	$\square$
0	1	0	$\square$	q_{rej}	$\square$

- The height of a valid tableau is the number of rows of the table that represents it. This corresponds to the number of configuration of the TM from the start of its execution to the moment that it halts. This also corresponds to the time cost of the TM.
- The width of a valid tableau is the length of the strings that represents the configuration. For convenience, we add blank symbols ($\square$) to represent empty tape cells so that the configurations all have same length and line up in the example above.

Note 10.10 (Observations and questions):
The width represents the space cost of the Turing machine. How about the size of the entire table? That would represent the width time height, which is the product of the space and time cost. Note that the height is bounded above by time cost, and width is bounded above by space cost.
Now I have one interesting questions:

- Given an algorithm. For example merge sort. Its time complexity of $O(n \log n)$ and its space complexity is $O(n)$. Can we implement a TM that represents this data? i.e. height being $O(n \log n)$ and width being $O(n)$. (I think yes by the Cobham-Edmonds Thesis).
- Suppose given a TM, let x_1 be the string that it takes the longest time to decide. Let x_2 be the string that it takes the longest space to decide. How are x_1, x_2 related? We would expect that $x_2 = x_1$ because the longest space implies the largest amount of time.

10.3.3 Tableaux and SAT

To prove that SAT is NP-complete, it suffices to show that every language $L \in NP$, and for every string w, we can construct in polynomial time a boolean formula ϕ_w that is satisfiable if and only if $w \in L$.

Note 10.11:
Consider the follow challenge

- Complete the proof that $SAT \in NP$-complete, assuming that the construction above is already completed.
- How can you use tableaux to obtain the construction of ϕ_w, for a given language $L \in NP$?

10.4 Lesson 34. Cook-Levin Theorem

10.4.1 Proof overview

Let L be any language in NP. We want to describe a poly-time algorithm that for every string w construct a Boolean formula ϕ_w that is satisfiable if and only if $w \in L$.
Specifically, we let $L \in NP$ and let M be a NTM that decides L in poly-time. Then $w \in L$ if and only if there is an accepting Tableau for M on input w.
For a tableau T of size $h \times w$, let us define the Boolean variables:

$$x_{i,j,\sigma} = \begin{cases} True & \text{if } T_{i,j} = \sigma \\ False & \text{otherwise} \end{cases}$$

For each $i \leq h, j \leq w, \sigma \in Q \times \Gamma$.
We now will construct ϕ_w so that it is satisfied if and only if Boolean variables $x_{i,j,\sigma}$ represents an accepting tableau for M on input w. We can do this by introducing constraints to enforce the different conditions that the Boolean variables must satisfy to represent such a tableau.

10.4.2 Cell constraints

The key to prove the Cook-Levin theorem is to break up the diffrent types of conditions we need to enforce individual formulas, which we will combine at the end of the proof.
First, we ensure the boolean variables $x_{i,j,\sigma}$ really encode the tableau. Note each cell (i, j) of the tableau contains exactly one symbol.

Lemma 10.12:
For each cell (i, j), there is a Boolean formula $\phi_{i,j}$ that is satisfied if and only if there exist exactly one symbol σ for which $x_{i,j,\sigma}$ is true.
Furthermore, there is a Boolean formula ϕ_{cell} that is satisfied if and only if the condition above is true for all cells (i, j) in the tableau.

Proof: Each cell corresponds to some i, j. We define i, j this way:

$$\phi_{i,j} = \left(\bigvee_{\sigma \in Q \cup \Gamma} x_{i,j,\sigma} \right) \bigwedge_{\sigma_1 \neq \sigma_2 \in Q \cup \Gamma} (\overline{x_{i,j,\sigma_1} \wedge x_{i,j,\sigma_2}})$$

That is, there is some σ such that i, j is filled with σ. But there cannot exist any two different σ on the spot.

To write this in CNF form,

$$\phi_{i,j} = \left(\bigvee_{\sigma \in Q \cup \Gamma} x_{i,j,\sigma} \right) \bigwedge_{\sigma_1 \neq \sigma_2 \in Q \cup \Gamma} (\overline{x_{i,j,\sigma_1}} \vee \overline{x_{i,j,\sigma_2}})$$

Now, we want

$$\phi_{cell} = \bigwedge \phi_{i,j} \text{ is true}$$

if and only if there is a valid cell satisfaction.

□

10.4.3 Initial and Final Constraints

Boolean variables that satisfy ϕ_{cell} encodes symbols that we can write as tableau. Our next step is to verify this tableau indeed corresponds to configuration of N when it is run on w.

Lemma 10.13:
There is a boolean formula $\phi_{start,w}$ that is satisfied by some Boolean variables that satisfy ϕ_{code} if and only if the first row of the tableau encoded in Boolean variables corresponds to the initial configuration of M on input w.

Proof:
Let $w = w_1 w_2 \dots w_n$. We want to make sure the first row of tableau corresponds to some initial configuration

$$\underbrace{\square, \dots, \square}_{\text{at most } h \text{ squares}}, q_0, w_1, \dots, w_n, \underbrace{\square, \dots, \square}_{\text{at most } h \text{ squares}}$$

So $\phi_{start,w}$ is defined as

$$\phi_{start,w} = \bigwedge (x_{1,1,\square}, \dots, x_{1,h,\square}, x_{1,h+1,q_0}, x_{1,h+2,w_1}, x_{1,h+3,w_1}, \dots x_{1,h+n+2,w_n}, x_{1,h+n+3,\square} \dots, x_{1,2h+n+2,\square})$$

□

We also present a lemma to verify the last configuration in the tableau is an accepting configuration.

Lemma 10.14:
There is a Boolean formula ϕ_{acc} that is satisfied by some Boolean variables that satisfy ϕ_{code} if and only if the last row of the tableau encoded in the Boolean variables corresponds ot an accepting configuration.

Proof: All we need to do is to make sure there is q_{acc} in the last row, represented by h. Note that this is indeed all we have to do, because the validity of the last row is given by induction, where first row is valid,

and the valid boolean formula will make sure if row i is valid, then so it $i+1$. This proves the lemma.

$$\bigvee_{1 \leq j \leq 2h+n+2} x_{h,j,q_{acc}}$$

□

10.4.4 Valid Tableau Constraints

We only need to verify the last condition that the tableau is valid, that is the consecutive configurations actually can be obtained by following a single transition in M.
The key observation is to check a local 2×3 window of the tableau, and checking every such window allow us to check the validity of the whole tableau.

Lemma 10.15:
For each cell (i, j), there is a boolean formula ϕ_{valid} that is satisfied by some Boolean variables, which satisfy ϕ_{code} if and only if they encode a valid tableau for M.

Proof:
Consider the nine possible types of 2×3 windows:
This one is when there are no changes

a_1	a_2	a_3
a_1	a_2	a_3

This one is when the transition is to the left, that is $\delta(q, a_3) = (q', b, L)$, and there are four different possible types of 2×3 windows

a_0	a_1	a_2	q	a_3	a_4
a_0	a_1	q'	a_2	b	a_4

Now, similarly, there is one when the transition is to the right, that is $\delta(q, a_2) = (q', b, R)$, and there are also four different possible types of 2×3 windows

a_0	a_1	q	a_2	a_3	a_4
a_0	a_1	b	q'	a_3	a_4

Note that there are at most $(|\Sigma| + |Q|)^{6 \cdot 9}$ possibilities of those 2×3 windows. We name them $\bigcup_{i \in W, j=1,2,3,4,5,6} w_{i,j}$, where W are the set of windows and each j represents the spot of the window. This is poly time number of phrases with respect to the Tableau.
So for each top left cell $x_{i,j}$ we need to have

$$\phi_{\text{window}(i,j)} = \bigvee_{w \in \text{window}} (x_{i,j,w_1} \wedge x_{i,j+1,w_2} \wedge x_{i,j+2,w_3} \wedge x_{i+1,j,w_4} \wedge x_{i+1,j+1,w_5} \wedge x_{i+1,j+2,w_6})$$

□

and we must have

$$\phi_{valid} = \bigwedge_{i,j \text{ a top left cell}} \phi_{\text{window}(i,j)}$$

Note 10.16 (My Own Ideas):
Why did we consider the local 2×3 windows, not the local $2 \times w$ windows? Also, why 3 instead of other constants?
I think 2 for row number makes sense because we rely on the invariant that any transition between two configuration is valid. As long as we have this information, it is enough.
Now, why not 2×2? Why not 2×4 or more rows? I think 2×2 does not capture enough information to verify the transition. But I think 2×4 would be unnecessarily many. Why not consider every $2 \times w$? For 2×3, there are at most $(|\Sigma \cup Q|)^6$ different windows. However, with $2 \times w$, the number of different windows cannot be bound by a certain $O(n^k)$. So the number of CNFs is not bounded by some $O(n^k)$. So we must have fixed window size.

10.4.5 Putting it together

Theorem 10.17 (Cook-Levin Theorem):
The language SAT is $NP-$complete.

Proof: Now we take the boolean formulas ϕ_{cell} and ϕ_{code}.

$$\phi_{code,w} = \phi_{cell} \wedge \phi_{valid} \wedge \phi_{acc} \wedge \phi_{start,w}$$

To show a langauge is NP complete, we must show that it is in NP and that it is NP hard.
SAT is in NP, as shown earlier in the lemma.
We will show SAT is in NP hard. It suffices to show every language $A \in NP$, $A \leq_P SAT$.
Given $A \in NP$, there exists a NMTM that decides A in poly time. Note that $A \leq_P SAT$ because given $x \in \Sigma^*$, we can encode it by running it through the NMTM, and construct the corresponding TM in poly time. Then, we can encode the rules as ϕ in SAT. Note that $x \in A$ if and only if $\phi \in SAT$, that is, there is a satisfiable tableau. This proves that SAT is in NP hard.

$\square$

11 Week 11

11.1 Lesson 35 Beyond P and NP

11.1.1 coNP

Definition:

$$coNP = \{L \subseteq \Sigma^* : \overline{L} \in NP\}$$

We would ask, is NP closed under complementation? This question is equivalent to asking whether $NP = coNP$. This is still an open problem, and one possible answer to this problem would also resolve P vs NP.

Proposition 11.1:
If $NP \neq coNP$, then $P \neq NP$

Proof: We will show the contrapositive of the proof.
Suppose $P = NP$. Then $NP = coNP$. Because

- $L \in NP$ means $L \in P$. Since language in P is closed under complement (the DTM decides language in poly time), so $\overline{L} \in P = NP$. So $L \in coNP$.
- $L \in coNP$ means that $\overline{L} \in NP = P$. So $\overline{\overline{L}} = L$ is also in $NP = P$ as the decider can decide within poly time, and we output opposite answers. Hence $L \in P$.

□

11.1.2 $NP \cap coNP$

If $NP \neq coNP$, we ask whether the intersection of those two classes is P or not. Clearly, P is in both of the sets so P must be a subset of the intersection. $P \subseteq NP$ as shown before. $P \subseteq coNP$ because any $L \in P$, we can build a poly time verifier for it. However, this subset relation is actually strict.

Proposition:
The language

$$FACTOR = \{\langle M, N\rangle : M < N \in \mathbb{N} \text{ and } d \mid N \text{ for some } 1 < d < M\}$$

is in $NP \cap coNP$. Is it in P?

Proof:
We don't know if Factoring is in P. If so, we would have solved the P vs NP mystery.
It is in NP because it has a polynomial time verifier. (i.e. The verifier being d, q where $1 < d < M, dq = N, M < N$.)

It is in coNP because we can have a certificate of N's factorization. We check in poly time that all factors are indeed prime and add up to N, and all of which are at least M.

□

11.1.3 Ladner's Theorem

Theorem 11.2 (Ladner's Theorem):
If $P \neq NP$, there exist some languages in $NP \setminus P$ that are not NP-complete.

Proof:
The notes of the proof are taken from the original proof written by Ladner.
`http://oldblog.computationalcomplexity.org/media/ladner.pdf`
We start by assuming $P \neq NP$. From this assumption, we know that every language in NP complete will not be in P. We focus on SAT as such a language here.
Consider the following setup:

- We let $M_1, M_2, \ldots$ be an enumeration of DTMs such that $M_i(x)$ runs in time at most $|x|^i$. We let f_i be a list of polynomial time computable functions.
- We have two following conditions for a language A:
  1. $R_i : A \neq L(M_i)$
  2. S_i: there exists some string x, either ($x \in SAT$, $f_i(x) \notin A$) or ($f_i(x) \in A$ but $x \notin SAT$).
- the language A is in NP

So, the way we construct the language A requires a function f, which we will define later. We describe language A as

$$A = \{x, x \in SAT, f(|x|) \text{ is even}\}$$

To show A in NP, suffices to show $f(n)$ computation in poly time, with respect to $n = |x|$.
The intuition of defining the function f is follows:

1. On stage $2i$, we make $f(n) = 2i$ for large enough n until condition R_i is fulfilled, that is, large enough TM such that the language it corresponds to is not equal to A anymore. If R_i is never fulfilled, then $A = L(M_i)$ for all the large i. It would imply that $A \in L(M_i)$, which can tell SAT in poly time. This contradicts $P \neq NP$.
2. On stage $2i + 1$, keep $f(n) = 2i + 1$ until S_i is fulfilled. If S_i is never fulfilled, we have poly time $f(x) \in A$ if and only if $x \in SAT$, showing SAT would be in P. This again contracticts $P \neq NP$.

So for the above two stages, the process can go on forever. We consider the method called delayed diagnolization, where we donn't start a new stage until we can test that the condition to move to next stage can be tested in the new TM M_i, in poly time. We now formally define $f(n)$ inductively for $n \in \mathbb{N}$.

1. $f(0) = f(1) = 2$
2. For $n \geq 1$, we define $f(n + 1)$ as if $log^{f(n)} n \geq n$, then $f(n + 1) = n$. If not, we have two cases:
3. $f(n) = 2i$, we check if there is a string $x, |x| \leq \log n$ such that either
   (a) $M_i(x)$ accepts and either ($f(|x|)$ is odd or x not in SAT)
   (b) or $M_i(x)$ rejects and $f(|x|)$ is even and $x \in SAT$.

If such x exist, we let $f(n+1) = f(n) + 1$, otherwise $f(n+1) = n$. Essentially, this checks if it has fulfilled the conditions to jump into a new stage. If so, we update the stage, otherwise we let the stage stay same.

4. $f(n) = 2i + 1$, we check if there is a string $x, |x| \leq \log n$ such that either
   (a) x is in SAT and either $f(|f_i((x))|)$ is odd or $f_i(x)$ is not in SAT or
   (b) x is not in SAT and $f(|f_i((x))|)$ is even and $f_i(x)$ is in SAT.

   If such x exist, then $f(n+1) = f(n)+1$, otherwise $f(n+1) = f(n)$. This is again similar to setting a new stage if it has fulfilled one of the conditions R, S.

To conclude this proof, we observe that computing $f(n)$ only examine $|x| \leq \log n$. So x is polynomial time of n. We also have

$$|x|^i \leq \log^i(n) \leq \log^{f(n)} n < f(n)$$

so we can compute $f(n)$ in polynomial time of n. Note that $f(n)$ does not increase unless a requirement is fulfilled, and if f remains constant we (have shown before) will violate $P \neq NP$.
This shows that A is in NP, not in P. A is not NP complete, because $f(|x|)$ is poly time computable, then it misses part of SAT that have the power of being NP complete. □

11.1.4 Polynomial Hierarchy

We will define a $\forall\exists$ verifier.

Definition 11.1:
A $\forall\exists$ verifier be a Turing Machine V with two extra tapes, which includes certificates c_1, c_2 such that it recognizes the following language:

$$L(V) = \{w \in \Sigma^* : \forall c_1 \exists c_2 \text{ such that } V \text{ accepts } (w, c_1, c_2)\}$$

Definition 11.2:
Π_2^P is the class of all languages that can be decided by polynomial-time $\forall\exists$-verifiers.

Question: what is a language in Π_2^P that is not obviously in NP or in coNP as well?

Theorem 11.3:
If $P = NP$, then $P = \Pi_2^P$ as well.

Proof: Suppose $P = NP$ to begin with.
$\Pi_2^P \supseteq NP$
If $L \in NP$ there exists a poly verifier for L, call it V. Then, we make the $\forall\exists$ to take (w) that ignores the input c_1 and only sees if there exists c_2 such that $\forall\exists$ verifier accepts (w, c_1, c_2) where it checks whether V accepts c_2. This completes the proof.
$\Pi_2^P \subseteq NP$

Let $L \in \Pi_2^P$, so we have a polytime $\forall\exists$ verifier V. Consider the language

$$M = \{\langle w, c\rangle, w, c \in \Sigma^*, \exists d \in \sigma^*, V \text{ accepts } (w, c, d)\}$$

The class Π_2^P is not the only extension of $NP, coNP$. We can obtain an infinite class of complexity classes, known as the polynomial hierarchy. Note that $M \in NP = P$ because the verifier needs only c_2, and we can just modify V to accept w, c_1. By assumption there is a TM N that decides M. We know that $x \notin M$ if and only if N rejects $\langle x, c\rangle$ for some c. (That is, $x \notin M$ implies there does not exist d certificate. N rejects. On the other hand, N rejects some $\langle x, c\rangle$ implies there do not exist certificate, so it would reject). So $\overline{M} \in NP = P$ so M is in $coNP$. By our previous work, if $N = NP$ then $NP = coNP$ so we are done.

□

11.2 Lesson 36. SPACE complexity classes

Definition 11.3 (Space cost):
The space cost of the Turing machine M on input x is the total number of distinct tape cells that are visited by M's tape head before M halts.

Definition 11.4:
The worst case space cost of the Turing machine M is the function $s : \mathbb{N} \to \mathbb{N}$ where $s(n)$ is the maximum space cost of M on any input x of length $|x| = n$.

Note 11.4:
Consider the following questions:

- If M decides L, is its space cost always bounded for each input x?
- Show that $TIME(s) \subseteq SPACE(s)$. This is true because at each time step, it at most gets one new cell.

Definition 11.5:
For every function $s : \mathbb{N} \to \mathbb{N}$, the time complexity class $SPACE(s)$ is the set of all languages that can be decided by a multitape Turing Machine with worst case space cost bounded above by $O(s)$.

Note 11.5:
Some questions:

- What are some languages in $SPACE(n)$? Can you find some languages not known to be in P that are in this class?
- Let $SPACE_1(s)$ denote the set of languages that can be computed by single tape TM with space cost s. Does $SPACE_1(s) = SPACE(s)$ always hold?

11.2.1 Time and space

.
Time and space complexity classes are related to eachother. Note that for all $s : \mathbb{N} \to \mathbb{N}$, we have $TIME(s) \subseteq SPACE(s)$.

Theorem 11.6:
For every $s : \mathbb{N} \to \mathbb{N}$, we have

$$NTIME(s) \subseteq SPACE(s) \subseteq TIME(2^{O(s)})$$

Proof:
To show $NTIME(s) \subseteq SPACE(s)$, note that on any input string $x \in \Sigma^*$, if it is in $NTIME(s)$ that means there is a NTM that decides it in $s(x)$. Since the worst time cost is s, we know that the TM's tapehead will move to at most s different grids.
To show $SPACE(s) \subseteq TIME(2^{O(s)})$, let k be the minimum number such that $|\Gamma \cup \mathbb{Q}| \leq 2^k$. There are at most $(2^k)^{s(n)+1}$ configurations (assume greater than linear). Because at most $s(n)$ different tape constructions plus one tape head. Now, for contradiction, suppose there are more than $(2^k)^{s(n)+1}$ configurations. This implies that one configuration has appeared by more than once. This is impossible because this will loop forever, following this construction. (as it is deterministic.) So we must have at most this much configurations, and it will halt in polynomial time. □

11.3 Lesson 37. PSPACE

Definition 11.6:

$$PSPACE = \bigcup_{k \geq 1} SPACE(n^k)$$

Corollary 11.7:

$$P \subseteq NP \subseteq PSPACE \subseteq EXP$$

Proof:
$P \subseteq NP$ because it has been proven.
For every $n^k, k \in \mathbb{R}^+$, we have $NTIME(n^k) \subseteq SPACE(n^k)$.
So $NP = \bigcup_{k \geq 0} NTIME(n^k) = \bigcup_{k \geq 0} SPACE(n^k) = PSPACE$.
To show $PSPACE \subseteq EXP$, we have $PSPACE = \bigcup_{k \geq 0} SPACE(n^k) \subseteq \bigcup_{k \geq 0} TIME(2^{O(n^k)}) \subseteq EXP$. □

11.3.1 Savitch's Theorem

11.3.2 Nondeterministic Space complexity

Definition 11.7:
The space cost of the nondeterministic machine M on input x is maximum total number of distinct tape cells that are visited by $M's$ tapehead before M halts, maximum taken over all possible computational paths.
The worst case space cost of nondeterministic Turing machine M is the function $s : \mathbb{N} \to \mathbb{N}$ where $s(n)$ is maximum space cost of M on any input x of length $|x| = n$.

Definition 11.8:
For every function $s : \mathbb{N} \to \mathbb{N}$, the time complexity class NSPACE(s) is the set of all languages that can be decided by a multitape nondeterministic TM with worst-case space cost bounded above by $O(s)$.
We define

$$NPSPACE = \bigcup_{k \geq 1} SPACE(n^k)$$

11.3.3 The Derivation Language

We consider the language *Derive*, which includes all encodings $\langle N, x, c_1, c_2, t \rangle$ of a nondeterministic TUring machine N, an input x to N, two configurations c_1 and c_2 of N, and a value $t \in \mathbb{N}$ such that N can go from configuration c_1 to configuration c_2 by following at most t transitions.

Lemma 11.8:
There is a Turing machine that decides *Derive* and has space cost only $O(s \log t)$ on input $\langle N, x, c_1, c_2, t \rangle$ when N has space cost of $O(s)$ on input x.

Proof:
Consider a divide and conquer algorithm.
We can see this as a directed graph problem. Let the vertices be the $|Q \cup \Gamma|^{(|x|+t)}$ different configurations. There is a directed edge between $c \to d$ if and only if c transitions to d.
We are going to divide and conquer:

- def check(c_1, c_2, t):
- — if c_1, c_2 is adjacent configuration, return true.
- — for d is a configuration:
- — — t_1=check$(c_1, d, t/2)$
- — — t_2=check$(d, c_2, t/2)$
- — — if t_1 and t_2, return True
- — return False

This guarantees that there at at most $log(t)$ configurations on the stack at any time. Each configuration takes space at most $O(s)$. So this yields our space cost.

□

11.3.4 Proof of Savitch's Theorem

Theorem 11.9 (Savitch's theorem):
For every $s : \mathbb{N} \to \mathbb{N}$,

$$NSPACE(s(n)) \subseteq SPACE(s(n)^2)$$

Proof:
Consider the turing machine D that decides $Derive$ and has space cost $O(s \log t)$ on input $\langle N, x, c_1, c_2, t\rangle$.
Let N be a NTM such that it decides L in space $O(s(n) \log t)$.
Consider the algorithm that iterates through all possible turing configurations $(|Q \cup \Gamma|)^{len(n+s(n))}$ as c_2. There is only one c_1. So we are running $\langle N, x, c_1, c_2, t\rangle$ on only one c_1 and $(|Q \cup \Gamma|)^{len(n+s(n))}$ c_2s. Note that t is at most $(|Q \cup \Gamma|)^{len(n+s(n))}$. So $\log(t) = O(n + s(n)) = s(n)$.
So each iteration, the space cost is $O(s(n)s(n))$. Running it many times, the space cost is still the same as we can reuse space. Hence this completes the proof.

□

Corollary 11.10:

$$PSPACE = NPSPACE$$

Proof:

$$PSPACE = \bigcup_{k \geq 0} SPACE(n^k) \supseteq \bigcup_{k \geq 0} NSPACE(n^{k/2}) = NSPACE$$

□

11.4 Lesson 38. Logarithmic Space Complexity

11.4.1 Basic definitions

Definition 11.9:
An input/work tape Turing machine is a muultitape Turing machine whose input is on the first read-only tape and where the other tapes are standard read/write tapes.
Apart from the restriction that the first tape is read only, input/work tape Turing machines behave exactly as standard Turing machines.

How do we define the transition function of input/work tape Turing machines to ensure that first tape is

read only? To define it, any transition, in the input tape, the head will only move left and right without modifying any tape alphabets.
The space cost of an input/work tape Turing machine M on input x is the number of distinct cells in the work tapes that are accessed before M halts on input x.

Definition 11.10:
The space cost of an input/work tape Turing machine M is the minimal function $s : \mathbb{N} \to \mathbb{N}$ such that on every input of length n, the machine M scans at most $s(n)$ distinct cells in the work tapes.

11.4.2 Space complexity classes

Definition 11.11 (SPACE and NSPACE):
We now redefine out space complexity classes with redefined notion of space cost.
For every function $s : \mathbb{N} \to \mathbb{N}$,

- $SPACE(s)$ is the set of all languages that can be decided by a deterministic input/work tape Turing Machine with worst-case space cost bounded above by $O(s)$.
- $NSPACE(s)$ is the set of all languages that can be decided by a nondeterministic input/work tape Turing machine with worst-case space cost bouhded above by $O(s)$.

Question: Verify for every $s = \Omega(n)$, the classes $SPACE(s)$ and $NSPACE(s)$ in the definition above are identical to the ones introduced in the earlier lessons.
The extra elements on the work tape corresponds to the cells visited by the tapehead in the old definition. If we have $\Omega(n)$, we guarantee that tapehead must visit all tape cells of the input. (Similar to how TIME(o(n)) is always constant.)

11.4.3 L and NL

Definition 11.12:

$$L = SPACE(\log n) \text{ and } NL = NSPACE(\log n)$$

The classes L and NL are the set of languages that can be decided efficiently in in both time complexity and space complexity.

Proposition 11.11:

$$L \subseteq NL \subseteq P \subseteq NP$$

Proof:
$L \subseteq NL$ is true because for any language ℓ such that we can decide within L space, we can make a NTM corresponding to it such that it also decides within L space. The NTM would be the exact same.
To show $NL \subseteq P$.
Let $\ell \in NL$ be a language that can be determined by a NTM with space cost $\log(n)$.

There exist a constant c such that the space cost is at most $c\log(n)$. Trying all possibilities (implementing backtracking) takes exponential time, which is $EXP(c\log n)$ so it is in $O(n^k)$ time hence in P.

$\square$

12 Week 12

12.1 Lesson 39. NL Completeness

Definition 12.1 (Log-space reducible):
Given two languages A and $B \in \Sigma^*$, the langauge A is log-space reducible to B, denoted:

$$A \leq_L B$$

if there is a function $f : \Sigma^* \to \Sigma^*$ such that

1. For every $x \in \Sigma^*$, we have $x \in A \iff f(x) \in B$
2. There is a logarithmic-space Turing machine M that on input $x \in \Sigma^*$, erases it and replaces it with $f(x)$ on the tape and then halts.

This is an extra tape, not work tape. Work tape is still small, but the output tape can be big. Have the writing tape only to the right, but worktape can go anywhere.

Note 12.1:
Does $A \leq_P B$ imply $A \leq_L B$?
Does $A \leq_L B$ imply $A \leq_P B$?

Lemma 12.2:
For every two languages A, B satisfy $A \leq_L B$,

- If $B \in L$, then $A \in L$ as well
- If $B \in NL$, then $A \in NL$ as well.

Proof:
<u>$B \in L$ implies $A \in L$</u>

Suppose $B \in L$. This means there is a deterministic input/work tape TM with worst case space cost bounded above by $O(\log(n))$. Let $x \in \Sigma^*$. Compute $f(x)$, which can be done in log space. Then feed $f(x)$ into the TM that decides B. If it pass, then $x \in A$, otherwise $x \notin A$. What was described is a log space TM to decide whether $x \in A$.

<u>$B \in NL$ implies $A \in NL$</u>

Suppose $B \in L$. This means there is a non-deterministic input/work tape TM with worst case space cost bounded above by $O(\log(n))$. Let $x \in \Sigma^*$. Compute $f(x)$, which can be done in log space. Then feed $f(x)$ into the NTM that decides B. If it pass, then $x \in A$, otherwise $x \notin A$. What was described is a log space NTM to decide whether $x \in A$.

$\square$

12.1.1 NL completeness

Definition 12.2 (NL-complete):
The language L is NL-complete if

1. Every language $A \in NL$ satisfies $A \leq_L L$
2. $L \in NL$

By the last lemma, in order to show $L = NL$, it suffices to show any NL complete language is in L.

12.1.2 Connectivity

Definition 12.3 (CONN):
The (s,t)-connectivity problem asks: given a directed graph $G = (V, E)$ and two vertices $s, t \in V$, does there exist a path from s to t in G? The corresponding language is:

$$CONN = \{\langle G, s, t\rangle : \text{there is a path s to t in G}\}$$

Lemma 12.3:
$CONN \in NL$

Proof:
Make sure the NTM that you use decides CONN in every possible computational path. That is, it always halts.
Note that the path length, if there exists path from s to t, cannot exceed $|V|$. We will use a NTM that continually guess a next vertex, and store the current length of the path, and current vertex. (the storage for length is $O(\log(n))$ and the current vertex is $O(1)$.)
This NTM will terminate whenever the length is greater than $|V|$ without success, and terminate with accept whenever it finds path s to t. Note that the NTM does not store the path, because we don't need the path, we only need to know whether there exists a path. Again, it always halts because it runs for at most $|V|$ transitions. □

Theorem 12.4:
CONN is NL-complete.

Proof: Let M be an NTM that decides a language in NL. We first assume that M accepts on an empty working tape. (We can do so because in the automata, we can modify accepting state such that they each clear working tape before the final accepting state.) This, way, we assume there is only one accepting and one starting configuration.
Now, we put all the possible configuration of the NTM on the input x, with length n, into a directed graph G, where the edges represent transitions in the NTM. We let s denote the starting state, and t denote the accepting state, each on input x.
Note that M uses at most $O(\log(n))$ space on its working tape, there are at most $|\Gamma|^{\log(n)} \in Poly(n)$

configurations. So still, CONN can be solved with $\log(k)$ where k is the input to CONN. (i.e. log of $poly(n)$ is still $O(\log(n))$).
So, $x \in L(M) \iff \langle G, s, t\rangle \in CONN$. Therefore, $CONN$ is $NL-$complete. $\square$

12.2 Lesson 40. Immerman-Szelepcsenyi Theorem

Definition 12.4 (coNL):

$$coNL = \{L \subseteq \Sigma^* : \overline{L} \in L\}$$

Note 12.5:

1. Why does $L = NL$ imply $NL = coNL$?
2. Why does $NL \neq coNL$ imply $L = NL$?
3. In fact, opposite is the case, we have $NL = coNL$, but this result does not solve the L vs NL question.

Theorem 12.6 (Immerman-Szelepscenyi Theorem):

$$NL = coNL$$

We will prove this theorem later in this lesson.
We will show that there exists an $NL-$complete langauge that is contained in $coNL$. That is, the complement of some $NL-$complete langauge is also in NL.

Note 12.7:
Justify the following answers:

- Why does showing that some NL-complete language L belongs to $coNL$ suffices to show $NL = coNL$?
- Why does showing $\overline{L} \in NL$ for some $NL-$complete language L suffice to show same identity?

We will consider the connectivity language.

Lemma 12.8:
There is a nondeterministic Turing machine with logarithmic space cost that, given $\langle G, s, t, l, k\rangle$ where

- G is a directed graph
- s, t are vertices in G
- $\ell \in \mathbb{N}$ is a length parameter
- k denotes the number of vertices reachable from s by paths of length at most l in G.

has an accepting path if and only if there is a path from s to t of length at most l in G.

Proof:
Given input $\langle G, s, t, l, k\rangle$ assuming that k indeed denotes the right number of vertices reachable from s in G. The only thing that we need to check is that NTM has accepting path if and only if there is a path from s to t of length at most l in G.
For each $v \in G - s$, we can use NTM to check whether there exist a path s to v of length at most l in G, by guessing all the paths in NTM. (i.e. once longer than l, reject, and accept if has a path). Similarly to the previous lemma, we only need to write down v and $\log(n)$, which is indeed in log space.
If t is accepted, then accept. Otherwise, as soon as the number of passing v reach k, reject. Number of passing v will always reach k under the assumption that k is correct. □

Theorem 12.9:

$$\overline{CONN} \in NL$$

Proof:
Since we can accomplish the lemma with $log(n)$ space, it suffices to show that given $\langle G, s, t\rangle$, we have all the $l = 0, 1, 2, \ldots |V|$, $\langle G, s, t, l, k_l\rangle$ all reject, where k_l is the corresponding k for the l. This will show that the string is in $\overline{CONN}$.
The last thing we need to do is to compute correct k_l for each l.
We do so by induction. For $l = 0, k_l = 1$, because it would be the vertex s. Now, suppose we have the value k_{l-1} with $l - 1$. To compute k_l, we iterate through $v \in V$,. If $\langle G, s, v, l - 1, k_{l-1}\rangle$, we also count it. Given v, iterate through edges, if there is at least one edge uv such that $\langle G, s, u, l - 1, k_{l-1}\rangle$ returned true, then we also count it. After counting through all $v \in V$, we obtain a correct number for k_l. This concludes the proof. □

12.3 Lesson 41

12.3.1 PSPACE-completeness

Definition 12.5:
The language L is PSPACE-Complete if

- Every $A \in PSPACE$ satisfy

$$A \leq_P L$$

- $L \in PSPACE$

Similar to how NPcomplete is outside of P if $P \neq NP$, PSPACE-complete language is outside of P if $P \neq PSPACE$.

Proposition 12.10:
If L is PSPACE-complete and $P \subsetneq PSAPCE$, then $L \notin P$.

Proof:
We opt for contradiction. Suppose L is PSPACE-complete and $P \subsetneq PSPACE$. If $L \in P$, then every languages $A \in PSPACE$, we have $A \leq_P L$, so A is in P. So $PSPACE \subseteq P \subsetneq PSPACE$. This is a contradiction.

□

Definition 12.6:
We will define PSPACE-complete by considering the totally quantified Boolean formulae (t.q.b.f.). These are called Boolean formulae along with $\forall$ and $\exists$ quantifiers for each variable. For example

$$\phi(x, y, z) = \forall x \exists y \forall z : x \wedge (\overline{x} \vee y) \wedge (\overline{y} \vee z)$$

is a t.q.b.f.
Each t.q.b.f. always evaluates to True or False. The TQBF langauge is this:

$$TQBF = \{\langle \phi \rangle : \phi \text{ is a true t.q.b.f.}\}$$

Theorem 12.11:
TQBF is PSPACE-complete.

Proof: Proof is for future tasks. □

Note that whether $TQBF \in P$ (or not in P) is an open problem.

12.3.2 Small space complexity classes

We now ask about very small space complexity classes. The smallest complexity class is $SPACE(1)$ which contains all langauges that can be decided by a TM only using a constant amount of (space) space.

Theorem 12.12:
The class $SPACE(1)$ is the set of regular languages.

Proof:
Regular language is in $SPACE(1)$. It takes $SPACE(1)$ to encode the regular language. The working tape only needs to remember the current state and the symbol that the cursor is pointing to.
Now, we need to show $SPACE(1)$ is the set of regular languages.
Since it is space of 1, there exists some c such that the TM only needs c space. So there are c^k possible configurations of the work tape and at most k ($k = |\Sigma|$). For each configuration, there is only one valid choice. so we can construct this DFA. □

12.3.3 Time and Space

So far, we considered time and space complexity sparately. We can combine them and consider what if a TM can be decided with small time cost AND small space cost.

Definition 12.7:
The class SC is the class of langauge that can be decided by a DTM with time cost $O(n^k)$ and space cost $O(\log^l(n))$ for some parameters $k, l \in \mathbb{N}$.

Definition 12.8:

$$polyL = \bigcup_{\ell \geq 1} SPACE(\log^l n)$$

Proposition 12.13:

$$SC \subseteq P \cap polyL$$

Proof:
Given language $A \in SC$. Then L is decided by a DTM with time cost $O(n^k)$ and space cost $O(\log^l(n))$. So it is in both P (time wise) and $polyL$ (space wise) by definition. □